

ACP-137
INTRODUCCIÓN A IOT

MANUAL DE PRÁCTICAS:
**ESP32 Y
ASISTENTES
DE VOZ**



UNIVERSIDAD AUTÓNOMA DEL
ESTADO DE QUINTANA ROO

M.M. Jesús Orifiel Alvarez Ruiz
Dr. Freddy Ignacio Chan Puc
Dr. Homero Toral Cruz

ÍNDICE

INTRODUCCIÓN	1
PRÁCTICA I - CONEXIÓN CON ARDUINO IOT CLOUD	2
PRÁCTICA II – ARDUINO IOT CLOUD CON ALEXA	33
PRÁCTICA III – SINRIC CON ALEXA	48
PRÁCTICA IV – SINRIC CON GOOGLE HOME	68
PRÁCTICA V – ESP RAINMAKER CON ALEXA	80
PRÁCTICA VI – ESP RAINMAKER CON GOOGLE HOME.....	103

ÍNDICE DE ILUSTRACIONES

ILUSTRACIÓN 1.1 – ARDUINO IOT CLOUD.....	3
ILUSTRACIÓN 1.2 – DIAGRAMA A IMPLEMENTAR	4
ILUSTRACIÓN 1.3 – PANTALLA DE BIENVENIDA DE IOT CLOUD	5
ILUSTRACIÓN 1.4 – PÁGINA PARA LA CREACIÓN DE UNA CUENTA GRATUITA.	5
ILUSTRACIÓN 1.5 – PANTALLA DE CONFIGURACIÓN PARA DISPOSITIVOS IOT.	6
ILUSTRACIÓN 1.6 – CREACIÓN DE UNA “COSA”.	6
ILUSTRACIÓN 1.7 – ASIGNACIÓN DE NOMBRE A NUESTRA “THING”.	7
ILUSTRACIÓN 1.8 – ASIGNAMOS VARIABLES.	7
ILUSTRACIÓN 1.9 – CREAMOS LA VARIABLE PARA LA TEMPERATURA.....	8
ILUSTRACIÓN 1.10 – CREAMOS LA VARIABLE PARA LA HUMEDAD.....	9
ILUSTRACIÓN 1.11 – CREAMOS LA VARIABLE PARA EL ENCENDIDO Y APAGADO DEL LED.....	10
ILUSTRACIÓN 1.12 – VARIABLES CREADAS EN IOT CLOUD.	11
ILUSTRACIÓN 1.13 – PUERTO DE COMUNICACIONES ASIGNADO POR EL SISTEMA OPERATIVO.	11
ILUSTRACIÓN 1.14 – VINCULACIÓN DE LA TARJETA ESP32 CON ARDUINO IOT CLOUD.....	12
ILUSTRACIÓN 1.15 – SELECCIÓN DE LA MARCA DE LA TARJETA DE DESARROLLO.	12
ILUSTRACIÓN 1.16 – SELECCIÓN DEL MODELO DE LA TARJETA DE DESARROLLO.	13
ILUSTRACIÓN 1.17 – ESP32 WROOM.....	13
ILUSTRACIÓN 1.18 – SE GENERA EL ID Y LA CLAVE SECRETA.....	14
ILUSTRACIÓN 1.19 – MENÚ PARA LA CONFIGURACIÓN DE LA RED WIFI.....	15
ILUSTRACIÓN 1.20 – VENTANA DE CONFIGURACIÓN DE LA RED WIFI.	15
ILUSTRACIÓN 1.21 –PESTAÑA DE SKETCH EN IOT CLOUD.....	16
ILUSTRACIÓN 1.22 –INSTALACIÓN DEL AGENTE DE IOT CLOUD.....	16
ILUSTRACIÓN 1.23 –OPCIÓN PARA TRABAJAR CON EL CÓDIGO COMPLETO.	17

ILUSTRACIÓN 1.24 – ARDUINO CLOUD EDITOR.....	18
ILUSTRACIÓN 1.25 –SELECCIÓN DE LA TARJETA DE DESARROLLO Y EL PUERTO DE COMUNICACIONES.	18
ILUSTRACIÓN 1.26 – LIBRERÍAS REQUERIDAS PARA EL SENSOR DHT11.	19
ILUSTRACIÓN 1.27 –LIBRERÍAS DHT11.....	20
ILUSTRACIÓN 1.28 –CARGANDO EL CÓDIGO A LA TARJETA ESP32.....	22
ILUSTRACIÓN 1.29 –TARJETA ESP32 CONECTADA A LA NUBE.	23
ILUSTRACIÓN 1.30 –SELECCIÓN DEL DASHBOARD.	24
ILUSTRACIÓN 1.31 – MENÚ PRINCIPAL DEL DASHBOARD.	24
ILUSTRACIÓN 1.32 – PANEL DE DISEÑO DEL DASHBOARD.....	25
ILUSTRACIÓN 1.33 –WIDGETS.....	25
ILUSTRACIÓN 1.34 –WIDGET GAUGE.....	26
ILUSTRACIÓN 1.35 –WIDGET RDUINOJE.	27
ILUSTRACIÓN 1.36 – WIDGET SWITCH.....	28
ILUSTRACIÓN 1.37 – DASHBOARD IOT.	28
ILUSTRACIÓN 1.38 – MODO MONITOREO.	29
ILUSTRACIÓN 1.39 – INSTALACIÓN DE LA APP “ARDUINO IOT CLOUD”.	29
ILUSTRACIÓN 1.40 – LOG-IN EN LA APLICACIÓN.	30
ILUSTRACIÓN 1.41 – DASHBOARD EN LA APLICACIÓN MÓVIL.....	30
ILUSTRACIÓN 1.42 – MONITOREANDO EL SISTEMA IOT MEDIANTE LA APLICACIÓN MÓVIL.....	31
ILUSTRACIÓN 2.1 – ARDUINO IOT CLOUD.....	34
ILUSTRACIÓN 2.2 – DIAGRAMA A IMPLEMENTAR	35
ILUSTRACIÓN 2.3 – MENÚ DEL APARTADO “THINGS”	36
ILUSTRACIÓN 2.4 – ADICIÓN DE UNA VARIABLE	36
ILUSTRACIÓN 2.5 – ACTIVAR COMPATIBILIDAD CON ALEXA	37
ILUSTRACIÓN 2.6 – CONFIGURACIÓN FINAL DEL LED	37
ILUSTRACIÓN 2.7 – ARDUINO CLOUD.....	38
ILUSTRACIÓN 2.8 – VINCULAR NUESTRO LED CON EL SWITCH	40
ILUSTRACIÓN 2.9 – AMAZON ALEXA	40
ILUSTRACIÓN 2.10 – APARTADO DE “MAS”	41
ILUSTRACIÓN 2.11 – SKILL DE ARDUINO.....	42
ILUSTRACIÓN 2.12 – ACTIVAR LA SKILL DE ARDUINO.....	43
ILUSTRACIÓN 2.13 – INICIO DE SESION CON LA SKILL	44
ILUSTRACIÓN 2.14 – ALEXA BUSCARÁ NUESTRO DISPOSITIVO AUTOMÁTICAMENTE	45

ILUSTRACIÓN 2.15 – VINCULACIÓN EXITOSA DE NUESTRO LED.....	46
ILUSTRACIÓN 3.1 – SINRIC PRO.....	49
ILUSTRACIÓN 3.2 – DIAGRAMA A IMPLEMENTAR	51
ILUSTRACIÓN 3.3 – REGISTRO EN SINRIC.....	52
ILUSTRACIÓN 3.4 – AÑADIR UN DISPOSITIVO.....	52
ILUSTRACIÓN 3.5 – CONFIGURACIÓN DEL DISPOSITIVO.....	53
ILUSTRACIÓN 3.6 – “SALVAR” LA CONFIGURACIÓN.	53
ILUSTRACIÓN 3.7 – CREDENCIALES DEL DISPOSITIVO.	54
ILUSTRACIÓN 3.8 – SUBIR EL CODIGO A NUESTRA ESP32.....	57
ILUSTRACIÓN 3.9 – DISPOSITIVO EN LINEA.....	58
ILUSTRACIÓN 3.10 – APP DE SINRIC PRO.....	59
ILUSTRACIÓN 3.11 – APLICACIÓN DE SINRIC PRO	60
ILUSTRACIÓN 3.12 – ENCENDIDO Y APAGADO DE LED MEDIANTE LA APP.....	61
ILUSTRACIÓN 3.13 – APARTADO MÁS.....	62
ILUSTRACION 3.14 – SKILLS Y JUEGOS	63
ILUSTRACIÓN 3.15 – ELEGIR LA SKILL DE SINRIC PRO	64
ILUSTRACIÓN 3.16 – ACTIVAR LA SKILL.....	64
ILUSTRACIÓN 3.17 – EXITOSA VINCULACIÓN CON SINRIC PRO.....	65
ILUSTRACIÓN 3.18 – ALEXA DETECTARÁ NUESTROS DISPOSITIVOS VINCULADOS A SINRIC PRO	65
ILUSTRACIÓN 3.19 – ENCENDIDO O APAGADO DEL LED.....	66
ILUSTRACIÓN 4.1 – SINRIC PRO.....	69
ILUSTRACIÓN 4.2 – DIAGRAMA A IMPLEMENTAR	71
ILUSTRACIÓN 4.3 – DISPOSITIVO EN LINEA.....	72
ILUSTRACIÓN 4.4 – APP DE GOOGLE HOME.....	73
ILUSTRACIÓN 4.5 – SELECCIONAMOS EL SERVICIO “FUNCIONA CON GOOGLE”	73
ILUSTRACIÓN 4.6 – SINRIC PRO.....	74
ILUSTRACIÓN 4.7 – VINCULAR SINRIC PRO CON GOOGLE HOME.....	74
ILUSTRACIÓN 4.8 – LED CORRECTAMENTE AGREGADO	75
ILUSTRACIÓN 4.9 – VENTANA DEL LED	76
ILUSTRACIÓN 4.10 – VENTANA DE CONFIGURACIÓN	77
ILUSTRACIÓN 4.11 – ENCENDER EL LED MEDIANTE EL ASISTENTE DE VOZ.....	78
ILUSTRACIÓN 5.1 – ESP RAINMAKER.....	81
ILUSTRACIÓN 5.2 – DIAGRAMA A IMPLEMENTAR	83

ILUSTRACIÓN 5.3 – APP DE ESP RAINMAKER	92
ILUSTRACIÓN 5.4 – REGISTRO EN LA APLICACIÓN.....	92
ILUSTRACIÓN 5.5 – SUBIDA DE CÓDIGO EXITOSA	93
ILUSTRACIÓN 5.6 – MONITOR SERIAL.....	93
ILUSTRACIÓN 5.7 – CONFIGURACIONES PARA EL MONITOR SERIAL	93
ILUSTRACIÓN 5.8 – BOTON IO0	93
ILUSTRACIÓN 5.9 – BOTON PRESIONADO CORRECTAMENTE	94
ILUSTRACIÓN 5.10 – ENLACE GENERADO	94
ILUSTRACIÓN 5.11 – CÓDIGO QR	95
ILUSTRACIÓN 5.12 – SELECCIONAR EL ÍCONO “+”	95
ILUSTRACIÓN 5.13 – PERMISOS DE LA APP.....	96
ILUSTRACIÓN 5.14 – SELECCIONAR NUESTRA RED	96
ILUSTRACIÓN 5.15 – VINCULACIÓN EXITOSA.....	97
ILUSTRACIÓN 5.16 – DISPOSITIVOS AÑADIDOS Y EN LINEA.....	97
ILUSTRACIÓN 5.17 – “SWITCHES” DESDE LA APLICACIÓN	98
ILUSTRACIÓN 5.18 – VOICE SERVICES	99
ILUSTRACIÓN 5.19 – AMAZON ALEXA	99
ILUSTRACIÓN 5.20 – VINCULACIÓN CON AMAZON ALEXA	100
ILUSTRACIÓN 5.21 – DISPOSITIVOS AGREGADOS CORRECTAMENTE	100
ILUSTRACIÓN 5.22 – DISPOSITIVOS DISPONIBLES	101
ILUSTRACIÓN 6.1 – ESP RAINMAKER.....	104
ILUSTRACIÓN 6.2 – DIAGRAMA A IMPLEMENTAR	106
ILUSTRACIÓN 6.3 – SELECCIONAR “GOOGLE ASSISTANT”	115
ILUSTRACIÓN 6.4 – PASOS A SEGUIR.....	115
ILUSTRACIÓN 6.5 – FUNCIONA CON GOOGLE	116
ILUSTRACIÓN 6.6 – SELECCIONAR ESP RAINMAKER.....	117
ILUSTRACIÓN 6.7 – INICIAR SESION.....	117
ILUSTRACIÓN 6.8 – DISPOSITIVOS AÑADIDOS CON ÉXITO	118
ILUSTRACIÓN 6.9 – INTERRUPTOR VIRTUAL	118

INTRODUCCIÓN

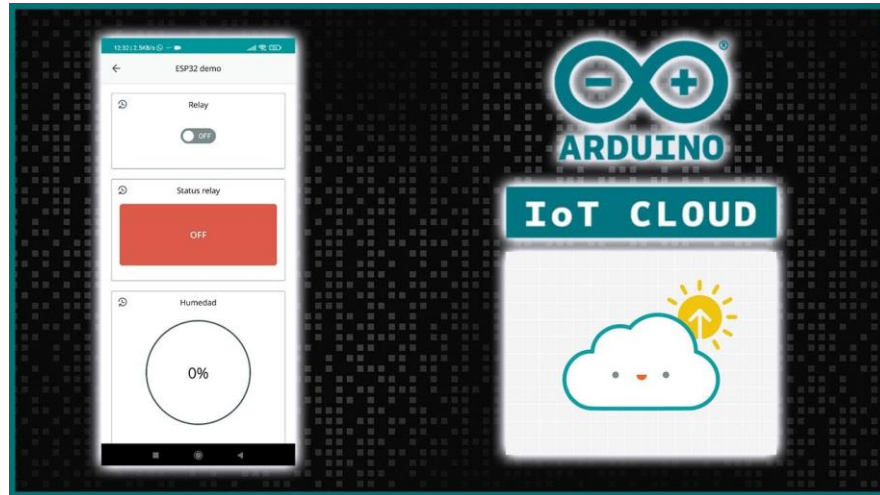
En este documento se presenta el “Manual de Prácticas de Laboratorio” de la asignatura “ACP-137-Introducción a IoT” denominado “ESP32 y Asistentes de Voz”. Las prácticas están basadas en el microcontrolador ESP32, el cual permite conexiones a WiFi y Bluetooth, facilitando la realización de aplicaciones relacionados con el IoT y la automatización. Además, en este documento se busca que el alumnado aprenda a programar el microcontrolador y configurar las aplicaciones necesarias para permitir el enlace con asistentes de Voz como: Alexa y Google Assistant.

Este manual de prácticas integra seis prácticas de laboratorio, con la finalidad de incrementar el aprendizaje teórico del estudiante, llevándolo a realizar diferentes prácticas de control de cargas activadas a través de asistentes de Voz. Con lo anterior se busca que el estudiante implemente aplicaciones de IoT con las tecnologías que están a la vanguardia.

De esta forma, la práctica 1 consiste en el monitoreo de variables a través del Dashboard de la plataforma de IoT Cloud, una vez realizado esto la práctica 2 se da seguimiento, sin embargo, ahora la plataforma IoT Cloud se configurará para poder enlazarse con la aplicación de Amazon “Alexa” y poder activar cargas a través de comandos de Voz. La práctica 3 y 4 nos muestra otras plataformas alternativas como lo es Sinric para conectarse con las IA de voz, la primera con Alexa y la segunda con Google Assistant. La práctica 5 y 6 usa una aplicación desarrollada por los mismos fabricantes de la ESP32 “Spresif System” denominada ESP RainMaker de igual manera para la conexión con Alexa y Google Assistant.

Este documento tiene la finalidad que este material sea de utilidad en la formación profesional de los alumnos del programa educativo de ingeniería en sistemas de energía e Ingeniería en Redes.

PRÁCTICA I - CONEXIÓN CON ARDUINO IOT CLOUD



OBJETIVO DE LA UNIDAD:

Conocerá los estándares de comunicación más utilizados en sistemas de IoT para implementar una aplicación funcional.

OBJETIVOS DE LA PRACTICA:

Conectar prototipos implementados con la tarjeta de desarrollo ESP32 y conectarlos a la nube mediante la aplicación web y móvil de IoT Cloud.

ANTECEDENTES

Si se requiere conectar nuestro Arduino o alguna de las tarjetas de desarrollo más comunes a Internet, para enviar información, una de las maneras más sencillas es empleando el IoT y la plataforma online Arduino IoT Cloud.

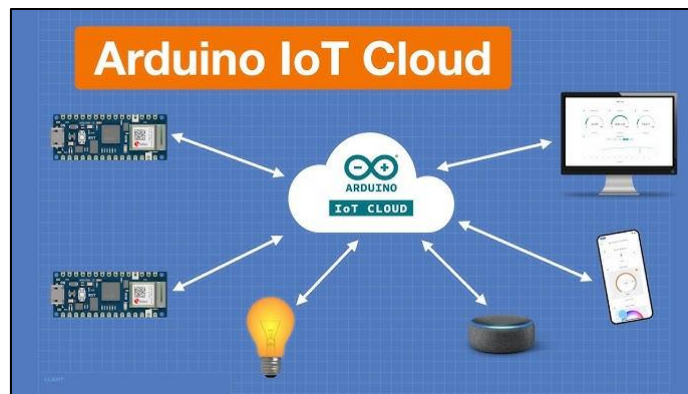


Ilustración 1.1 – Arduino IoT Cloud

Los usos que se le pueden dar a esta plataforma son:

- **Monitorización** de datos: es posible monitorizar fácilmente los valores de los sensores de Arduino a través de un panel de control.
- **Sincronización** de variables: la sincronización de variables nos permite sincronizar variables entre dispositivos, permitiendo la comunicación entre dispositivos con una codificación mínima.
- **Programación**: podemos programar trabajos para que se activen/desactiven diferentes funciones durante una cantidad de tiempo específica (segundos, minutos, horas).
- **Cargas Over-The-Air (OTA)**: permite cargar nuestro código a dispositivos no conectados al ordenador.
- **Webhooks**: integra tu proyecto con otro servicio, como IFTTT.
- **Soporte de Amazon Alexa**: permite que nuestro proyecto sea controlado por voz con la integración de Amazon.
- **Dashboard Sharing**: comparte tus datos con otras personas de todo el mundo

MATERIAL A UTILIZAR

- NodeMCU-32
- LED de cualquier color
- Jumpers Macho-Macho
- Resistencia de 220Ω
- Resistencia de 330Ω
- Sensor de temperatura y Humedad DHT11
- Protoboard

SOFTWARE NECESARIO

IoT Cloud: <https://cloud.arduino.cc/>

DESARROLLO DE LA PRÁCTICA

Práctica 1. Encendido de un led y monitoreo de temperatura y humedad a través de la plataforma IoT Cloud.

1.1 Implementar el circuito del diagrama que se muestra en la *Ilustración 1.2*

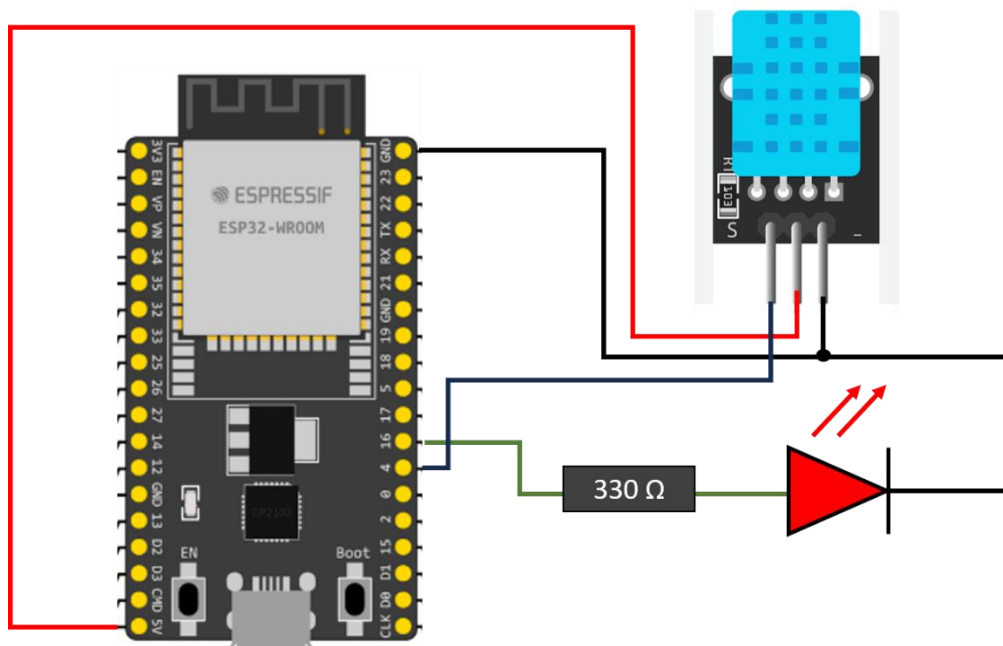


Ilustración 1.2 – Diagrama a implementar

1.2 Configuración y programación de la plataforma IoT Cloud

El primer paso es crear una cuenta en la plataforma de IoT Cloud, para esto dirigirse al sitio web <https://cloud.arduino.cc/>, y se abrirá la página de bienvenida mostrada en la *ilustración 1.3*.

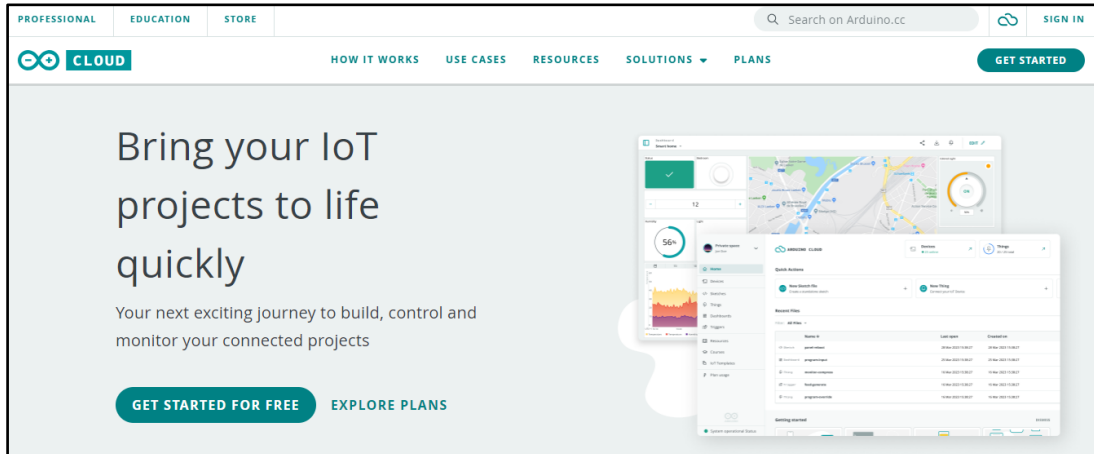


Ilustración 1.3 – Pantalla de bienvenida de IoT Cloud

Nos dirigimos a la opción de **“GET STARTED FOR FREE”** y nos dirigirá a la página de la *ilustración 1.4*.

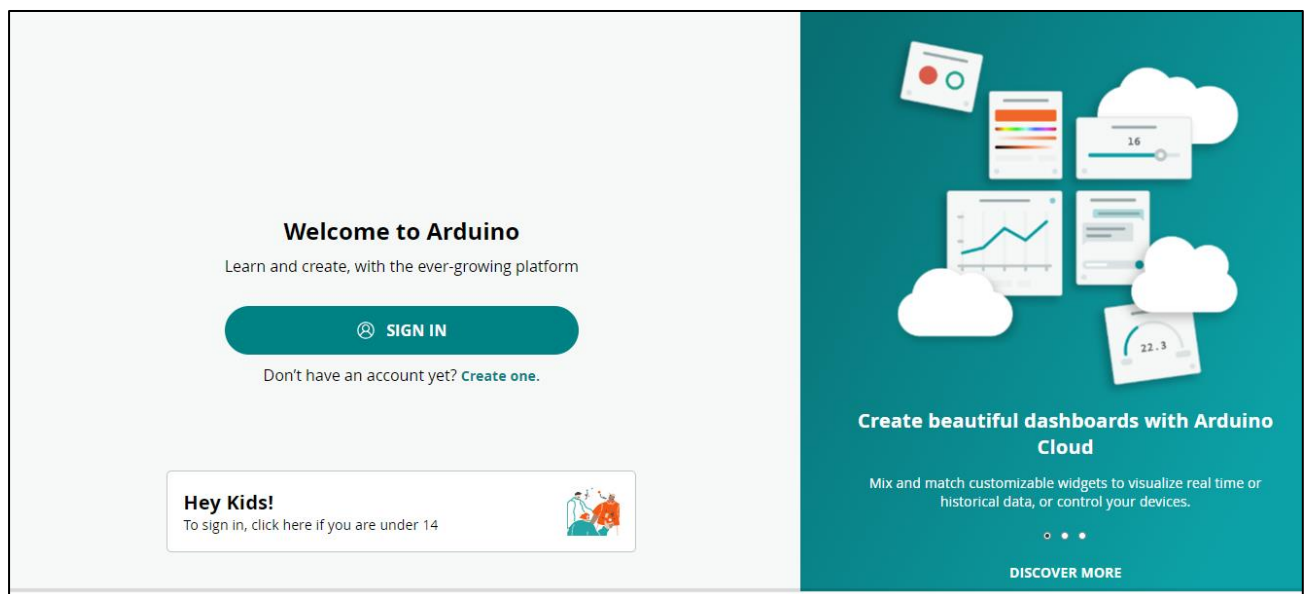


Ilustración 1.4 – Página para la creación de una cuenta gratuita.

Crea una cuenta siguiendo los pasos que se te solicitan. Una vez creada tu cuenta ingresa y te aparecerá la pantalla de la *ilustración 1.5*.

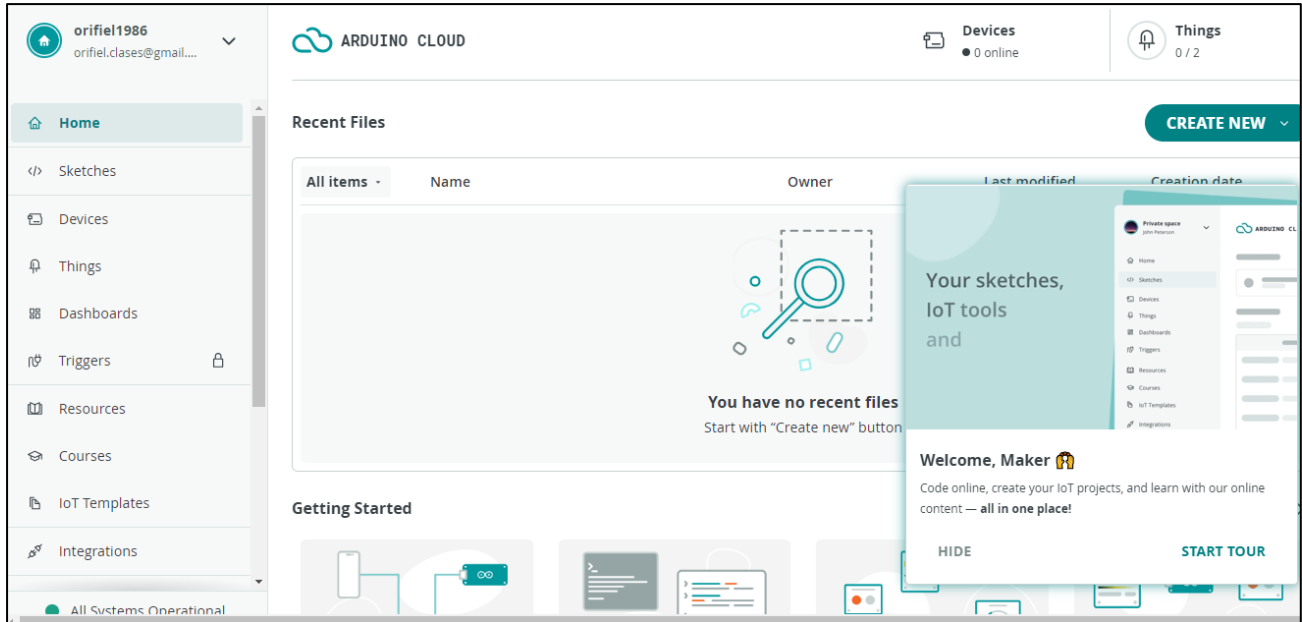


Ilustración 1.5 – Pantalla de configuración para dispositivos IoT.

Del lado izquierdo de la pantalla seleccionamos **“Things”** Y posteriormente **“Create a Thing”**, como muestra la *ilustración 1.6*.

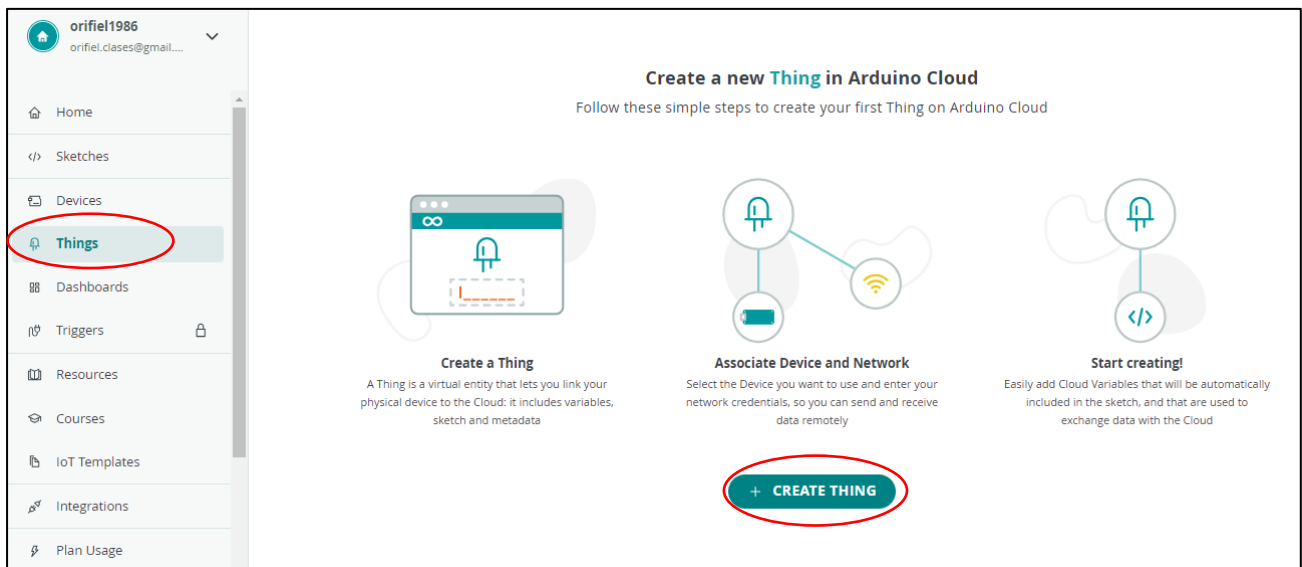


Ilustración 1.6 – Creación de una “cosa”.

Ponemos un nombre a nuestro objeto o cosa en la pestaña que se muestra en la *ilustración 1.7*.

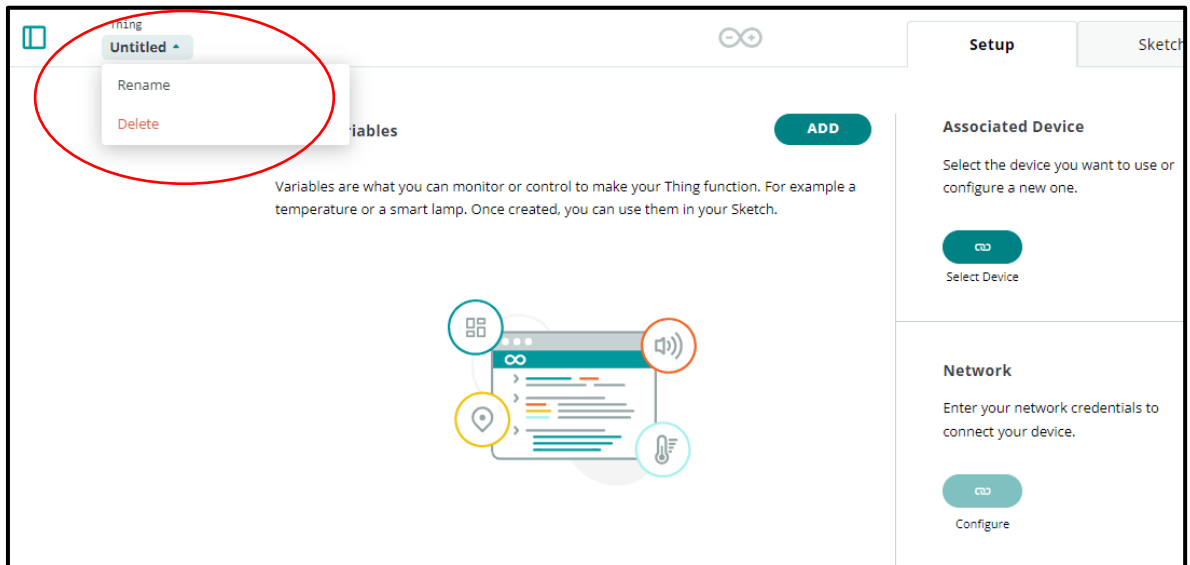


Ilustración 1.7 – Asignación de nombre a nuestra “Thing”.

Una vez realizado esto agregamos nuestras variables, en el botón de **“ADD”**, *ilustración 1.8*.

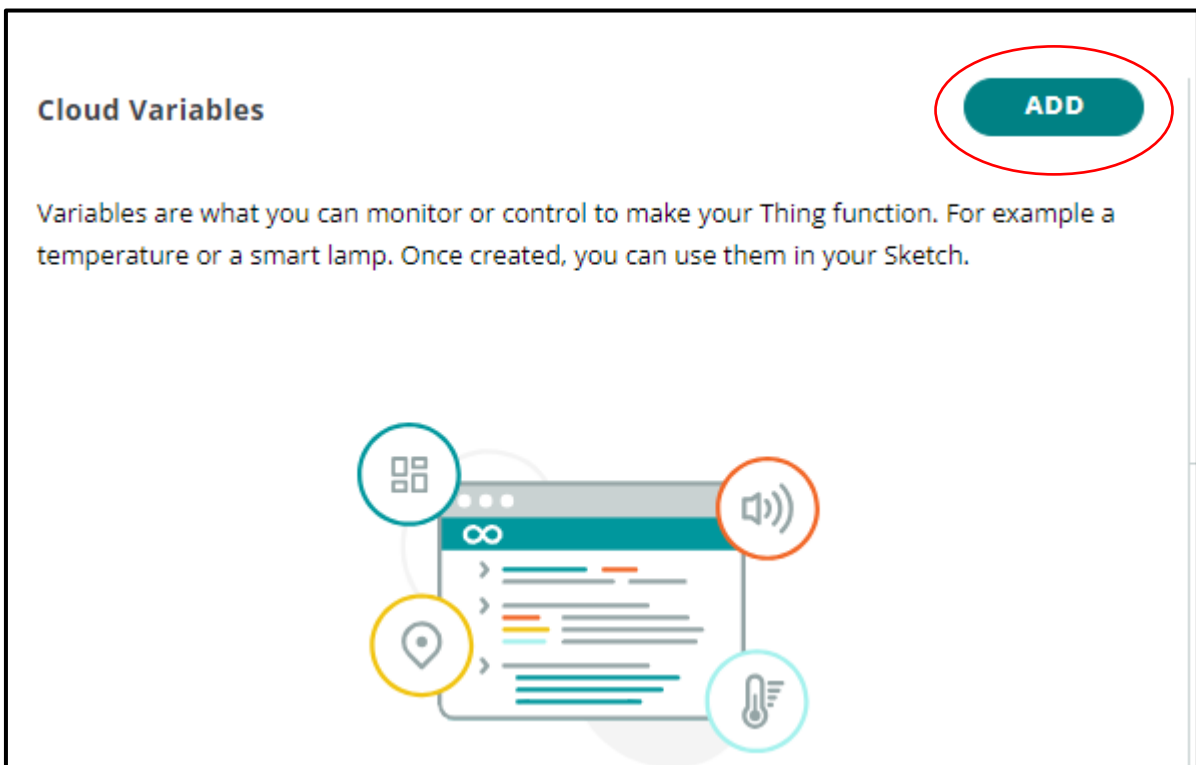


Ilustración 1.8 – Asignamos variables.

Cuando hacemos clic en **“ADD”** se abrirá una ventana que deberás llenar con los datos que se muestran en la *ilustración 1.9*.

Observa que se elige la opción **“Read Only”** porque, solamente leemos los datos del sensor DHT11, sin necesidad de escribir. La variable es de tipo flotante porque admite punto decimal. La opción **“On change”**, sirve para que solamente se actualice el valor si hay un cambio en la variable. Finalizamos en la opción **“ADD VARIABLE”**.

The screenshot shows a modal window titled "Add variable" with a close button (X) in the top right corner. The window contains the following elements:

- Name:** A text input field containing "dht11_temperatura".
- Sync with other Things:** A button with a circular refresh icon and an information icon (i).
- Type:** A dropdown menu showing "Floating Point Number eg. 1.55".
- Declaration:** A text input field containing the code `float dht11_temperatura;` with an information icon (i) to its right.
- Variable Permission:** Two radio buttons: "Read & Write" (unselected) and "Read Only" (selected).
- Variable Update Policy:** Two radio buttons: "On change" (selected) and "Periodically" (unselected).
- Buttons:** At the bottom, there are two buttons: "CANCEL" and "ADD VARIABLE".

Ilustración 1.9 – Creamos la variable para la temperatura.

Del mismo modo agregamos una nueva variable para la Humedad. Configuramos la ventana como se muestra en la *ilustración 1.10*.

The screenshot shows a dialog box titled "Edit variable". It contains the following elements:

- A text input field labeled "Name" containing the text "dht11_temperatura".
- A button with a circular arrow icon and the text "Sync with other Things", followed by a small information icon (i).
- A dropdown menu showing "Floating Point Number" with a small example "eg. 1.55" and a downward arrow.
- A text input field labeled "Declaration" containing the code "float dht11_temperatura ;", followed by a small information icon (i).
- A section titled "Variable Permission" with a small information icon (i), containing two radio buttons: "Read & Write" (unselected) and "Read Only" (selected).
- A section titled "Variable Update Policy" with a small information icon (i), containing two radio buttons: "On change" (selected) and "Periodically" (unselected).
- At the bottom right, there are two buttons: "CANCEL" and "SAVE".

Ilustración 1.10 – Creamos la variable para la Humedad.

Por último, creamos una nueva variable para el encendido y apagado del LED. Con la configuración que se muestra en la *ilustración 1.11*. Observe que el tipo de variable es booleana ya que podremos apagar y encender el LED mediante un botón virtual. Por lo tanto, necesitamos configurar **“Read & Write”**, ya que se requiere leer el estado del botón virtual.

The screenshot shows a dialog box titled "Add variable". It contains the following fields and options:

- Name:** A text input field containing "esp32_led".
- Sync with other Things:** A button with a circular arrow icon and an information icon.
- Boolean:** A dropdown menu showing "eg. true".
- Declaration:** A text input field containing the code `bool esp32_led ;`, with an information icon to its right.
- Variable Permission:** Two radio buttons: "Read & Write" (which is selected) and "Read Only".
- Variable Update Policy:** Two radio buttons: "On change" (which is selected) and "Periodically".
- Buttons:** At the bottom, there are two buttons: "CANCEL" and "ADD VARIABLE".

Ilustración 1.11 – Creamos la variable para el encendido y apagado del LED.

Con lo anterior tenemos las variables que requerimos para el proyecto, como se observa en la *ilustración 1.12*.



Cloud Variables			ADD
	Name ↓	Last Value	Last Update
☐	dht11_humedad float dht11_humedad;	-	⋮
☐	dht11_temperatura float dht11_temperatura;	-	⋮
☐	esp32_led bool esp32_led;	-	⋮

Ilustración 1.12 – Variables creadas en IoT Cloud.

El siguiente paso es conectar físicamente la ESP32 con la PC y observar el número de puerto de comunicaciones (COM) que asignó el sistema operativo. En nuestro caso se asignó el COM3 como se muestra en la *ilustración 1.13*. *Esto se observa en “Administrador de dispositivos” de Windows.*



Ilustración 1.13 – Puerto de comunicaciones asignado por el Sistema Operativo.

Una vez confirmado el puerto COM, regresamos a la página de IoT Cloud y localizamos **“Associated Device”**, y hacemos clic en el botón **“Select device”**, como se muestra en la *ilustración 1.14*.

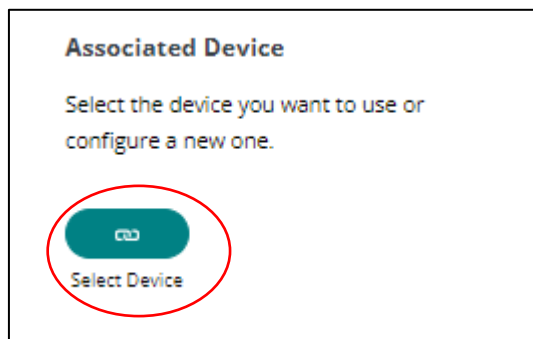


Ilustración 1.14 – Vinculación de la tarjeta ESP32 con Arduino IoT Cloud.

Se abrirá una ventana y elegimos la opción de **“SET UP NEW DEVICE”** que nos llevará a la ventana mostrada en la *ilustración 1.15*. Seleccionamos la opción de **“Third Party Device”**.

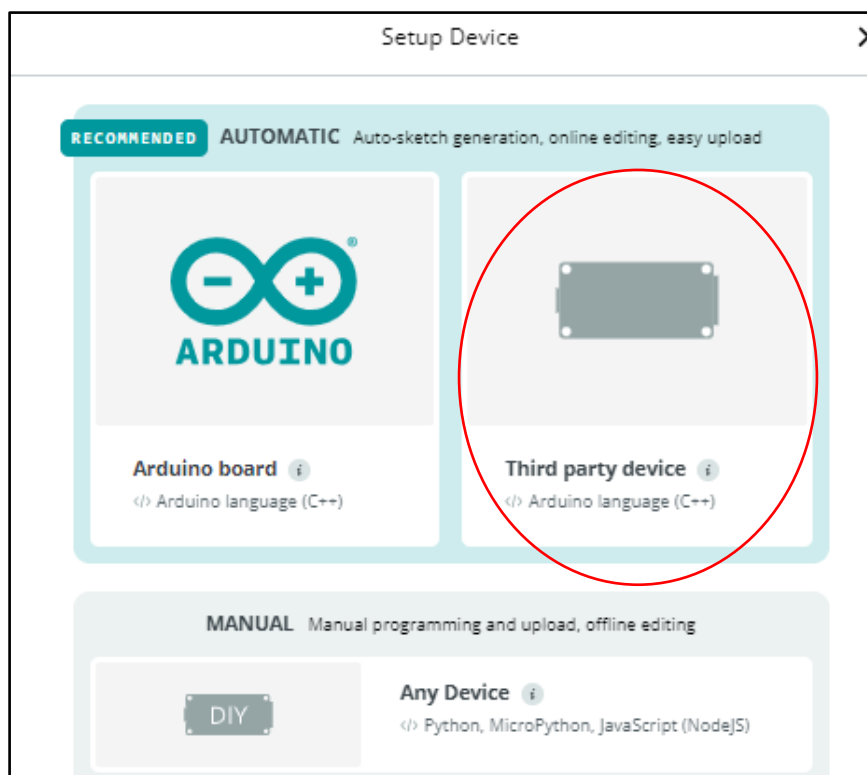


Ilustración 1.15 – Selección de la marca de la tarjeta de desarrollo.

Posteriormente elegimos la tarjeta y el modelo con el que se implementó el circuito físico. En este caso se eligió la configuración de la ventana mostrada en la *ilustración 1.16*. Hacemos clic en “**CONTINUE**”

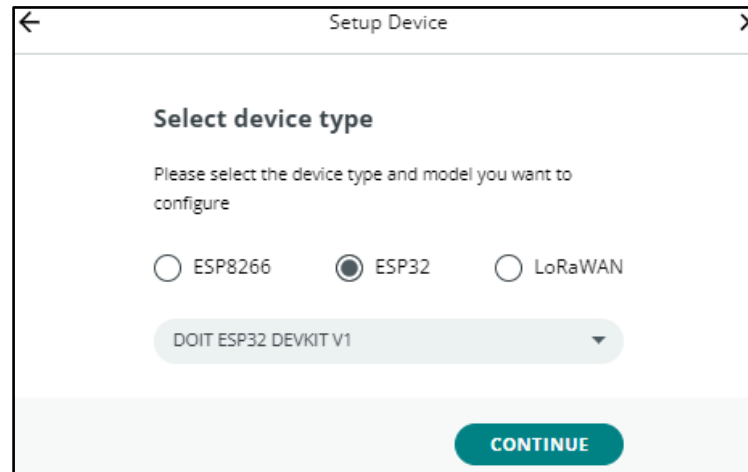


Ilustración 1.16 – Selección del modelo de la tarjeta de desarrollo.

En la *ilustración 1.17* se muestra el modelo y pines de conexión de la tarjeta con la cual se desarrolló esta práctica.

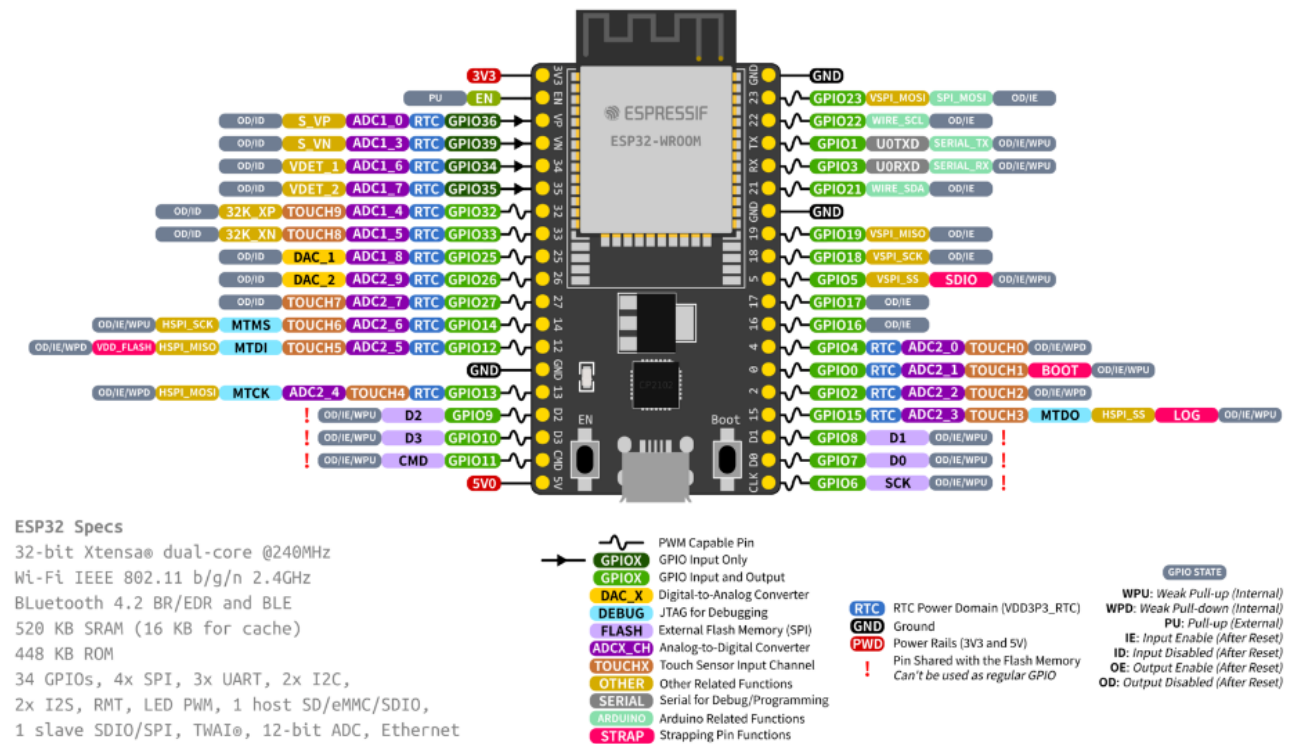


Ilustración 1.17 – ESP32 WROOM.

Le ponemos nombre a nuestro dispositivo. Clic en “**NEXT**” y se abrirá una ventana como la de la *Ilustración 1.18*. Esta es **muy importante** ya que nos dará un **ID** y una **clave secreta** que nos servirá para el código de programación. La ventana nos da la opción de descargar esta información en formato PDF. Se recomienda descargarla y guardarla. Activamos la casilla y hacemos clic en “**CONTINUE**”. Se abrirá otra ventana y elegimos “**DONE**”.

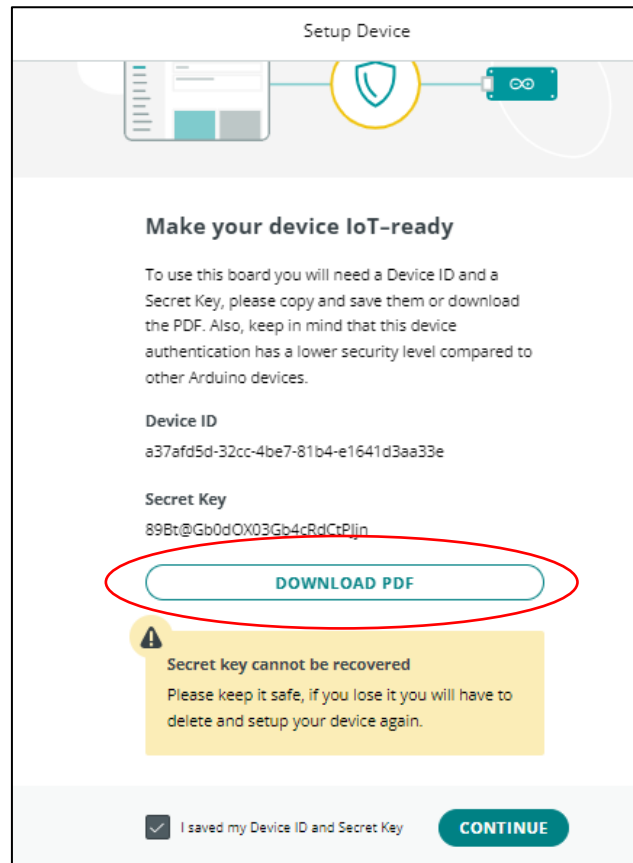


Ilustración 1.18 – Se genera el ID y la clave secreta.

Realizado lo anterior hay que cargar los datos de la red WiFi, haciendo clic en el botón de **“Configure”** del menú **“Network”**. Como se muestra en la *ilustración 1.19*.

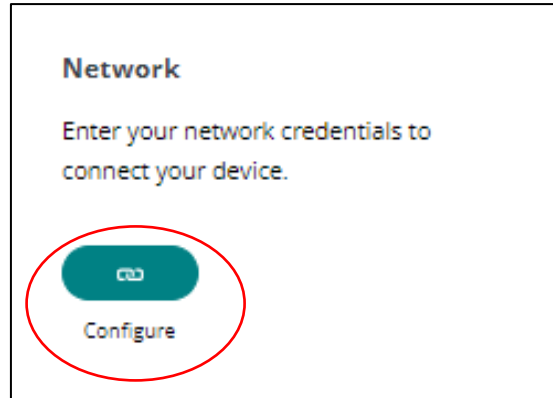


Ilustración 1.19 – Menú para la configuración de la red WiFi.

Agregamos los datos de la red a la que se conectará nuestra tarjeta ESP32 y también escribimos la clave secreta que se generó anteriormente, que puedes consultarla del PDF descargado en el paso anterior, como se observa en la *ilustración 1.20*. Clic en **“SAVE”** para finalizar.

A screenshot of a "Configure network" dialog box. It has a title bar with a close button (X). The main text says "Enter your network credentials to allow your device to connect to the Cloud." There are three input fields: "Wi-Fi Name *" with the value "Linksys07982", "Password" with masked characters "*****" and an eye icon, and "Secret Key *" with the value "j9Bt@Gb0dOX03Gb4cRdCtPJjn" and a key icon. Below the fields, there is an "IMPORTANT" note: "Remember to go to the 'Sketch' tab and upload the sketch to load the credentials on the board." At the bottom right, there is a teal "SAVE" button.

Ilustración 1.20 – Ventana de configuración de la red WiFi.

Una vez configurado lo anterior nos vamos a la pestaña de “**SKETCH**”, como se muestra en la *ilustración 1.21*.

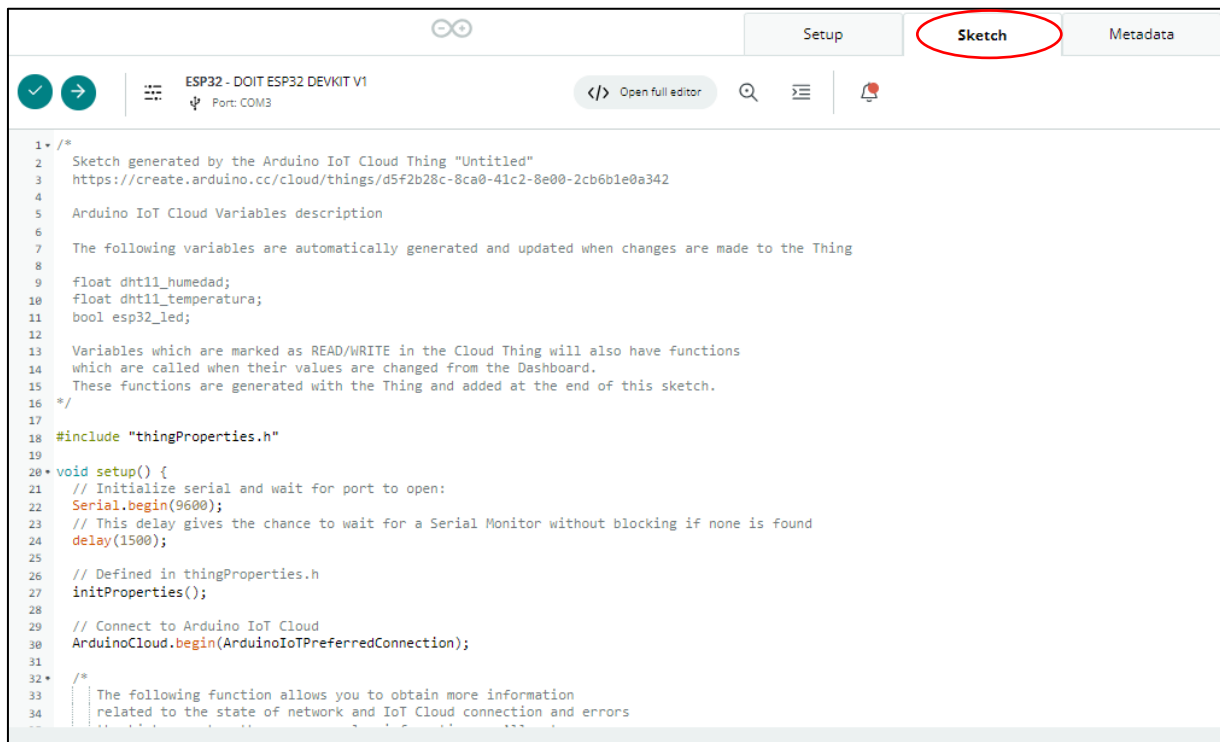


Ilustración 1.21 –Pestaña de SKETCH en IoT Cloud.

Al hacer el paso anterior nos saldrá un mensaje indicando que el navegador requiere instalar un programa o agente (*ilustración 1.22*) que sirve como PLUG-IN para que el navegador trabaje con el IoT cloud, lo instalamos. Si usa un navegador como Google Chrome bloqueará por defecto la ventana emergente. En este caso permita el acceso a la ventana. Descargue el programa e instale.



Ilustración 1.22 –Instalación del agente de IoT Cloud.

Hecho lo anterior empezamos a trabajar con el código haciendo clic en la opción “**OPEN FULL EDITOR**”, como se muestra en la *ilustración 1.23*.

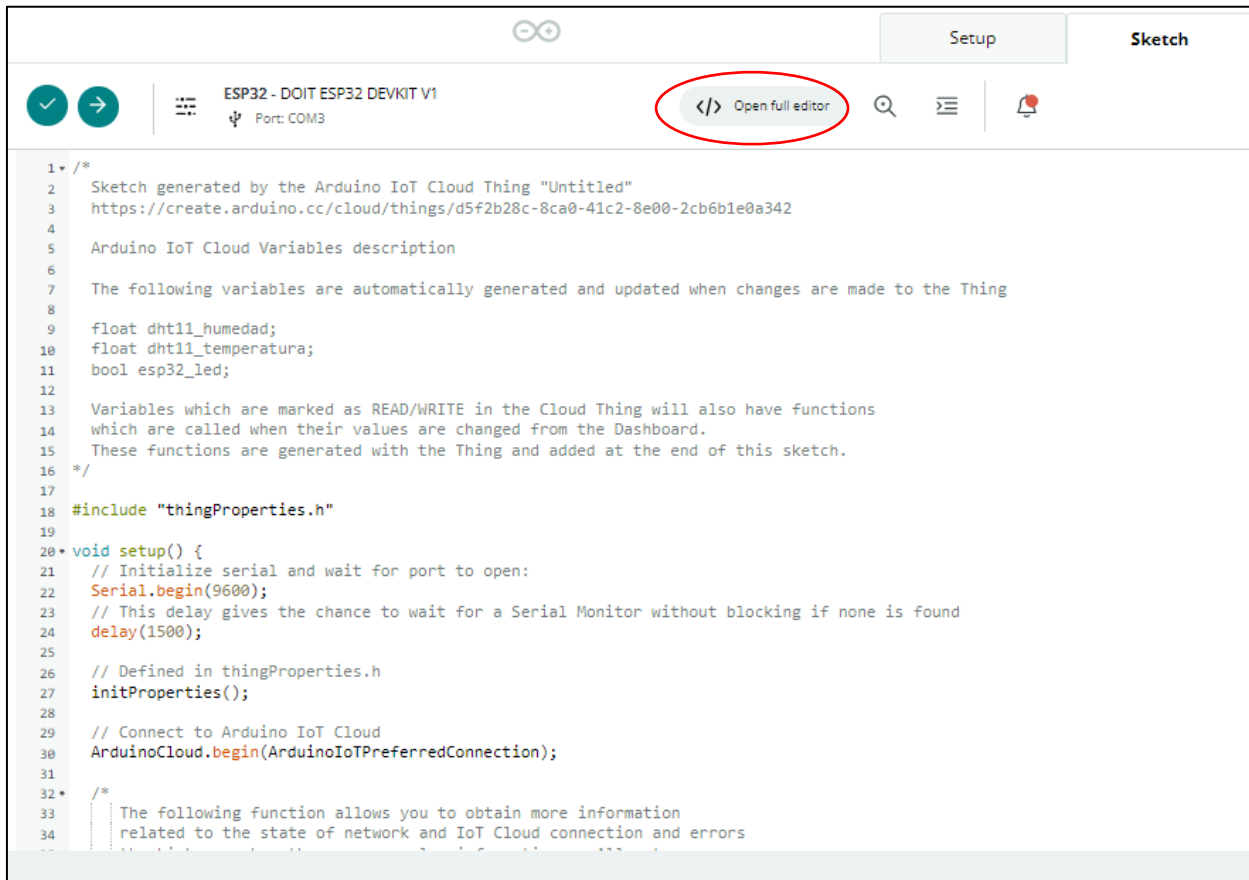


Ilustración 1.23 –Opción para trabajar con el código completo.

Se abrirá la ventana mostrada en la *ilustración 1.24* para poder modificar el código. Esta aplicación es conocida como **Arduino Cloud Editor**.

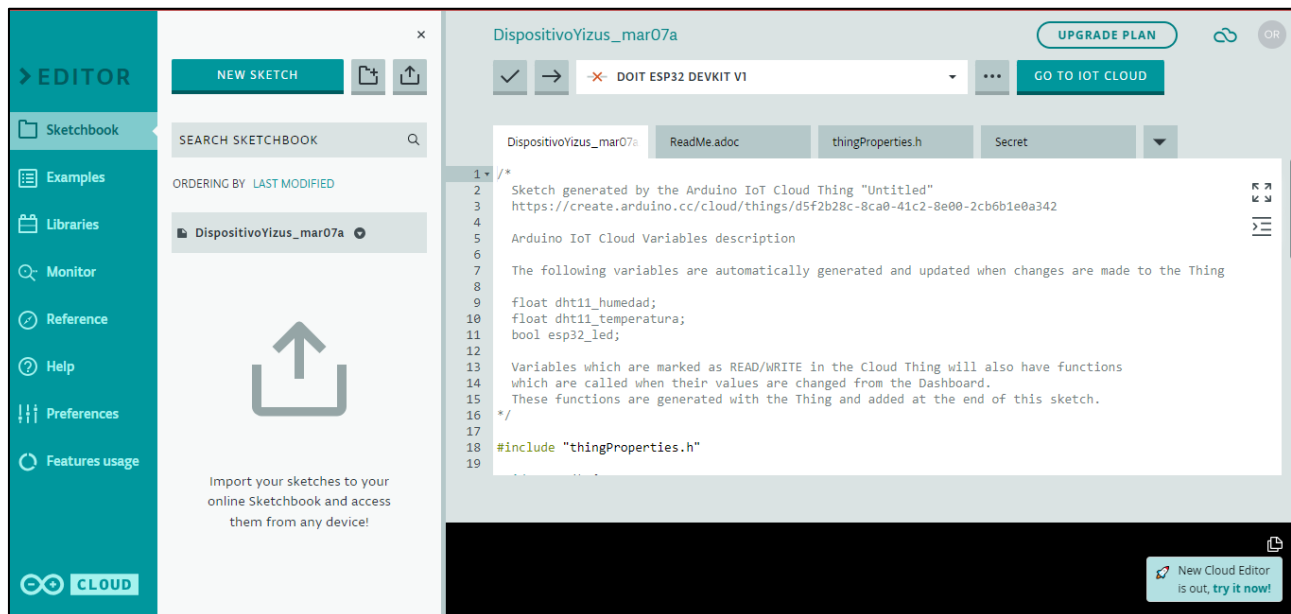


Ilustración 1.24 – Arduino Cloud Editor.

En esta ventana elegimos nuevamente la tarjeta con la que estamos trabajando y el puerto de comunicaciones asignado, como se muestra en la *ilustración 1.25*.



Ilustración 1.25 –Selección de la tarjeta de desarrollo y el puerto de comunicaciones.

Ahora instalamos las librerías necesarias para el funcionamiento del DHT11, nos dirigimos al menú de “**Libraries**” y luego en el botón de “**Library Manager**” y se abrirá la ventana mostrada en la *ilustración 1.26*, en el cuadro de texto escribimos DHT y localizamos la opción “**DHT SENSOR LIBRARY FOR ESPX**” activamos la estrella y clic en “**Done**”.

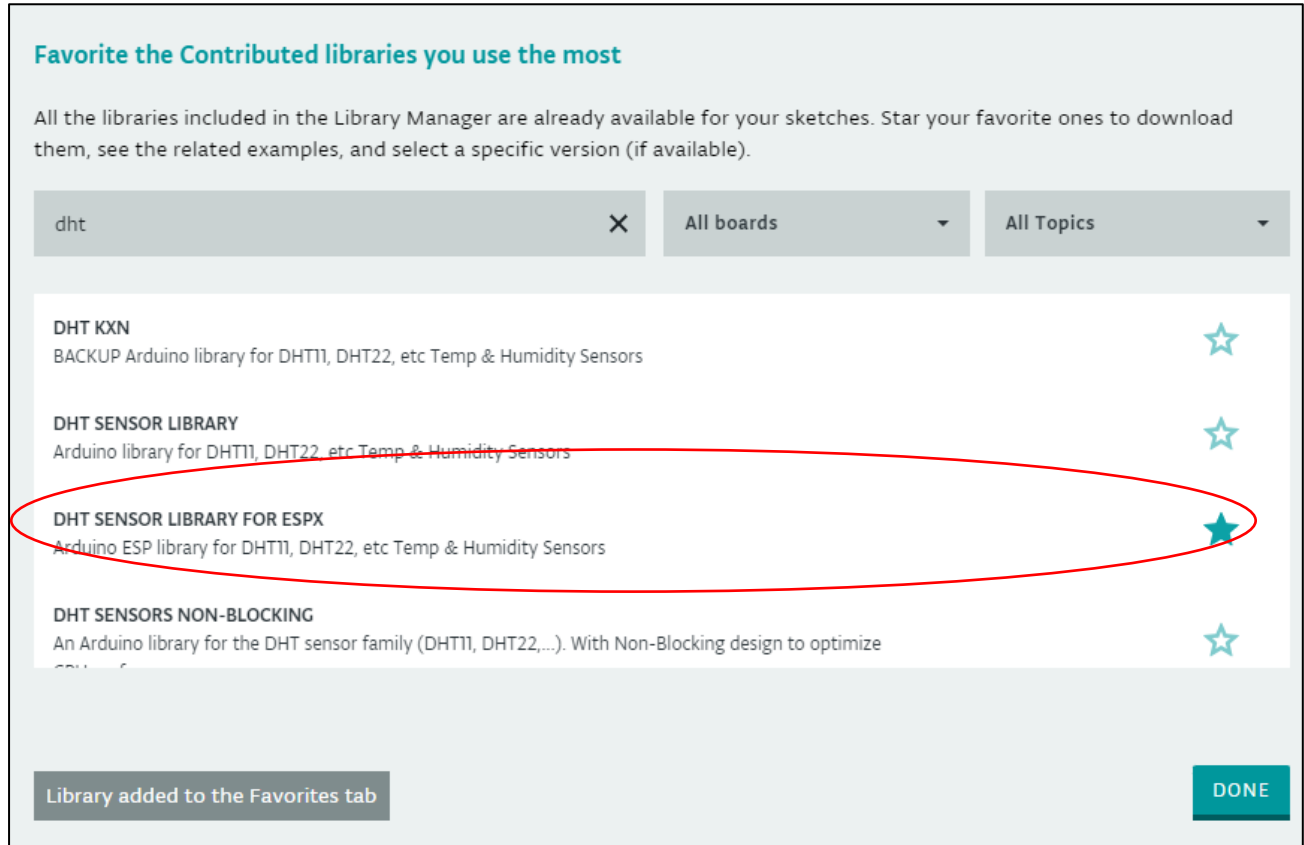


Ilustración 1.26 – Librerías requeridas para el sensor DHT11.

Ahora hacemos clic en **“INCLUDE”** para agregar al código la librería instalada, como se muestra en la *ilustración 1.27*.

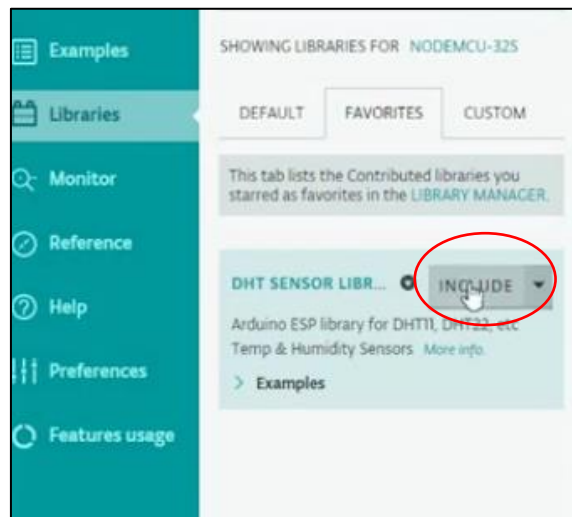


Ilustración 1.27 –Librerías DHT11.

La mayor parte del código se genera con la información que se ha agregado anteriormente: tipo de tarjeta, configuración del WiFi, bibliotecas, etc.

Ahora agregamos unas líneas de código. Para empezar, creamos un objeto del tipo DHTesp fuera del void Setup.

```
#include "thingProperties.h"
DHTesp dht;
void setup() {
  // Initialize serial and wait for port to open:
  Serial.begin(9600);
  // This delay gives the chance to wait for a Serial
  delay(1500);
```

← Objeto DHT.

Al final de la función “Void Setup” agregamos lo siguiente:

```
  // Maximum is 4
  */
  setDebugMessageLevel(2);
  ArduinoCloud.printDebugInfo();
  dht.setup(4,DHTesp::DHT11);
}

void loop() {
  ArduinoCloud.update();
  // Your code here
```

← El pin donde se conectó el DHT11 es el 4 de la tarjeta.

Dentro de la función “Void loop” agregamos lo siguiente:

```
void loop() {  
  ArduinoCloud.update();  
  // Your code here
```

```
  dht11_temperatura=dht.getTemperature();  
  dht11_humedad=dht.getHumidity();  
  delay(5000);
```

```
}
```

Estas son las variables que configuraste en el IoT Cloud anteriormente.

Para configurar el Led conectado al puerto 16 según nuestro diagrama, agregamos lo siguiente fuera de la función Void Setup().

```
#include "thingProperties.h"
```

```
DHTesp dht;
```

```
#define led1 16
```

```
void setup() {  
  // Initialize serial and wait for p  
  Serial.begin(9600);
```

Asignamos nombre al pin del LED.

Agregamos el modo del Pin dentro de la función Void Setup().

```
// Connect to Arduino IoT Cloud  
ArduinoCloud.begin(ArduinoIoTPreferredConnection);
```

```
/*  
  The following function allows you to obtain more information  
  related to the state of network and IoT Cloud connection and errors  
  the higher number the more granular information you'll get.  
  The default is 0 (only errors).  
  Maximum is 4  
*/
```

```
setDebugMessageLevel(2);  
ArduinoCloud.printDebugInfo();  
dht.setup(4,DHTesp::DHT11);
```

```
pinMode(led1,OUTPUT);
```

```
}
```

Configuramos el pin 16 como salida.

```
void loop() {
```

Cuando creamos la variable `led_esp32` el IoT Cloud crea una función con el mismo nombre. Localiza esta función, debe encontrarse a final del código y agrega lo que se te indica.

```
void onEsp32LedChange() {  
  // Add your code here to act upon Esp32Led change  
  if(esp32_led){  
    digitalWrite(led1,HIGH);  
  }  
  
  else{  
    digitalWrite(led1,LOW);  
  }  
}
```

← Condicional para el encendido y apagado del LED.

Una vez realizado lo anterior cargamos el código en el botón **(Verify and Upload)** que se nos indica en la *ilustración 1.28*. Esperamos unos momentos hasta que en la parte inferior observemos que se cargó el código al 100% y nos aparezca el mensaje de **“Success”**. Indicando que todo fue correcto.

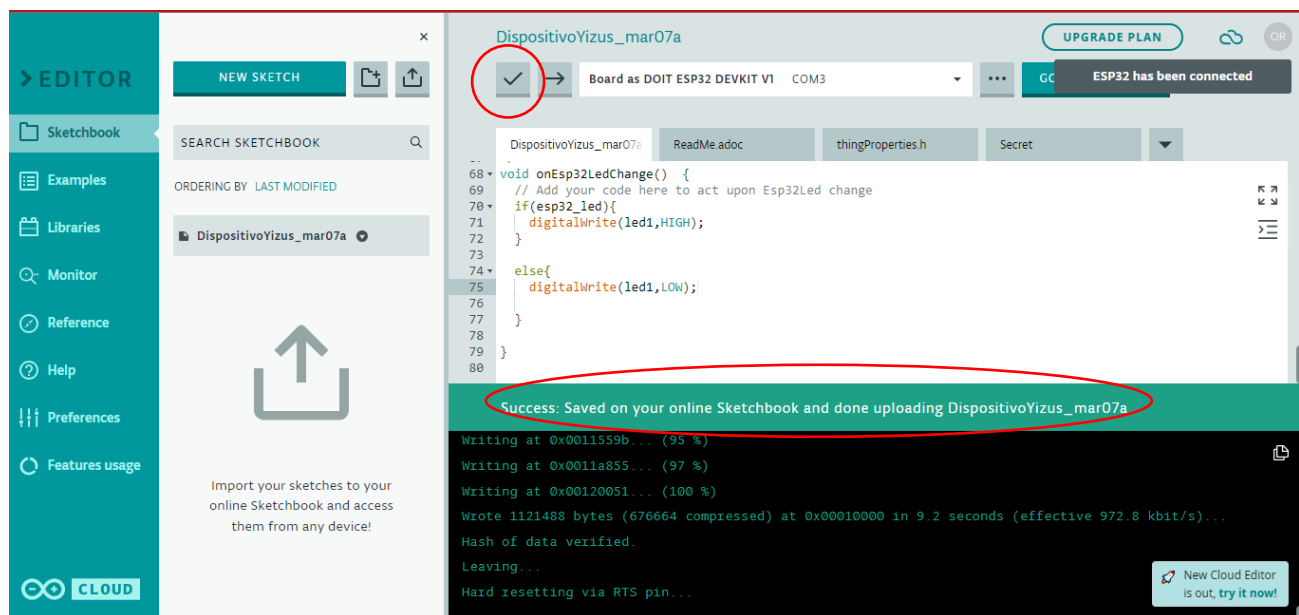
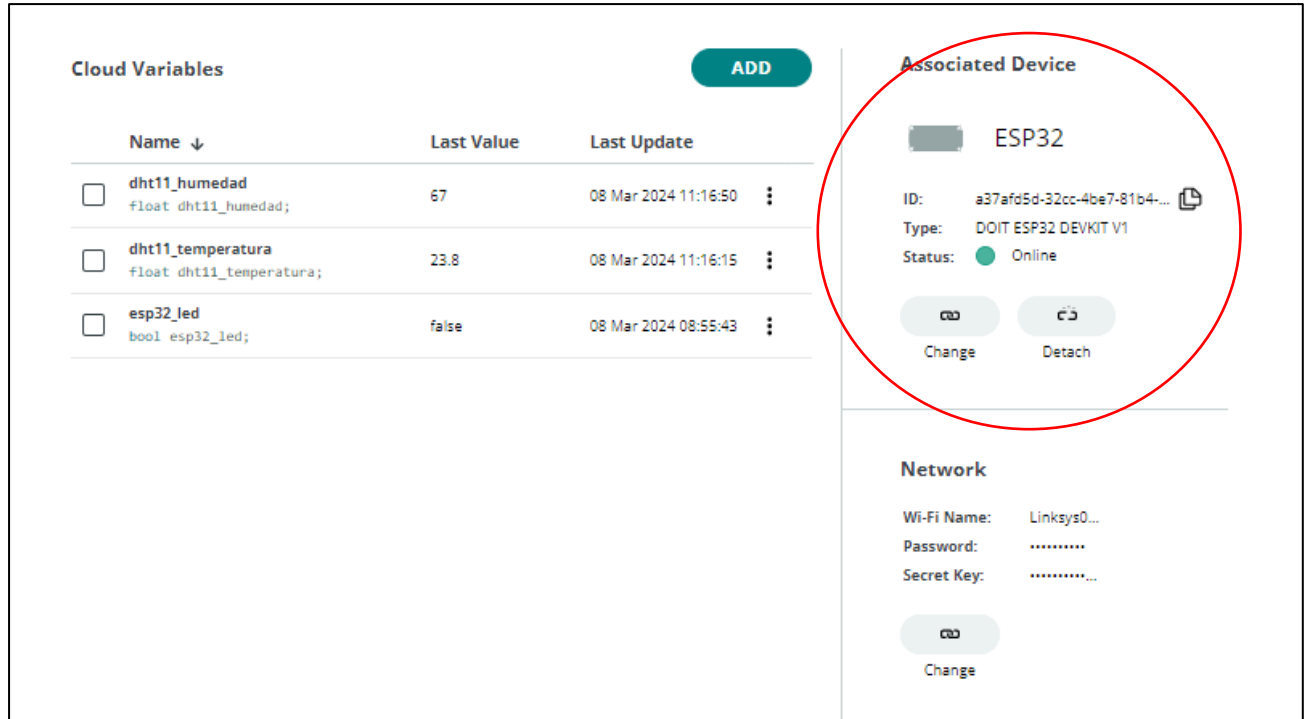


Ilustración 1.28 –Cargando el código a la tarjeta ESP32.

Una vez realizado esto regresamos a la página principal del IoT Cloud y observamos que el dispositivo ya se encuentra en línea, como se muestra en la *ilustración 1.29*.



The screenshot displays the IoT Cloud interface. On the left, under 'Cloud Variables', there is a table with three rows. Each row has a checkbox, a name with a dropdown arrow, a data type and variable name, a last value, a last update timestamp, and a menu icon. The variables are 'dht11_humedad' (float, 67), 'dht11_temperatura' (float, 23.8), and 'esp32_led' (bool, false). On the right, under 'Associated Device', an ESP32 device is shown with its ID, type, and status (Online). Below this, network details are listed, including Wi-Fi Name, Password, and Secret Key. A red circle highlights the 'Associated Device' section.

	Name ↓	Last Value	Last Update
☐	dht11_humedad float dht11_humedad;	67	08 Mar 2024 11:16:50
☐	dht11_temperatura float dht11_temperatura;	23.8	08 Mar 2024 11:16:15
☐	esp32_led bool esp32_led;	false	08 Mar 2024 08:55:43

Associated Device

ESP32

ID: a37afd5d-32cc-4be7-81b4-...
Type: DOIT ESP32 DEVKIT V1
Status: Online

Change Detach

Network

Wi-Fi Name: Linksys0...
Password:
Secret Key:
Change

Ilustración 1.29 –Tarjeta ESP32 conectada a la nube.

1.3 Diseño del panel de control IoT o “Dashboard”.

Para diseñar el Dashboard desplegamos el menú de la parte izquierda de la pantalla, haciendo clic en el botón que se muestra en el círculo rojo de la *ilustración 1.30*.

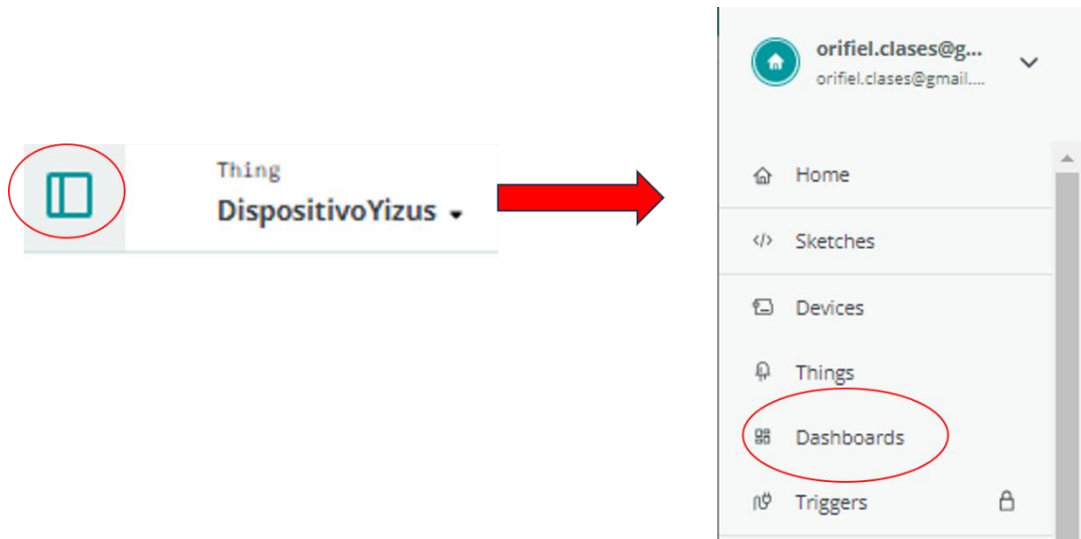


Ilustración 1.30 –Selección del Dashboard.

Al hacer clic en el menú de “**Dashboard**” se abrirán una página con lo mostrado en la *ilustración 1.31*. Seleccionamos “**CREATE DASHBOARD**”.

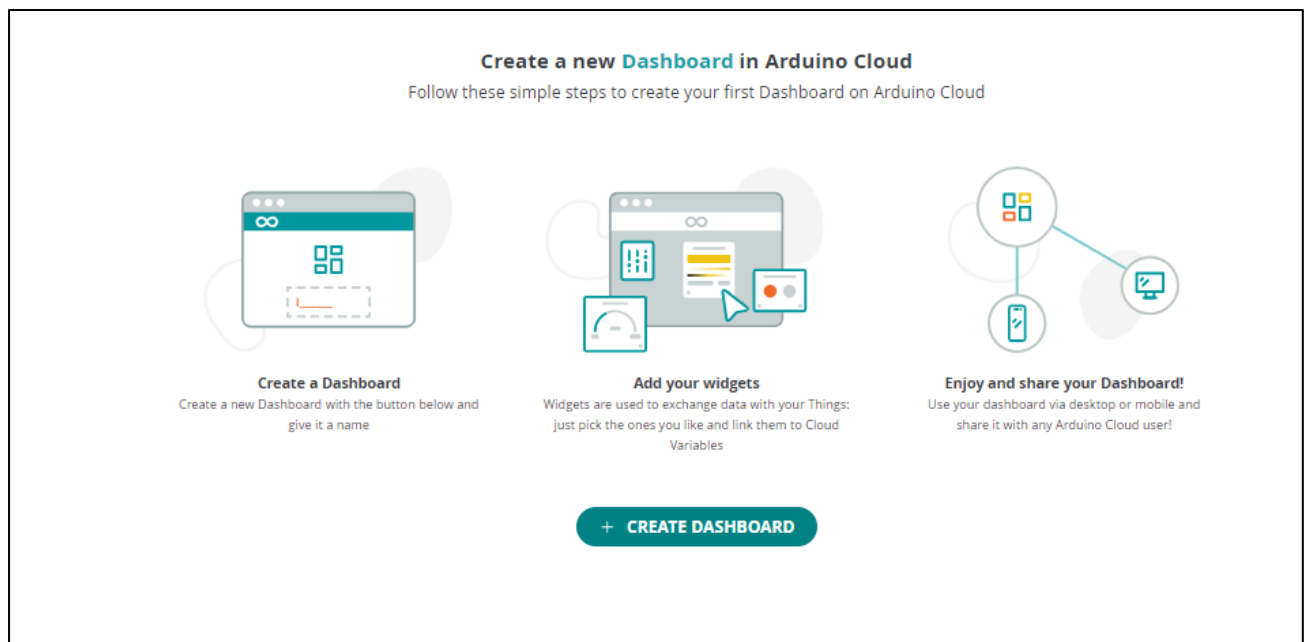


Ilustración 1.31 – Menú principal del Dashboard.

Se abrirá un panel donde nosotros podemos diseñar nuestro panel de control IoT, Mostrado en la *ilustración 1.32*.

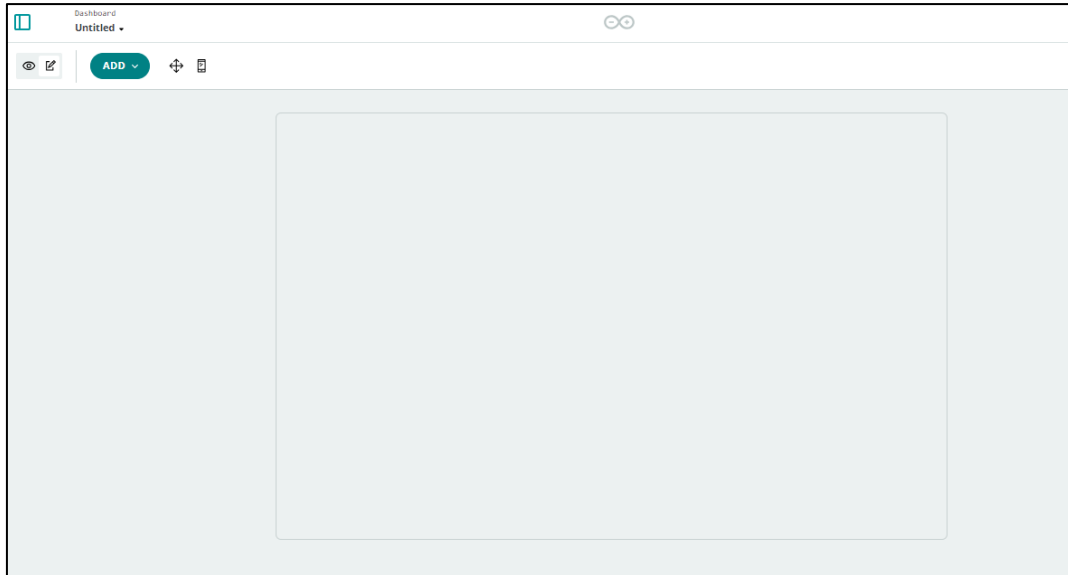


Ilustración 1.32 – Panel de diseño del Dashboard.

Hacemos clic en **“ADD”** para desplegar los Widgets disponibles, como se muestra en la *ilustración 1.33*.

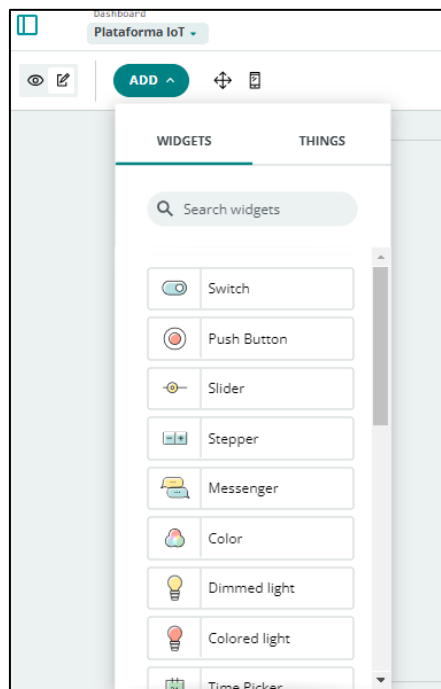


Ilustración 1.33 –Widgets.

En este ejemplo agregamos un widget del tipo “Gauge” para la medición de temperatura. Lo configuramos como se muestra en la *ilustración 1.34*. En el botón que dice **“Link variable”** hacemos clic y lo enlazamos a la variable **“dht11_temperatura”**, que habíamos creado con anterioridad. Cuando finalicemos la configuración hacemos clic en **“DONE”**.

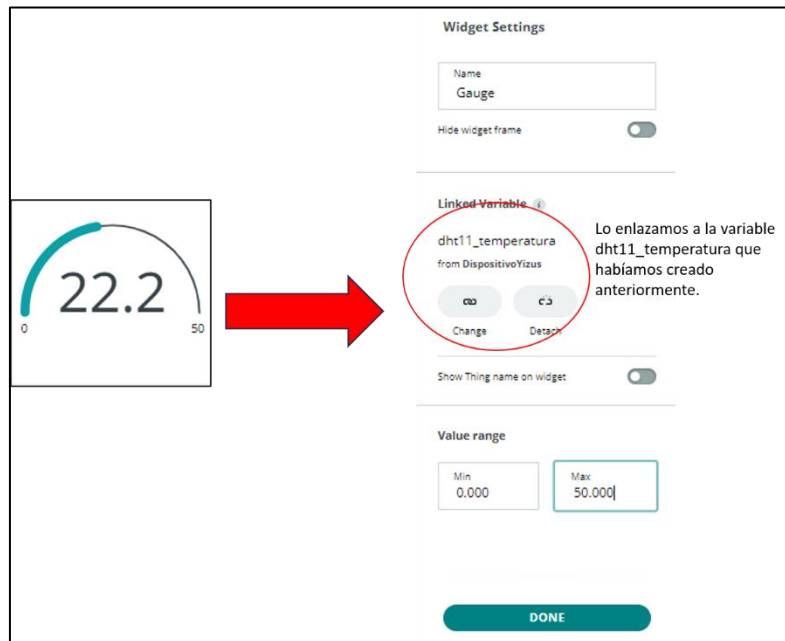


Ilustración 1.34 –Widget Gauge.

Seguidamente agregamos un widget de “27rduino27je” y lo configuramos como se muestra en la *ilustración 1.35*. En este caso este widget se enlaza con la variable “dht11_humedad”.

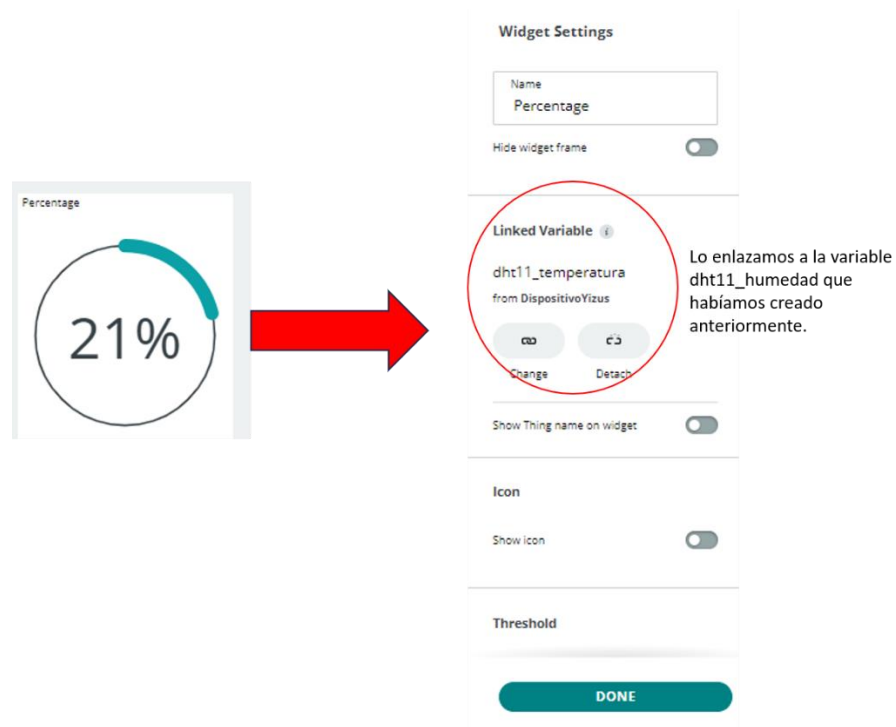


Ilustración 1.35 –Widget 27rduino27je.

Por último, un widget del tipo **switch** que enlazamos a la variable booleana llamada “**esp32_led**” y se configura como se muestra en la *Ilustración 1.36*.

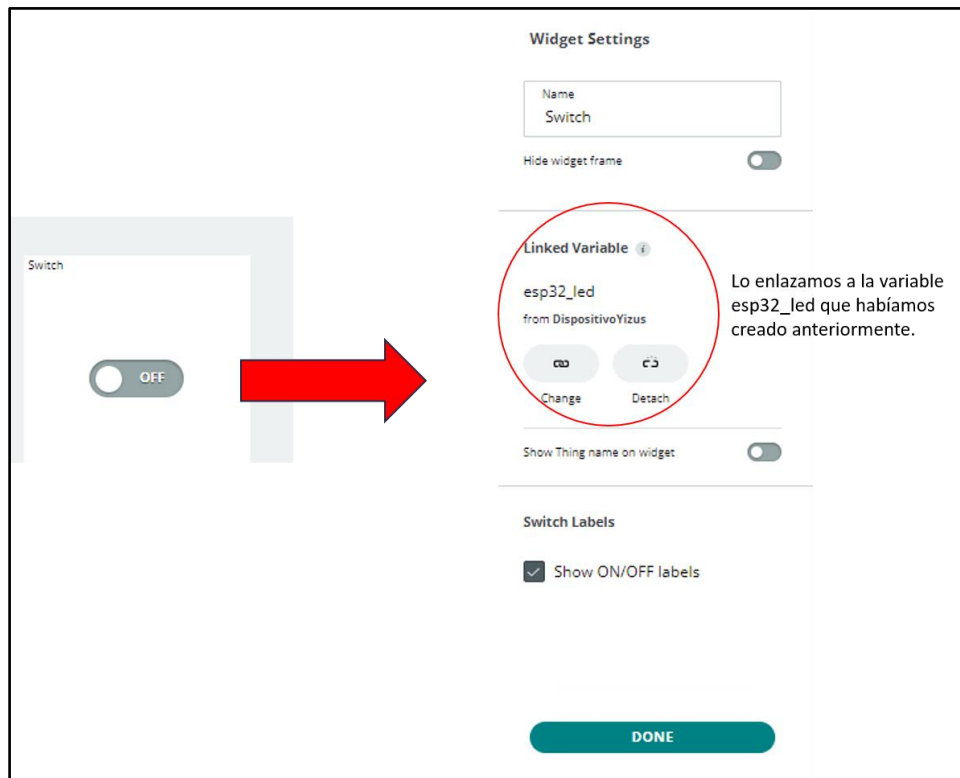


Ilustración 1.36 – Widget switch.

Una vez agregados los widgets puedes cambiar el tamaño y la posición de ellos en la opción “**arrange widgets**”. Para el ejemplo, quedó como se muestra en la *ilustración 1.37*.

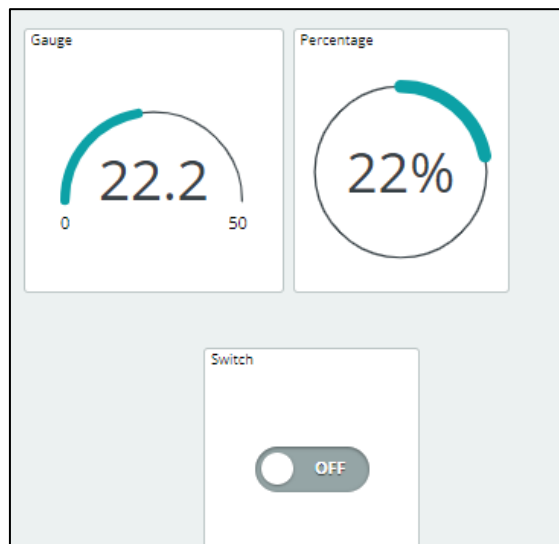


Ilustración 1.37 – Dashboard IoT.

Hacemos clic en modo monitoreo en el botón mostrado en la *ilustración 1.38*, para poder monitorear nuestro sistema.

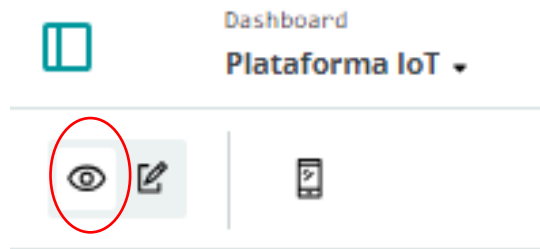


Ilustración 1.38 – Modo monitoreo.

1.4 Usando la aplicación móvil

En la **play store** de Android buscamos la app “**29rduino IoT Cloud**” y procedemos a instalarla como se muestra en la *ilustración 1.39*.

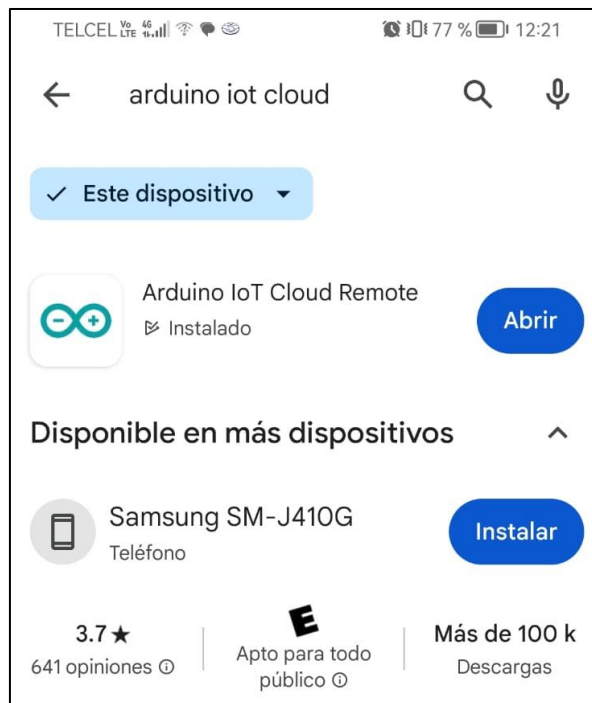


Ilustración 1.39 – Instalación de la app “Arduino IoT Cloud”.

Posteriormente ingresamos con nuestro usuario y contraseña, como se muestra en la *ilustración 1.40*.

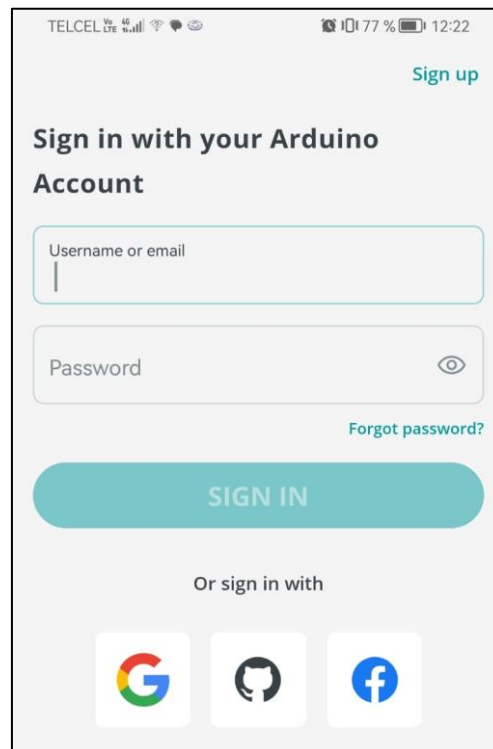


Ilustración 1.40 – Log-in en la aplicación.

Una vez ingresado con tu usuario y contraseña nos aparecerá el dashboard creado, como se muestra en la *ilustración 1.41*.

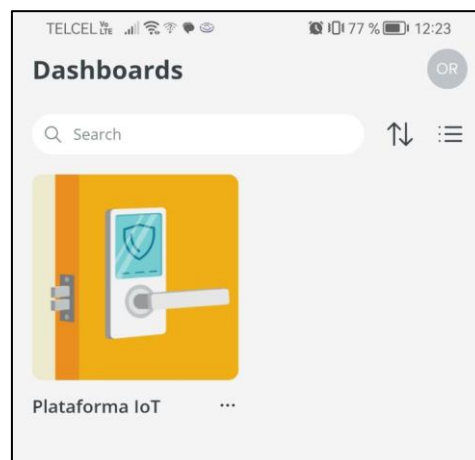


Ilustración 1.41 – Dashboard en la aplicación móvil.

Ingresamos al Dashboard y ya podremos monitorear nuestro sistema IoT, como se muestra en la *ilustración 1.42*.

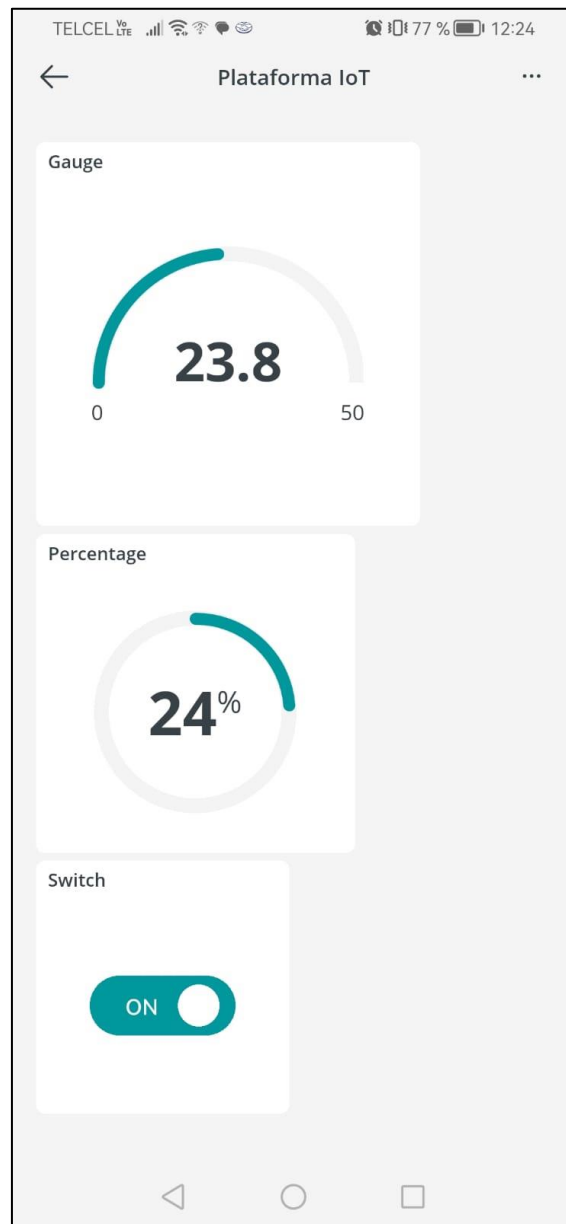
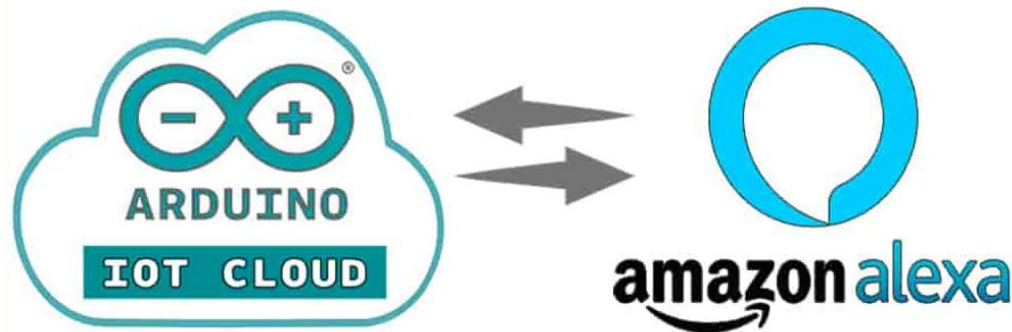


Ilustración 1.42 – Monitoreando el sistema IoT mediante la aplicación móvil.

[illegible]

PRÁCTICA II – ARDUINO IOT CLOUD CON ALEXA



OBJETIVO DE LA UNIDAD:

Conocerá los estándares de comunicación más utilizados en sistemas de IoT para implementar una aplicación funcional.

OBJETIVOS DE LA PRACTICA:

Explorar y poner en práctica la integración de dispositivos IoT con servicios en la nube y enlazarlo al asistente virtual “Alexa” para lograr un control remoto de estos.

ANTECEDENTES

Si se requiere conectar nuestro Arduino o alguna de las tarjetas de desarrollo más comunes a Internet, para enviar información, una de las maneras más sencillas es empleando el IoT y la plataforma online Arduino IoT Cloud.

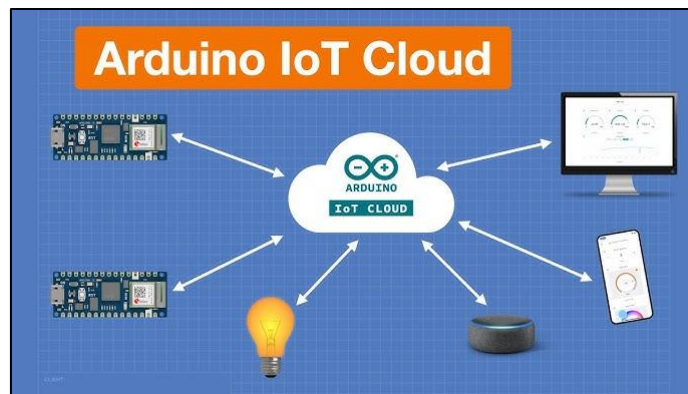


Ilustración 2.1 – Arduino IoT Cloud

Los usos que se le pueden dar a esta plataforma son:

- **Monitorización** de datos: es posible monitorizar fácilmente los valores de los sensores de Arduino a través de un panel de control.
- **Sincronización** de variables: la sincronización de variables nos permite sincronizar variables entre dispositivos, permitiendo la comunicación entre dispositivos con una codificación mínima.
- **Programación**: podemos programar trabajos para que se activen/desactiven diferentes funciones durante una cantidad de tiempo específica (segundos, minutos, horas).
- **Cargas Over-The-Air (OTA)**: permite cargar nuestro código a dispositivos no conectados al ordenador.
- **Webhooks**: integra tu proyecto con otro servicio, como IFTTT.
- **Soporte de Amazon Alexa**: permite que nuestro proyecto sea controlado por voz con la integración de Amazon.
- **Dashboard Sharing**: comparte tus datos con otras personas de todo el mundo

MATERIAL A UTILIZAR

- NodeMCU-32
- LED de cualquier color
- Jumpers Macho-Macho
- Resistencia de 330Ω
- Protoboard

SOFTWARE NECESARIO

IoT Cloud: <https://cloud.arduino.cc/>

Amazon Alexa:

<https://play.google.com/store/apps/details?id=com.amazon.dee.app&hl=es&gl=US>

DESARROLLO DE LA PRÁCTICA

Práctica 2. Encendido de un led a través del asiste de voz de Amazon Alexa

2.1 Implementar el circuito del diagrama que se muestra en la *Ilustración 2.2*

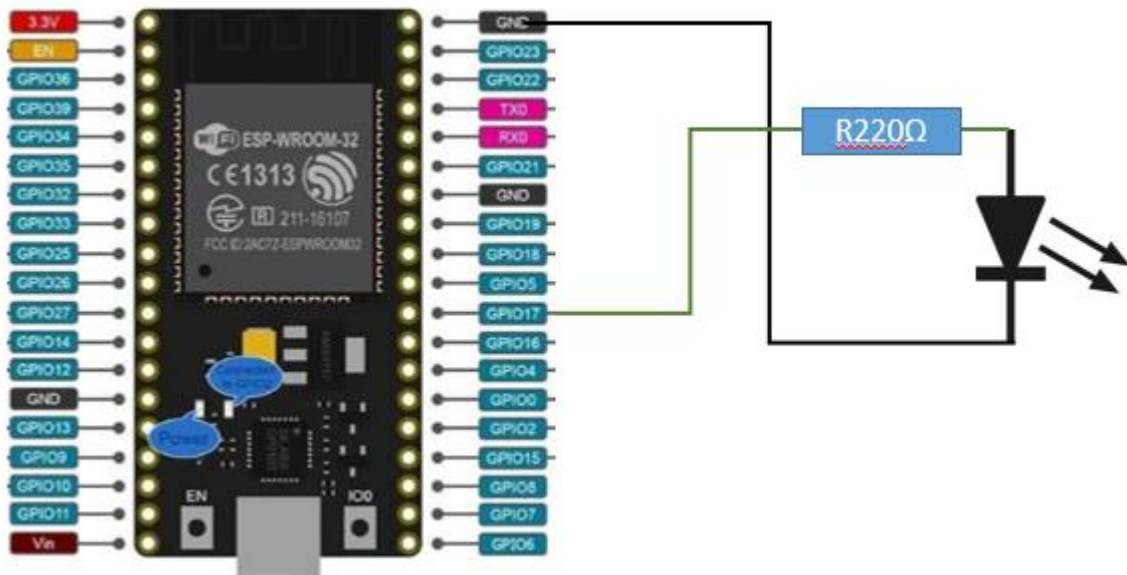
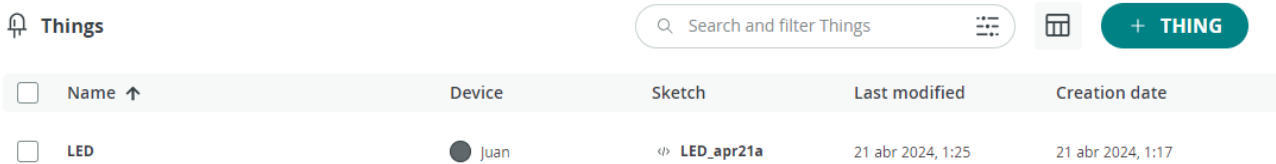


Ilustración 2.2 – Diagrama a implementar

2.2 Configuración y programación de la plataforma IoT Cloud

En la práctica anterior se realizó la configuración necesaria para conectar nuestra tarjeta ESP32 con la página web “Arduino Cloud”. Esa configuración es esencial para realizar el enlace con el asistente de voz “Alexa”.

Para iniciar, debemos dirigirnos al menú principal de “Arduino Cloud”. Damos click en “Things” y seleccionamos nuestro “Proyecto” como se observa en la *Ilustración 2.3*.



☐ Name ↑	Device	Sketch	Last modified	Creation date
☐ LED	Juan	LED_apr21a	21 abr 2024, 1:25	21 abr 2024, 1:17

Ilustración 2.3 – Menú del apartado “Things”

Una vez en este apartado, procedemos a crear una variable como se muestra en la *Ilustración 2.4*.



Name ↓	Last Value	Last Update
☐ esp32_led bool esp32_led;	false	23 Apr 2024 20:00:14

Ilustración 2.4 – Adición de una variable

Configuramos un led para que sea compatible con Alexa, para esto debemos seguir la configuración mostrada en la *Ilustración 2.5*.

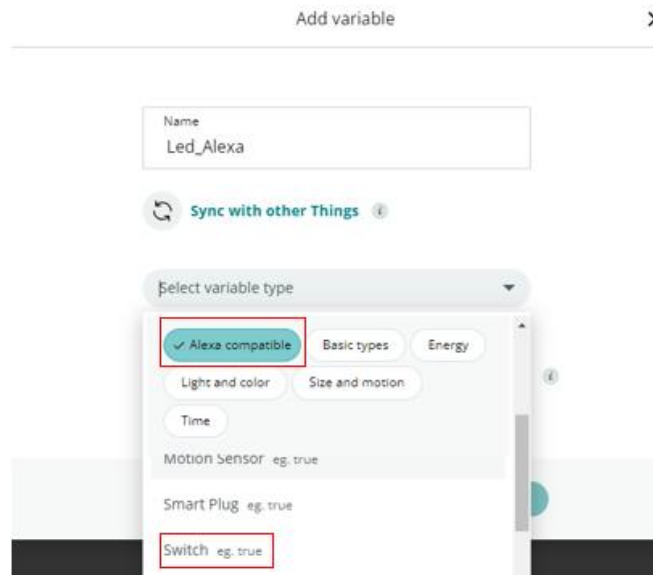


Ilustración 2.5 – Activar compatibilidad con Alexa

Seguidamente, realizamos las configuraciones finales a nuestro led, como se muestra en la *Ilustración 2.6*.

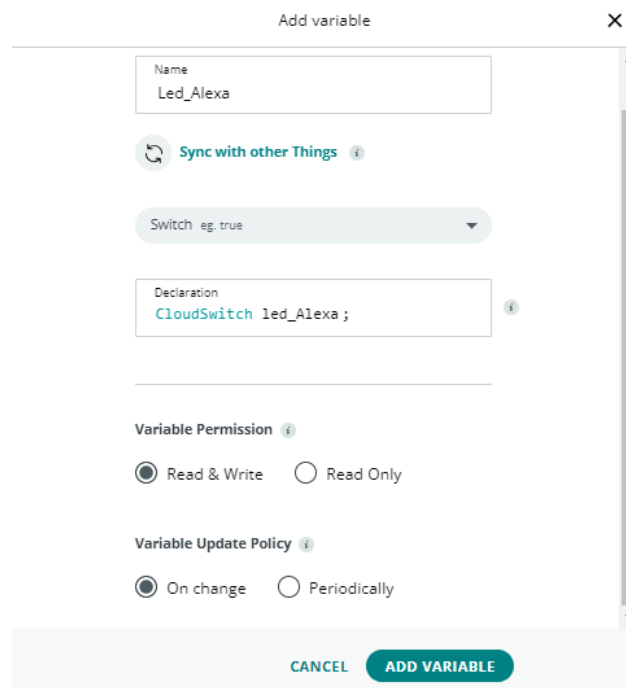


Ilustración 2.6 – Configuración final del LED

Debemos realizarle ciertas modificaciones a nuestro código, para esto nos dirigimos a “Arduino Cloud Editor” como se aprecia en la *Ilustración 2.7*.

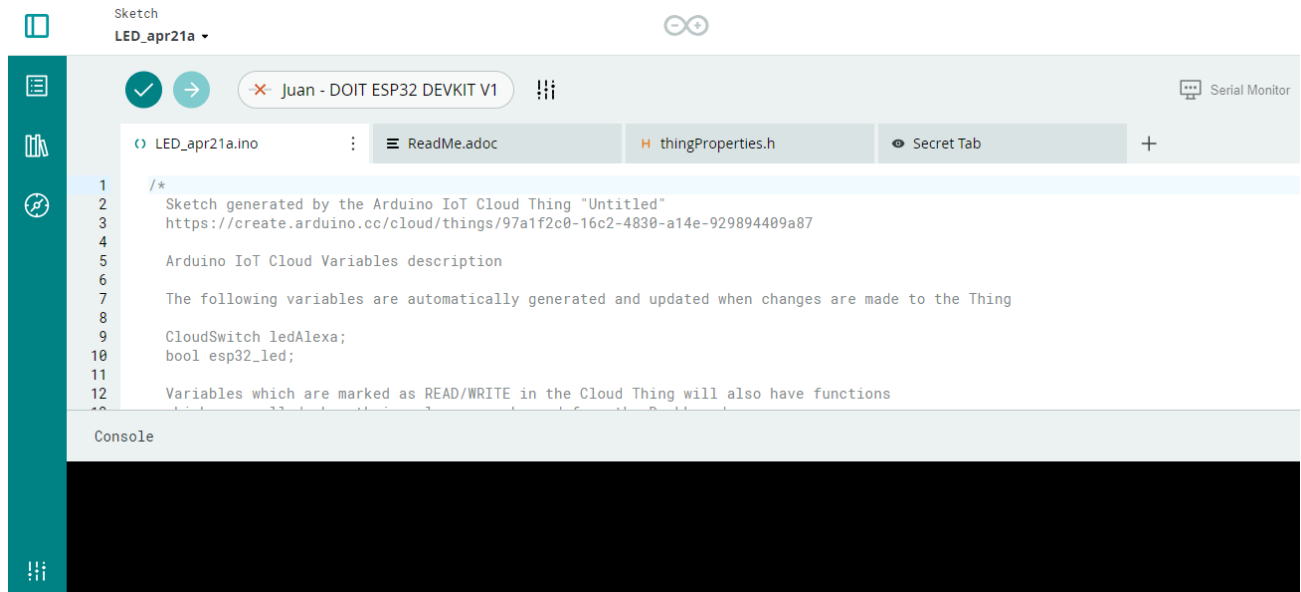


Ilustración 2.7 – Arduino Cloud

Implementamos las siguientes modificaciones a nuestro Código.

En el apartado void setup() asignamos el puerto de nuestro LED como salida.

```
*/  
setDebugMessageLevel(2);  
ArduinoCloud.printDebugInfo();  
  
pinMode(led1, OUTPUT);  
pinMode(17, OUTPUT);  
}
```

← Configuramos el pin 17 como salida.

En pasos anteriores configuramos un LED para que sea compatible con Alexa, el nombre de la variable será relevante para la siguiente modificación, el nombre que se le asignó a la variable debe aparecer de la siguiente manera.

```
/*
Sketch generated by the Arduino IoT Cloud Thing "Untitled"
https://create.arduino.cc/cloud/things/97a1f2c0-16c2-4830-a14e-929894409a87

Arduino IoT Cloud Variables description

The following variables are automatically generated and updated when changes are made to the Thing

CloudSwitch ledAlexa;
bool esp32_led;

Variables which are marked as READ/WRITE in the Cloud Thing will also have functions
which are called when their values are changed from the Dashboard.
These functions are generated with the Thing and added at the end of this sketch.
*/
```



En este caso, nuestra variable se llama ledAlexa.

Al configurar nuestra variable para que sea compatible con Alexa se nos creara la función “void OnLedAlexaChange()”, debemos encontrar esta función y codificar lo siguiente. Cabe mencionar que el nombre “LedAlexa” cambiará dependiendo del nombre que le hayamos asignado a nuestra variable al momento de crearla.

```
/*
Since ledAlexa is READ_WRITE variable, onLedAlexaChange() is
executed every time a new value is received from IoT Cloud.
*/
void onLedAlexaChange() {
  // Add your code here to act upon ledAlexa change
  digitalWrite(17, ledAlexa);
}
```



En este apartado estamos declarando que nuestra variable “LedAlexa” se encuentra en el puerto 17.

Una vez terminadas nuestras modificaciones procedemos a subir el código a nuestra tarjeta. Después agregamos nuestra nueva variable en el “Dashboard”. Recuerda que escogemos un “Switch gadget” para seguidamente vincularlo con nuestro LED como se muestra en la *Ilustración 2.8*.

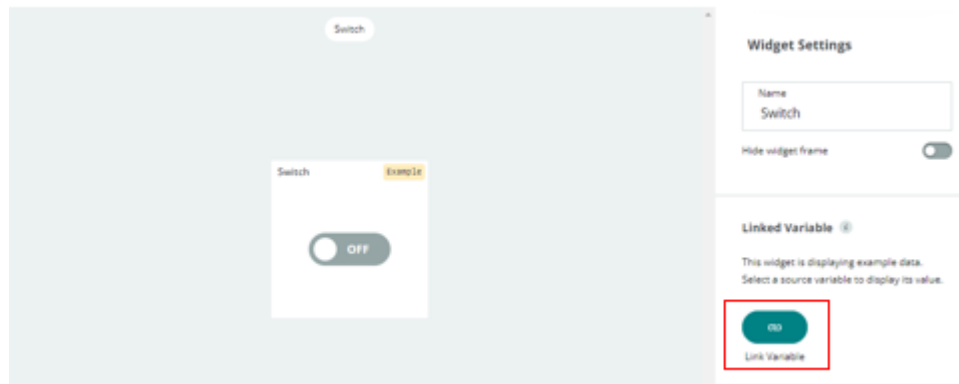


Ilustración 2.8 – Vincular nuestro LED con el Switch

Nos vamos a nuestro dispositivo móvil para instalar la aplicación “Amazon Alexa”. *Ilustración 2.9*.

Enlace: <https://play.google.com/store/apps/details?id=com.amazon.dee.app&hl=es&gl=US>



Ilustración 2.9 – Amazon Alexa

Una vez descargado, abrimos la aplicación y creamos una cuenta. Al hacerlo nos mandará a la pantalla principal de la aplicación, seguidamente nos dirigimos a “Mas” y seleccionamos “Skills y juegos” como se muestra en la *Ilustración 2.10*.

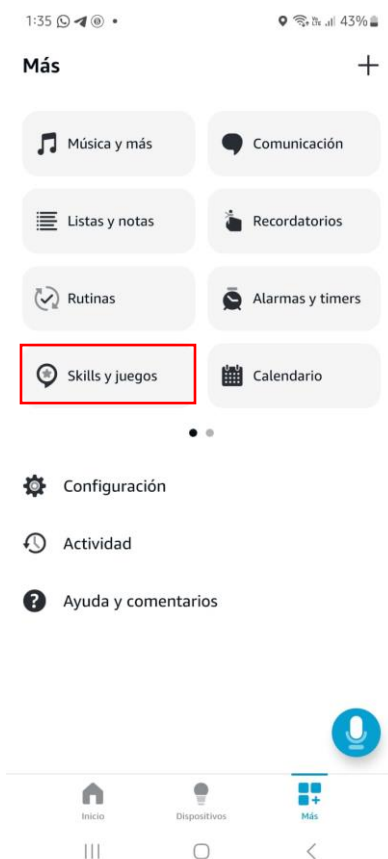


Ilustración 2.10 – Apartado de “Mas”

Una vez seleccionado, buscamos “Arduino” y lo seleccionamos como se muestra en la *Ilustración 2.11*.

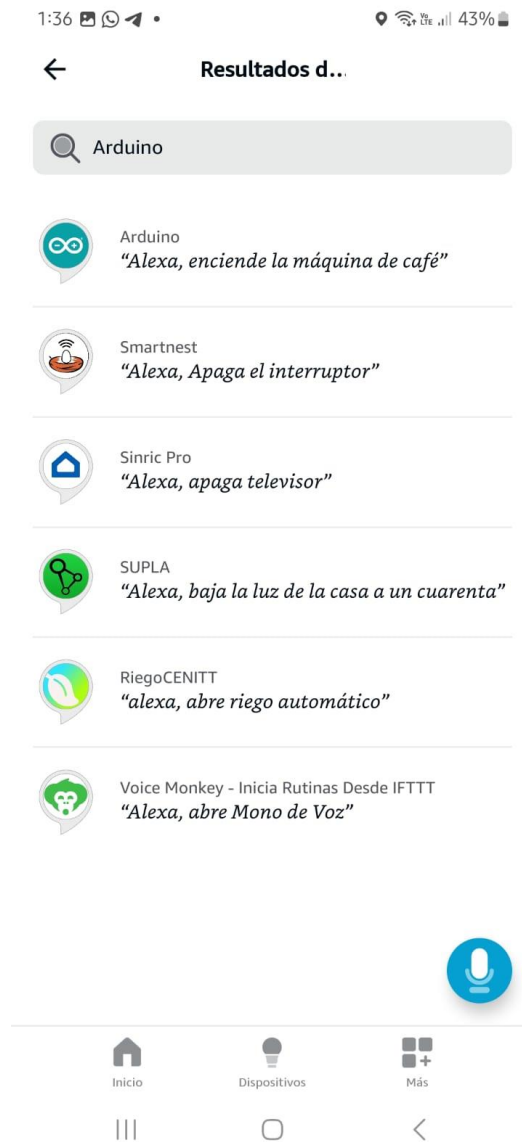


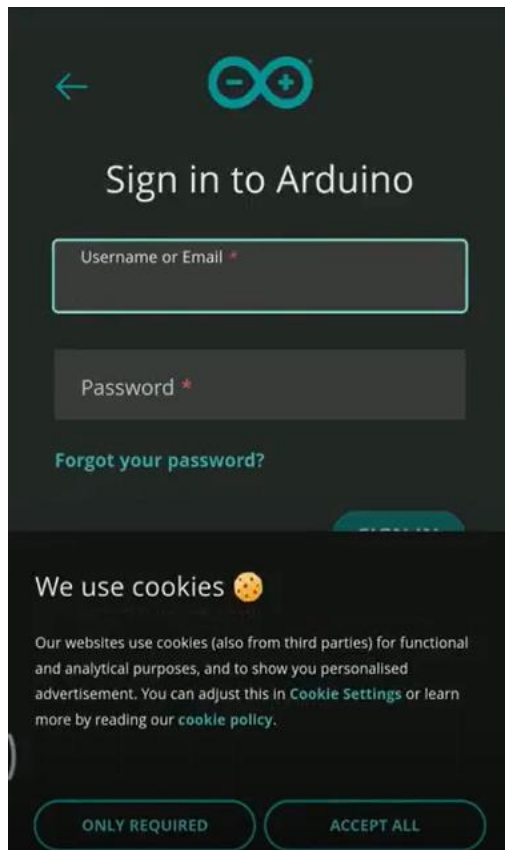
Ilustración 2.11 – Skill de Arduino

Una vez seleccionado, activamos la skill apretando el botón “Activar para usar” como se observa en la *Ilustración 2.12*.



Ilustración 2.12 – Activar la Skill de Arduino

Una vez activada, nos pedirá loguearnos, por lo que ingresamos nuestros datos para iniciar sesion. *Ilustración 2.13.*



← ∞

Sign in to Arduino

Username or Email *

Password *

[Forgot your password?](#)

We use cookies 🍪

Our websites use cookies (also from third parties) for functional and analytical purposes, and to show you personalised advertisement. You can adjust this in [Cookie Settings](#) or learn more by reading our [cookie policy](#).

ONLY REQUIRED ACCEPT ALL

Ilustración 2.13 – Inicio de Sesion con la Skill

Una vez la sesión está iniciada, volvemos a la aplicación de Amazon Alexa y automáticamente se buscará el LED que configuramos en pasos anteriores. Por lo tanto, lo seleccionamos para que quede vinculada con la aplicación. *Ilustración 2.14.*



Ilustración 2.14 – Alexa buscará nuestro dispositivo automáticamente

El LED vinculado debe aparecer en la pestaña de “Dispositivos”. *Ilustración 2.15.*

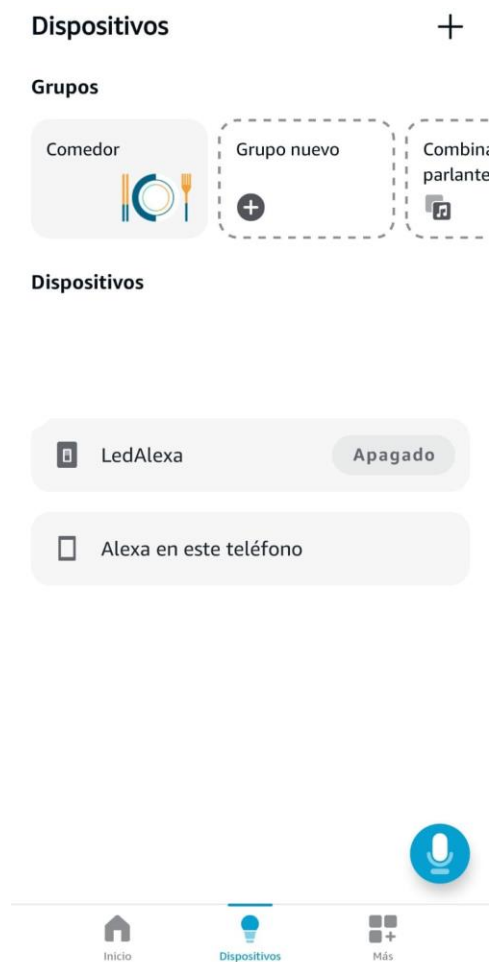


Ilustración 2.15 – Vinculación exitosa de nuestro LED

Ahora, ya podemos manipular nuestro LED desde la aplicación de Amazon Alexa, tanto con el switch como por medio del asistente de voz.

CONCLUSIONES

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

PRÁCTICA III – SINRIC CON ALEXA



OBJETIVO DE LA UNIDAD:

Conocerá los estándares de comunicación más utilizados en sistemas de IoT para implementar una aplicación funcional.

OBJETIVOS DE LA PRACTICA:

Explorar y poner en práctica la integración de dispositivos IoT con servicios en la nube utilizando la aplicación Sinric Pro, y enlazarlos con el asistente virtual “Alexa” para lograr un control remoto a través de la voz de estos dispositivos.

ANTECEDENTES

Si deseamos conectar nuestro ESP32 o cualquier otra tarjeta de desarrollo popular a la nube para poder enviar y recibir datos, una de las soluciones más accesibles y prácticas es utilizar el IoT junto con la plataforma Sinric Pro.

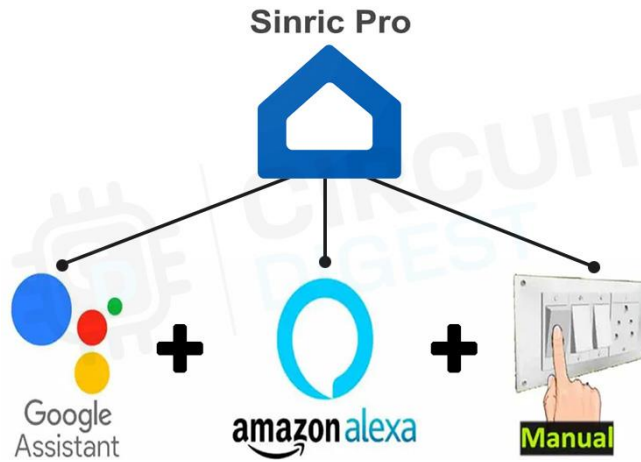


Ilustración 3.1 – Sinric Pro

Los usos y características de esta plataforma son:

- **Control Remoto de Dispositivos:** Permite encender, apagar o ajustar dispositivos conectados desde cualquier lugar a través de la aplicación móvil de Sinric Pro o mediante comandos de voz usando Amazon Alexa o Google Assistant.
- **Monitorización de Sensores:** Facilita la monitorización en tiempo real de los datos provenientes de sensores conectados al ESP32 un panel de control en la plataforma Sinric Pro.
- **Automatización del Hogar:** Ayuda a crear rutinas y automatizaciones para que los dispositivos funcionen de manera autónoma según condiciones específicas, como horarios predefinidos o la lectura de sensores.
- **Cargas Over-The-Air (OTA):** Permite actualizar el firmware del ESP32 de forma remota sin necesidad de estar físicamente conectado al dispositivo, lo que facilita el mantenimiento y la mejora continua de los proyectos.
- **Sincronización de Dispositivos:** Ofrece la capacidad de sincronizar múltiples dispositivos conectados, lo que facilita la comunicación y el control entre ellos con una configuración mínima.
- **Notificaciones y Alertas:** Configura notificaciones para recibir alertas en tiempo real sobre el estado de los dispositivos o cambios en las lecturas de los sensores, mejorando la seguridad y el seguimiento del entorno.

- **Interfaz Amigable para el Usuario:** Proporciona una interfaz intuitiva y fácil de usar para configurar y controlar los dispositivos, lo que reduce la barrera de entrada para usuarios sin experiencia técnica.

MATERIAL A UTILIZAR

- NodeMCU-32
- LED de cualquier color
- Jumpers Macho-Macho
- Resistencia de 330Ω
- Protoboard

SOFTWARE NECESARIO

SINRIC Pro:

<https://sinric.pro/>

Amazon Alexa:

<https://play.google.com/store/apps/details?id=com.amazon.dee.app&hl=es&gl=US>

Aplicación de Sinric Pro:

<https://play.google.com/store/apps/details?id=pro.sinric&hl=es&gl=US>

DESARROLLO DE LA PRÁCTICA

Práctica 3. Encendido de un led a través del asiste de voz de Amazon Alexa mediante SINRIC.

3.1 Implementar el circuito del diagrama que se muestra en la *Ilustración 3.2*

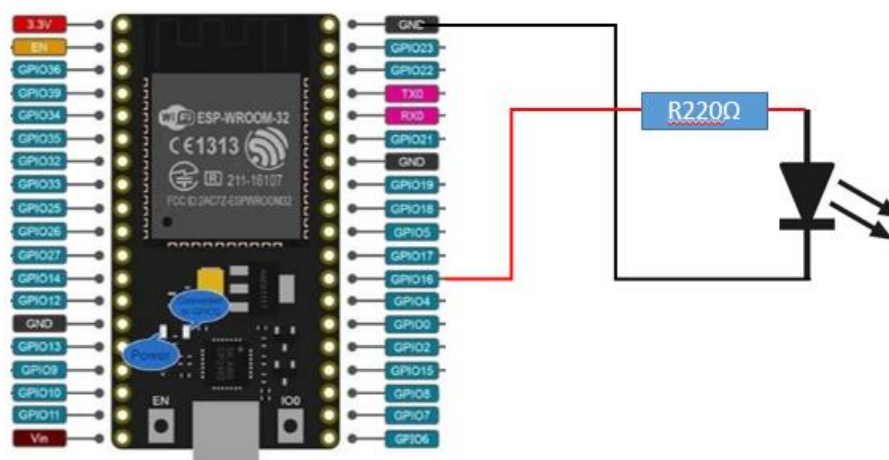


Ilustración 3.2 – Diagrama a implementar

3.2 Configuración de la plataforma Sinric Pro

Para iniciar, nos dirigimos al siguiente enlace: <https://sinric.pro/> Este enlace nos dirige a la página principal de Sinric Pro, procedemos a crear nuestra cuenta. *Ilustración 3.3.*

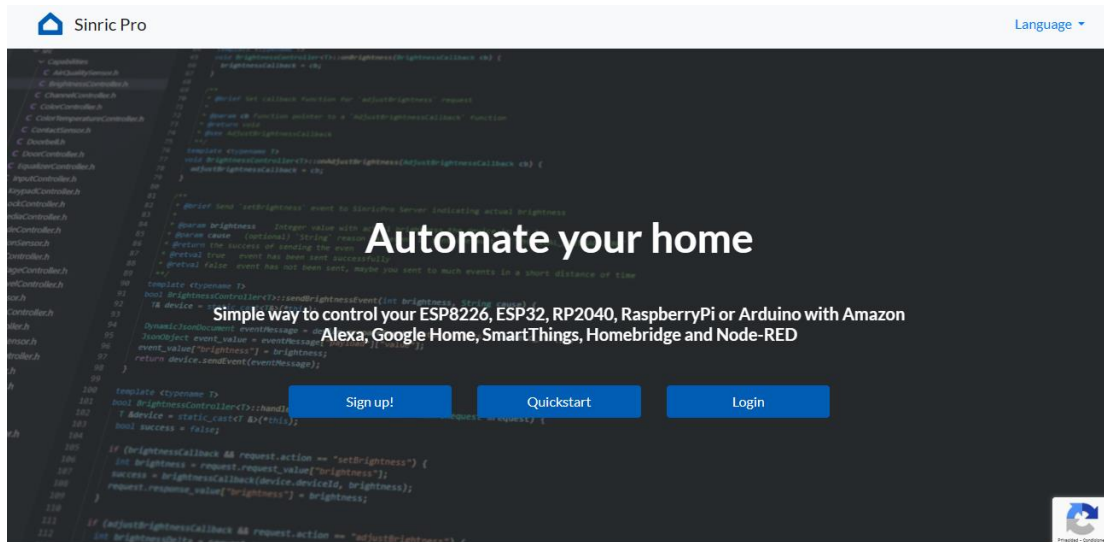


Ilustración 3.3 – Registro en Sinric

Una vez creada nuestra cuenta nos loguearemos y nos redirigirá a la ventana principal, seleccionamos el apartado “Dispositivos” y seguidamente agregamos un dispositivo, como se muestra en la *Ilustración 3.4.*

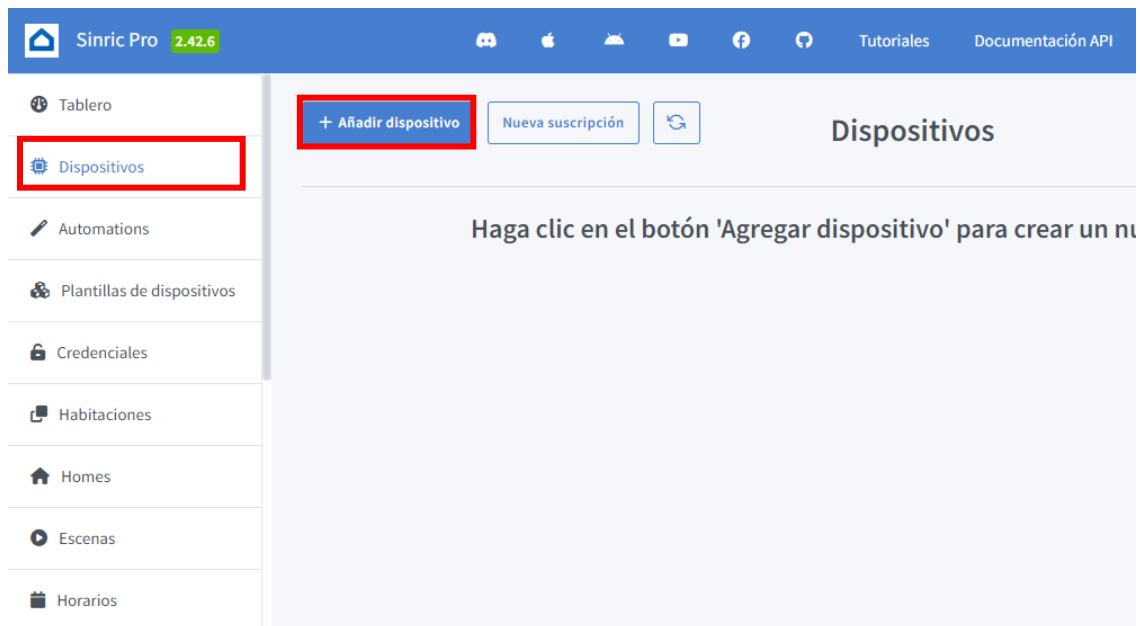


Ilustración 3.4 – Añadir un dispositivo

Seguimos la configuración visualizada en la *Ilustración 3.5*. Después le damos click en “Próxima”.

The screenshot shows the first step of the configuration process, 'Información del dispositivo'. At the top, there are four tabs: '1 Información del dispositivo' (active), '2 Notificaciones', '3 Temporizadores', and '4 Otro'. Below the tabs, there are several input fields: 'Nombre del dispositivo' with the value 'ESPLed', 'Descripción' with the value 'Led con SinricPRO', 'Tipo de dispositivo' with a dropdown menu showing 'Switch', 'Clave de aplicación' with a dropdown menu showing 'default', and 'Habitación' with a dropdown menu showing 'Living Room (Home)'. At the bottom, there is a blue button labeled 'Próxima' and two links: '¿Cómo conectar mi dispositivo?' and 'Expresiones para Alexa y Google Home'.

Ilustración 3.5 – Configuración del dispositivo

A continuación, le damos click en “Próxima”, nos aparecerán tres ventanas más, las cuales son “Notificaciones”, “Temporizadores” y “Otro”, cuando estas aparezcan no modificamos ningún valor y simplemente le damos en “Próxima” hasta que lleguemos a la ventana “Otro” y le damos click en “Salvar” como se muestra en la *Ilustración 3.6*.

The screenshot shows the fourth step of the configuration process, 'Otro'. At the top, there are four tabs: '1 Información del dispositivo', '2 Notificaciones', '3 Temporizadores', and '4 Otro' (active). Below the tabs, there is a section titled 'Uso de energía' with a toggle switch. Under this section, there are two input fields: 'Stand by Wattage (W)' and 'En potencia (W)'. At the bottom, there is a blue button labeled 'Salvar' and two links: '¿Cómo conectar mi dispositivo?' and 'Expresiones para Alexa y Google Home'.

Ilustración 3.6 – “Salvar” la configuración.

Una vez que guardamos nuestro dispositivo, nos aparecerán las credenciales de nuestro dispositivo. Se recomienda guardar estos datos ya que se utilizarán en el programa. Ilustración 3.7.



Ilustración 3.7 – Credenciales del dispositivo.

A continuacion, copiamos el siguiente codigo:

```
/*
 * If you encounter any issues:
 * - check the readme.md at https://github.com/sinricpro/esp8266-esp32-
sdk/blob/master/README.md
 * - ensure all dependent libraries are installed
 * - see https://github.com/sinricpro/esp8266-esp32-
sdk/blob/master/README.md#arduinoide
 * - see https://github.com/sinricpro/esp8266-esp32-
sdk/blob/master/README.md#dependencies
 * - open serial monitor and check whats happening
 * - check full user documentation at https://sinricpro.github.io/esp8266-esp32-sdk
 * - visit https://github.com/sinricpro/esp8266-esp32-sdk/issues and check for
existing issues or open a new one
 */

// Uncomment the following line to enable serial debug output
// #define ENABLE_DEBUG

#ifdef ENABLE_DEBUG
    #define DEBUG_ESP_PORT Serial
    #define NODEBUG_WEBSOCKETS
    #define NDEBUG
#endif

#include <Arduino.h>
#if defined(ESP8266)
    #include <ESP8266WiFi.h>
#elif defined(ESP32) || defined(ARDUINO_ARCH_RP2040)
    #include <WiFi.h>
#endif

#include "SinricPro.h"
#include "SinricProSwitch.h"

#define WIFI_SSID "IZZI-AA9A"
#define WIFI_PASS "DCA633B0AA9A"
#define APP_KEY "cdf5e3cc-c379-4a5f-ac7a-3a0cba1c9aee" // Should
look like "de0bxxxx-1x3x-4x3x-ax2x-5dabxxxxxxxx"
#define APP_SECRET "4de7a36c-4e16-487f-bc5d-183e07c8632f-12bff510-f8a3-4a46-
bcad-c2999930f33d" // Should look like "5f36xxx-x3x7-4x3x-xexe-e86724a9xxx-
4c4axxxx-3x3x-x5xe-x9x3-333d65xxxxx"
#define SWITCH_ID "662df0d87c9e6c6fe87696d4" // Should look like
"5dc1564130xxxxxxxxxxxxxxxx"
```

```

#define BAUD_RATE          115200                // Change baudrate to your need

#if defined(ESP8266)
    #define RELAY_PIN      D5                    // Pin where the relay is connected
    (D5 = GPIO 14 on ESP8266)
#elif defined(ESP32) || defined(ARDUINO_ARCH_RP2040)
    #define RELAY_PIN      16                    // Pin where the relay is connected
    (GPIO 16 on ESP32)
#endif

bool onPowerState(const String &deviceId, bool &state) {
    digitalWrite(RELAY_PIN, state);              // set pin state
    return true;                                // request handled properly
}

void setup() {
    pinMode(RELAY_PIN, OUTPUT);                  // set relay-pin to output mode

    #if defined(ESP8266)
        WiFi.setSleepMode(WIFI_NONE_SLEEP);
        WiFi.setAutoReconnect(true);
    #elif defined(ESP32)
        WiFi.setSleep(false);
        WiFi.setAutoReconnect(true);
    #endif

    WiFi.begin(WIFI_SSID, WIFI_PASS);           // start wifi
    while (WiFi.status() != WL_CONNECTED) {
        delay(250);
    }

    SinricProSwitch& mySwitch = SinricPro[SWITCH_ID]; // create new switch device
    mySwitch.onPowerState(onPowerState);           // apply onPowerState
    callback
    SinricPro.begin(APP_KEY, APP_SECRET);         // start SinricPro
}

void loop() {
    SinricPro.handle();                           // handle SinricPro commands
}

```

Una vez que copiamos el código, procedemos a realizar las siguientes modificaciones, usando los datos guardados anteriormente:

SSID y password de nuestra red.

```
#define WIFI_SSID "IZZI-AA9A"
#define WIFI_PASS "DCA633B0AA9A"
#define APP_KEY "cdf5e3cc-c379-4a5f-ac7a-3a0cba1c9aee"
#define APP_SECRET "4de7a36c-4e16-487f-bc5d-183e07c8632f-12bfff510-f8a3-4a46-bcad-c2999930f33d"
#define SWITCH_ID "662df0d87c9e6c6fe87696d4" // Should look like "5dc1564130xxxxxxxxxxxxxx"
#define BAUD_RATE 115200 // Change baudrate to your need
```

Digitamos nuestro Device ID *Digitamos nuestro App Secret*

Ahora procedemos a cargar el código a nuestra ESP32. *Ilustración 3.8.*

Nota: Recuerda elegir el puerto y la board correcta antes de subir el código a la tarjeta.

```
Relay.ino
7  * - open serial monitor and check whats happening
8  * - check full user documentation at https://sinricpro.github.io/esp8266-esp32-sdk
9  * - visit https://github.com/sinricpro/esp8266-esp32-sdk/issues and check for existing issues or open
10 */
11
12 // Uncomment the following line to enable serial debug output
13 // #define ENABLE_DEBUG
14
15 #ifdef ENABLE_DEBUG
16     #define DEBUG_ESP_PORT Serial
17     #define NODEBUG_WEBSOCKETS
18     #define NDEB
19 #endif
20
21 #include <Arduino.h>
22 #if defined(ESP8266)
23     #include <ESP8266WiFi.h>
24 #elif defined(ESP32) || defined(ARDUINO_ARCH_RP2040)
25     #include <WiFi.h>

```

```
Output
Writing at 0x000c9f3f... (88 %)
Writing at 0x000d392f... (91 %)
Writing at 0x000d8e1b... (94 %)
Writing at 0x000de6fc... (97 %)
Writing at 0x000e3d8b... (100 %)
Wrote 884832 bytes (569093 compressed) at 0x00010000 in 8.7 seconds (effective 811.3 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
```

Ilustración 3.8 – Subir el código a nuestra ESP32

Una vez que subimos el código podemos corroborar que nuestro dispositivo este en línea. Para esto nos dirigimos a Sinric Pro, seleccionamos el “Tablero” y a continuación podremos visualizar que nuestro dispositivo se encuentra en línea. Incluso podremos encender o apagar nuestro LED desde este apartado. *Ilustración 3.9.*

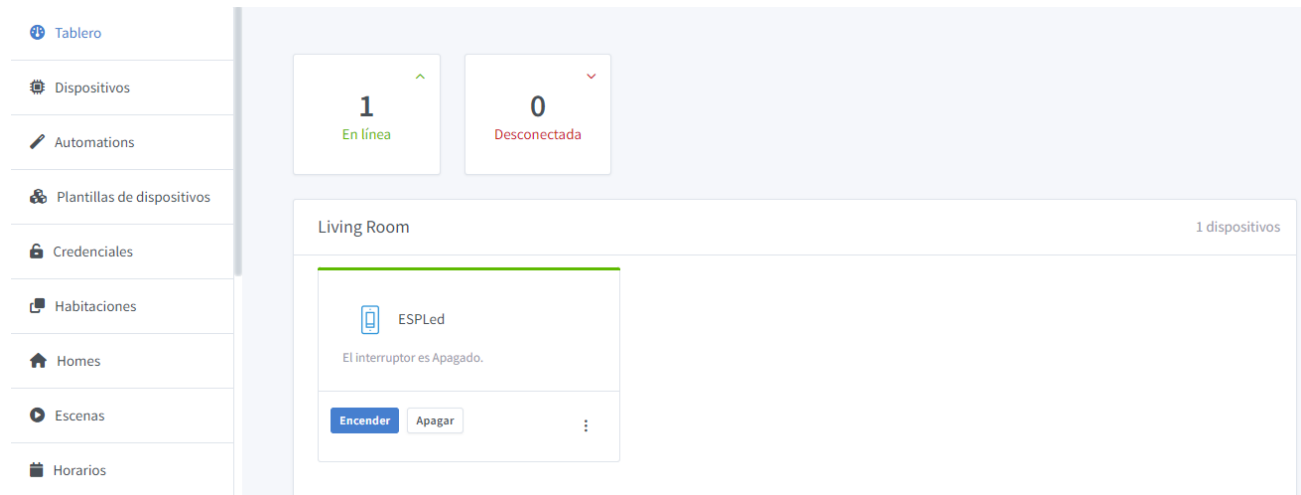


Ilustración 3.9 – Dispositivo en línea

Asimismo, es posible controlar este LED desde la aplicación móvil de Sinric Pro, para esto nos dirigimos a la PlayStore y descargamos la aplicación Sinric Pro. *Ilustración 3.10.*

Enlace: <https://play.google.com/store/apps/details?id=pro.sinric&hl=es&gl=US>

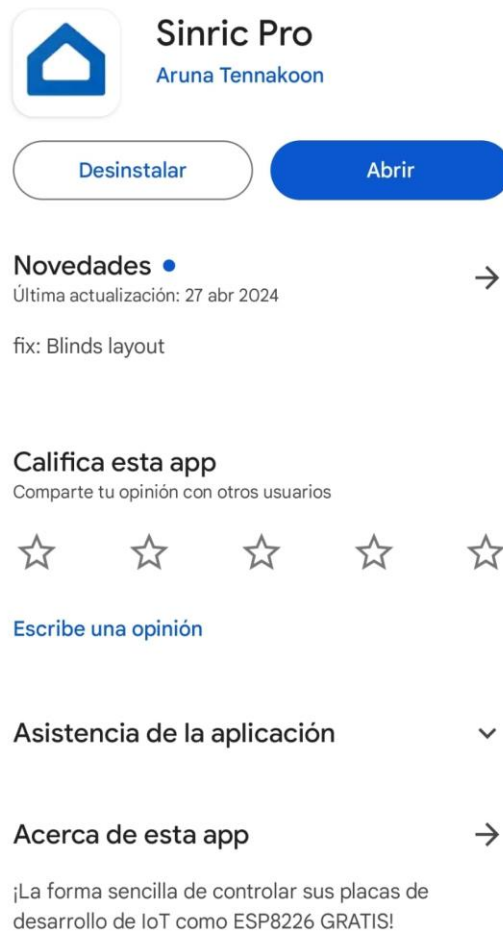


Ilustración 3.10 – App de Sinric Pro

Abrimos la aplicación e ingresamos con nuestras credenciales. Nos mandará a la pantalla principal de la aplicación, aquí podremos observar nuestro dispositivo y el estado en línea. *Ilustración 3.11.*

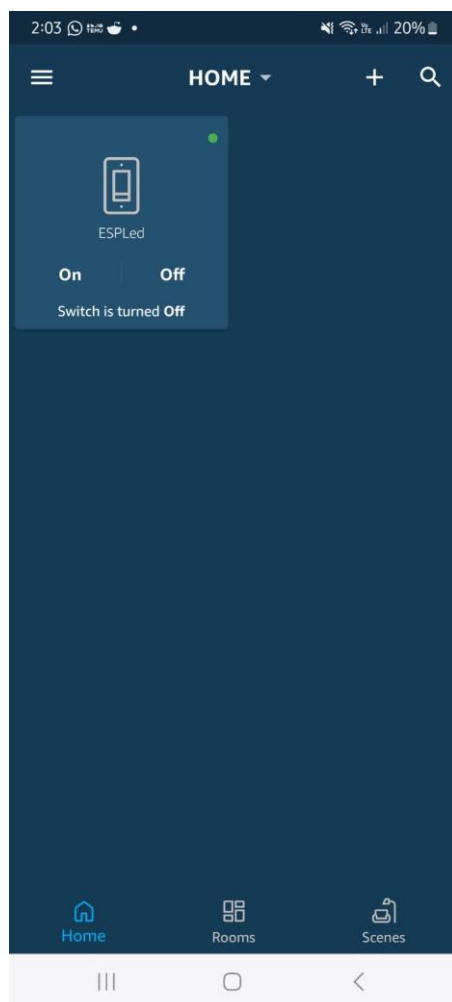


Ilustración 3.11 – Aplicación de Sinric Pro

Seleccionamos este dispositivo y se nos abrirá el siguiente menú, en el cual podremos encender y apagar el LED mediante el botón táctil como se muestra en la *Ilustración 3.12*.

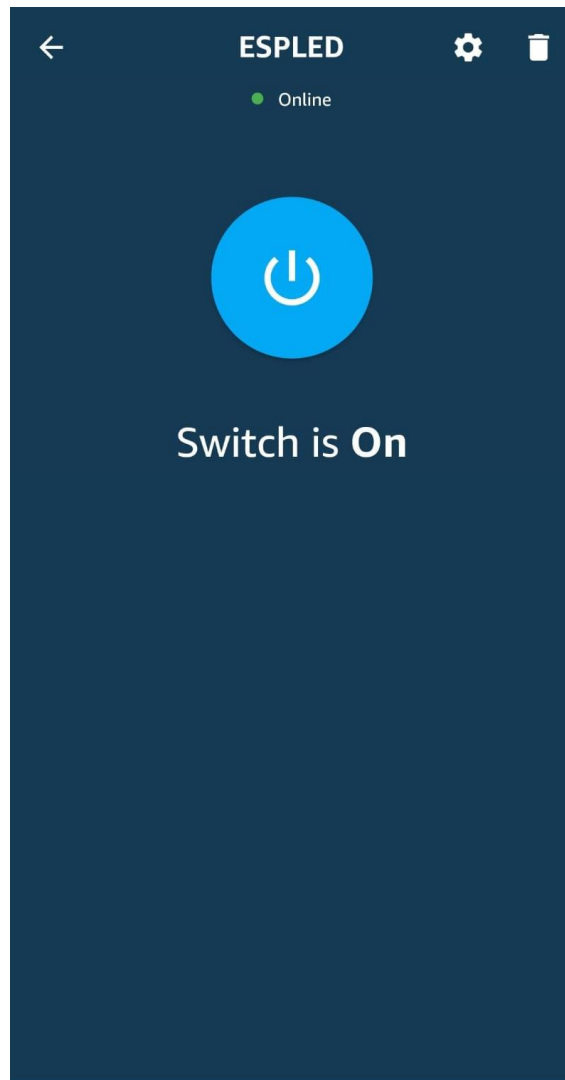


Ilustración 3.12 – Encendido y apagado de LED mediante la App

A continuación, nos dirigimos a la aplicación Amazon Alexa. Una vez dentro, nos dirigimos al apartado “Mas”. *Ilustración 3.13.*

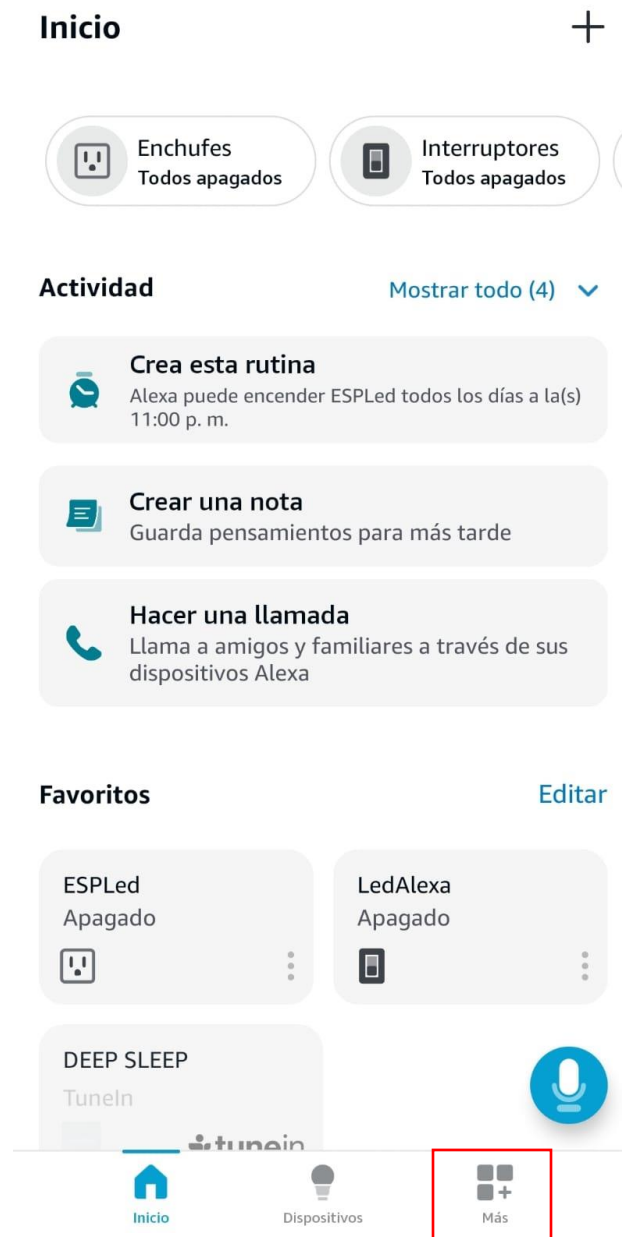


Ilustración 3.13 – Apartado Más

En este apartado, nos dirigimos a la sección de “Skills y Juegos” como se muestra en la *Ilustración 3.14*.

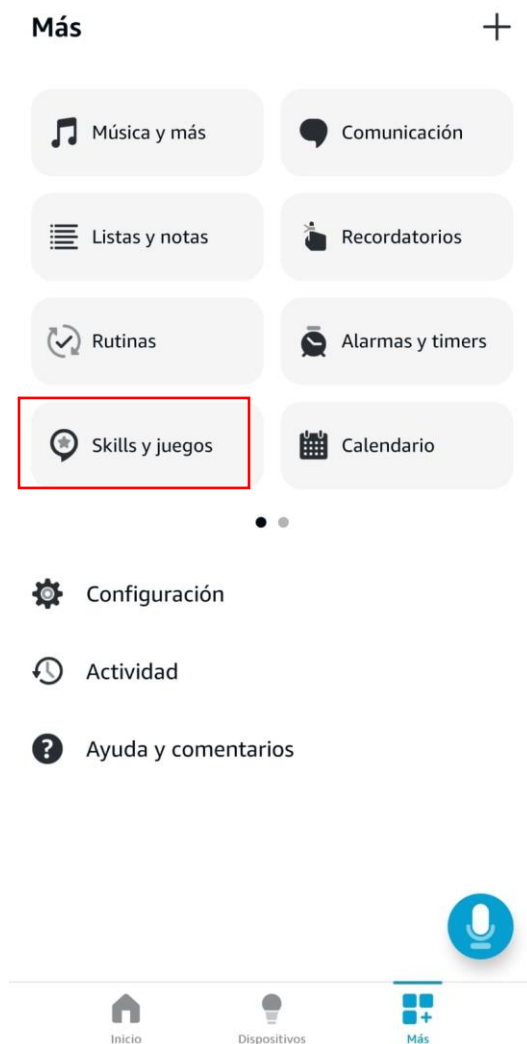


Ilustración 3.14 – Skills y Juegos

Después, buscamos la aplicación “Sinric Pro” y la seleccionamos como se aprecia en la *Ilustración 3.15*.

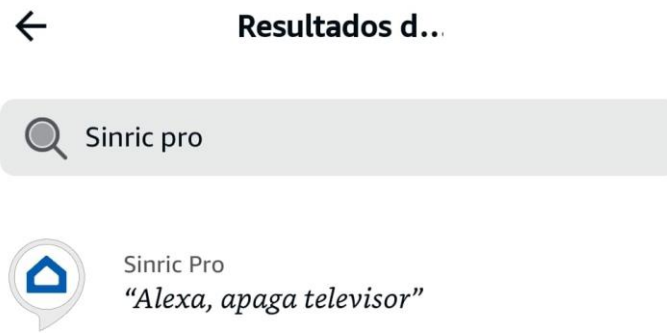


Ilustración 3.15 – Elegir la Skill de Sinric Pro

Ahora procedemos a darle “Enable to Use”, nos abrirá una página que nos pedirá iniciar sesión, por lo que procedemos a digitar nuestras credenciales. *Ilustración 3.16*.

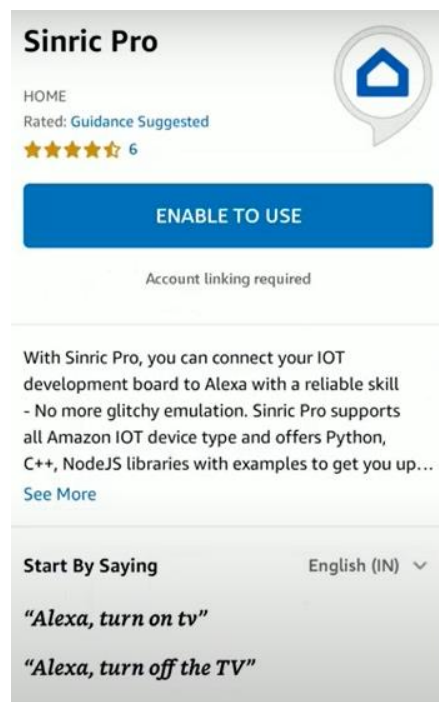


Ilustración 3.16 – Activar la Skill

Una vez que digitemos nuestras credenciales la aplicación estará correctamente vinculada. Esto puede apreciarse en la *Ilustración 3.17*.



Ilustración 3.17 – Exitosa vinculación con Sinric Pro

Una vez hecho esto, se nos mandará a la pantalla principal de Alexa, automáticamente se nos detectará el LED conectado a nuestro protoboard. *Ilustración 3.18*

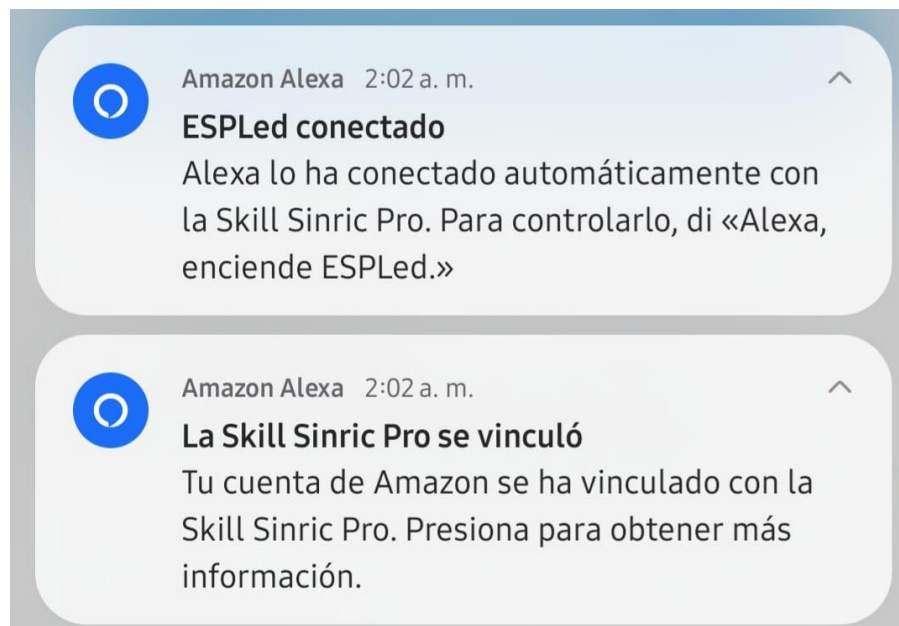


Ilustración 3.18 – Alexa detectará nuestros dispositivos vinculados a Sinric Pro

En la aplicación Alexa nos aparecerá el dispositivo agregado. Ahora podemos controlar el LED mediante comandos de voz, por ejemplo, diciendo "Alexa, enciende ESPLed". También es posible controlarlo desde la aplicación de Alexa mediante el interruptor virtual que nos permite encender o apagar el LED con un toque. *Ilustración 3.19.*



Ilustración 3.19 – Encendido o apagado del LED

CONCLUSIONES

[illegible]

PRÁCTICA IV – SINRIC CON GOOGLE HOME



OBJETIVO DE LA UNIDAD:

Conocerá los estándares de comunicación más utilizados en sistemas de IoT para implementar una aplicación funcional.

OBJETIVOS DE LA PRACTICA:

Explorar y poner en práctica la integración de dispositivos IoT con servicios en la nube utilizando la aplicación Sinric Pro, y enlazarlos con la aplicación “Google Home” para lograr un control remoto de estos dispositivos.

ANTECEDENTES

Si deseamos conectar nuestro ESP32 o cualquier otra tarjeta de desarrollo popular a la nube para poder enviar y recibir datos, una de las soluciones más accesibles y prácticas es utilizar el IoT junto con la plataforma Sinric Pro.

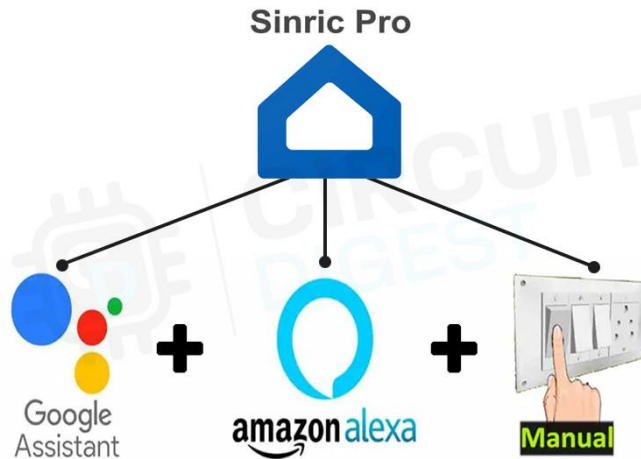


Ilustración 4.1 – Sinric Pro

Los usos y características de esta plataforma son:

- **Control Remoto de Dispositivos:** Permite encender, apagar o ajustar dispositivos conectados desde cualquier lugar a través de la aplicación móvil de Sinric Pro o mediante comandos de voz usando Amazon Alexa o Google Assistant.
- **Monitorización de Sensores:** Facilita la monitorización en tiempo real de los datos provenientes de sensores conectados al ESP32 un panel de control en la plataforma Sinric Pro.
- **Automatización del Hogar:** Ayuda a crear rutinas y automatizaciones para que los dispositivos funcionen de manera autónoma según condiciones específicas, como horarios predefinidos o la lectura de sensores.
- **Cargas Over-The-Air (OTA):** Permite actualizar el firmware del ESP32 de forma remota sin necesidad de estar físicamente conectado al dispositivo, lo que facilita el mantenimiento y la mejora continua de los proyectos.
- **Sincronización de Dispositivos:** Ofrece la capacidad de sincronizar múltiples dispositivos conectados, lo que facilita la comunicación y el control entre ellos con una configuración mínima.
- **Notificaciones y Alertas:** Configura notificaciones para recibir alertas en tiempo real sobre el estado de los dispositivos o cambios en las lecturas de los sensores, mejorando la seguridad y el seguimiento del entorno.

- **Interfaz Amigable para el Usuario:** Proporciona una interfaz intuitiva y fácil de usar para configurar y controlar los dispositivos, lo que reduce la barrera de entrada para usuarios sin experiencia técnica.

MATERIAL A UTILIZAR

- NodeMCU-32
- LED de cualquier color
- Jumpers Macho-Macho
- Resistencia de 330Ω
- Protoboard

SOFTWARE NECESARIO

SINRIC Pro:

<https://sinric.pro/>

Google Home:

https://play.google.com/store/apps/details?id=com.google.android.apps.chromecast.app&hl=es_MX&gl=US

DESARROLLO DE LA PRÁCTICA

Práctica 4. Encendido de un led a través de Google Home mediante SINRIC.

4.1 Implementar el circuito del diagrama que se muestra en la *Ilustración 4.2*

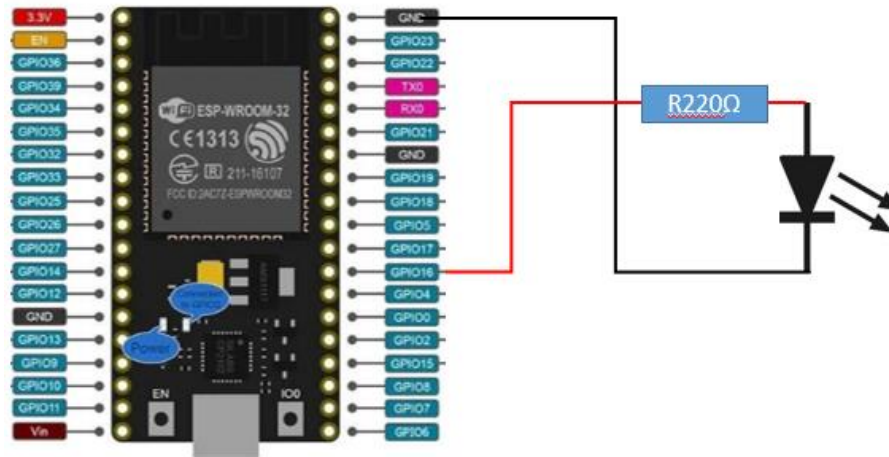


Ilustración 4.2 – Diagrama a implementar

4.2 Configuración de la plataforma Sinric Pro

La configuración necesaria se realizó en la práctica anterior. Por lo que, el dispositivo debe estar en línea, para revisar esto, nos dirigimos a la pantalla principal de Sinric, ubicamos la pestaña “Dashboard” y lo seleccionamos. En este apartado debemos observar que nuestro dispositivo está en línea. *Ilustración 4.3*

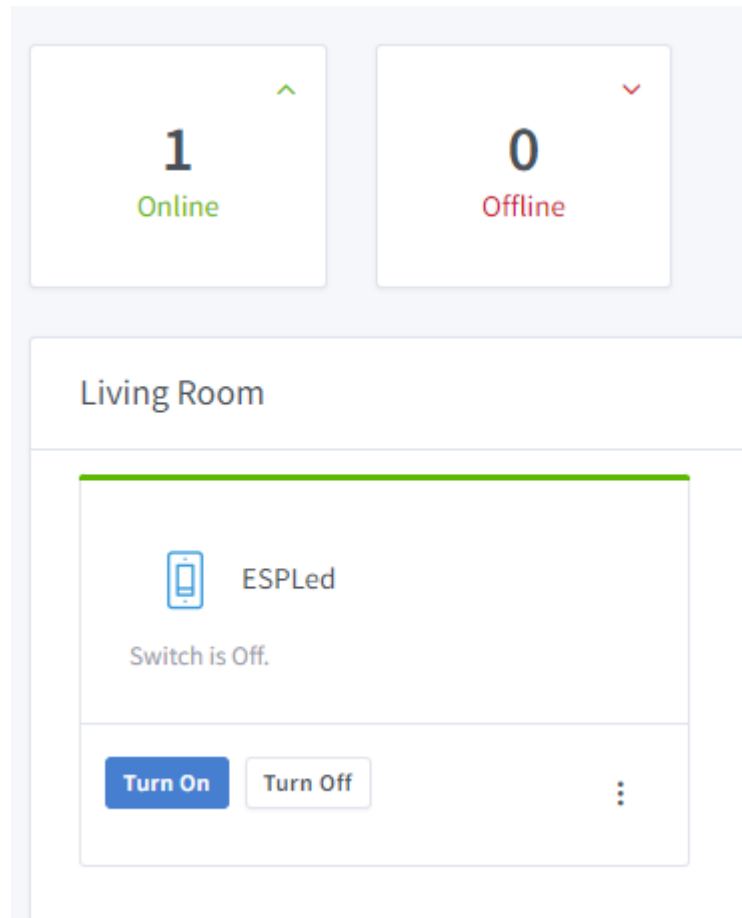


Ilustración 4.3 – Dispositivo en línea.

Una vez que nuestro dispositivo se encuentre en línea nos iremos a la Play Store, buscamos y descargamos la aplicación “Google Home”. *Ilustración 4.4.*



Ilustración 4.4 – App de Google Home

Nos pedirá crear una cuenta, una vez creada nos dirigirá a la pantalla principal. Una vez aquí, nos dirigimos al apartado “Config” y seguidamente seleccionamos “Funciona con Google”. *Ilustración 4.5.*

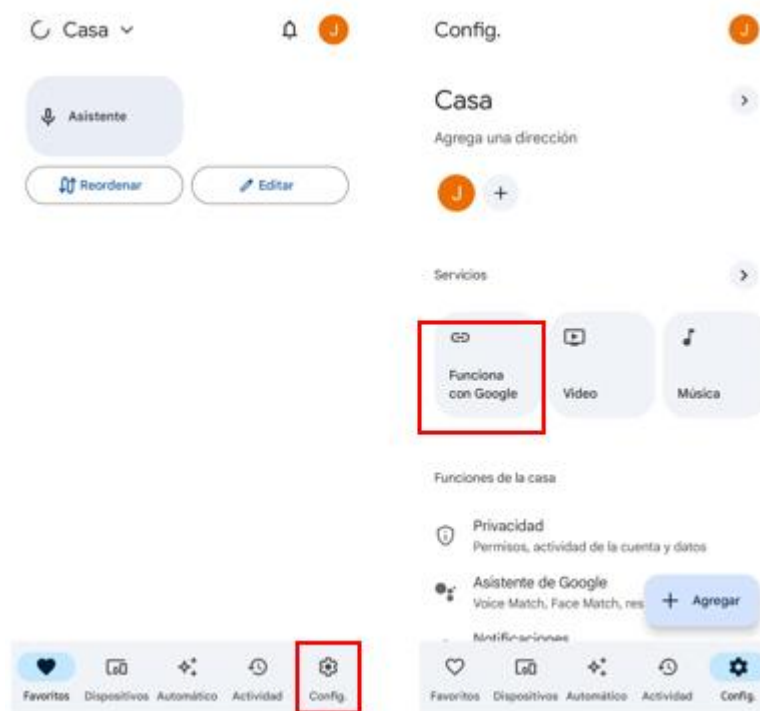


Ilustración 4.5 – Seleccionamos el servicio “Funciona con Google”

Una vez aquí, buscamos y seleccionamos “Sinric Pro”. *Ilustración 4.6.*

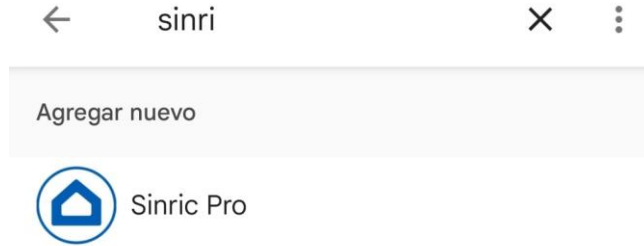


Ilustración 4.6 – Sinric Pro

Habilitamos la vinculación poniendo las credenciales de nuestra cuenta. *Ilustración 4.7.*

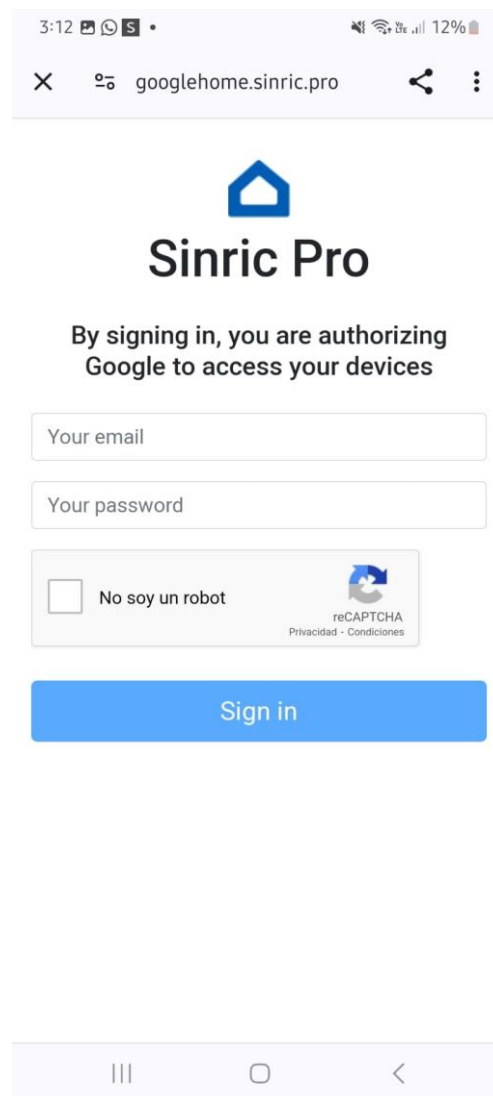


Ilustración 4.7 – Vincular Sinric Pro con Google Home

Una vez vinculemos Google Home con Sinric se nos redirigirá a la pantalla principal, podremos notar que se ha detectado y agregado automáticamente nuestro LED como se observa en la *Ilustración 4.8*.

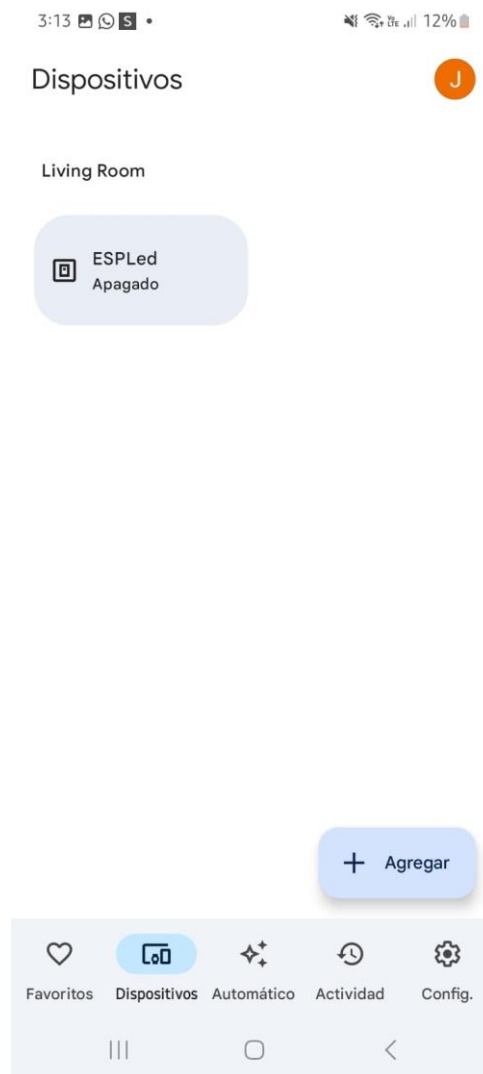


Ilustración 4.8 – LED correctamente agregado

Entramos al LED detectado y nos aparecerá la siguiente ventana, en la cual podremos apagar o encender el LED mediante un interruptor virtual como se muestra en la *Ilustración 4.9*.



Ilustración 4.9 – Ventana del LED

En esta misma pestaña, nos dirigimos a “Configuración”. Aquí podremos cambiar algunos valores de nuestro LED como el nombre, tipo de dispositivo, etc. *Ilustración 4.10*.

The image shows a mobile application interface for configuring a Sinric Pro device. At the top, there is a navigation bar with a back arrow, the title 'Config. del dispositivo', and a vertical ellipsis menu icon. Below the navigation bar, the 'General' tab is selected. The configuration fields are as follows: 'Nombre' is set to 'LedCasa'; 'Tipo de dispositivo' is set to 'Interruptor'; 'Casa' is set to 'Casa'; and 'Habitación' is set to 'Living Room'. A horizontal separator line is below these fields. Below the separator, there is a button labeled 'Desvincular Sinric Pro'. Another horizontal separator line is below the button. At the bottom, the connection details are displayed: 'Conectado mediante: Sinric Pro', 'Fabricante: Sinric Pro', 'Nombre de Sinric Pro: ESPLed', and 'Tipo de dispositivo: Interruptor'.

← Config. del dispositivo ⋮

General

Nombre
LedCasa

Tipo de dispositivo
Interruptor

Casa
Casa

Habitación
Living Room

Desvincular Sinric Pro

Conectado mediante: Sinric Pro
Fabricante: Sinric Pro
Nombre de Sinric Pro: ESPLed
Tipo de dispositivo: Interruptor

Ilustración 4.10 – Ventana de configuración

Recomendación: Es recomendable asignarle un nombre a nuestro LED que sea fácil de pronunciar. Si el nombre tiene palabras complejas es posible que nuestro asistente de voz no reconozca el dispositivo a la hora de ordenarle encenderlo o apagarlo.

Asimismo, es posible encender nuestro LED mediante el asistente de voz. Por ejemplo, podemos decir “Hey Google, enciende LedCasa”. El asistente ejecutará la acción solicitada como se observa en la *Ilustración 4.11*.

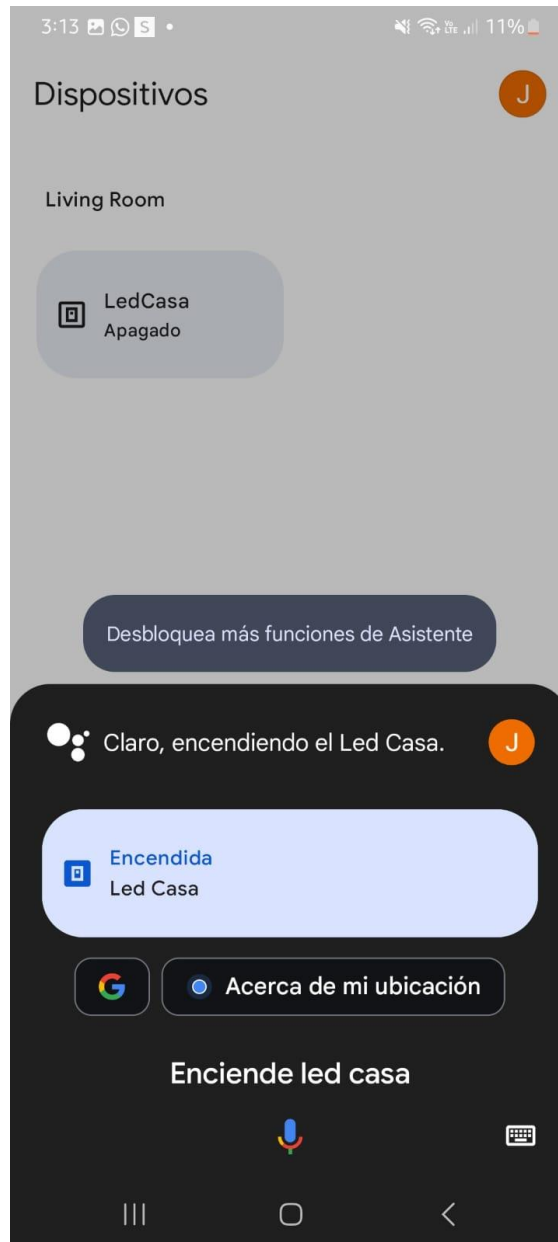


Ilustración 4.11 – Encender el LED mediante el asistente de voz

CONCLUSIONES

This image shows a blank sheet of white paper with horizontal blue ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

PRÁCTICA V – ESP RAINMAKER CON ALEXA



OBJETIVO DE LA UNIDAD:

Conocerá los estándares de comunicación más utilizados en sistemas de IoT para implementar una aplicación funcional.

OBJETIVOS DE LA PRACTICA:

Comprender y aplicar la integración de dispositivos IoT con servicios en la nube mediante la utilización de ESP RainMaker, y conectarlos a la aplicación 'Amazon Alexa' para habilitar el control remoto de dichos dispositivos.

ANTECEDENTES

ESP RainMaker es una plataforma desarrollada por Espressif Systems para facilitar el desarrollo y la implementación de soluciones IoT. Espressif Systems, una compañía conocida por sus chips Wi-Fi y Bluetooth, como el ESP8266 y el ESP32, ha sido un actor clave en el mercado de IoT, proporcionando hardware accesible y fácil de usar para desarrolladores y aficionados.



Ilustración 5.1 – ESP RainMaker

Características clave:

- **Configuración Simplificada:** ESP RainMaker simplifica el proceso de configuración de dispositivos IoT, eliminando muchas de las complejidades asociadas con la configuración de red y la conexión a la nube.
- **Control Remoto:** Los dispositivos integrados con ESP RainMaker pueden ser controlados remotamente a través de la aplicación ESP RainMaker, Google Home y Amazon Alexa.
- **Seguridad y Privacidad:** Espressif ha puesto un fuerte énfasis en la seguridad, asegurando que las comunicaciones entre los dispositivos y la nube estén encriptadas y sean seguras.
- **Actualizaciones OTA (Over-The-Air):** Permite la actualización de firmware de los dispositivos de manera remota, facilitando el mantenimiento y la implementación de nuevas funcionalidades sin necesidad de intervención física.

MATERIAL A UTILIZAR

- NodeMCU-32
- 2 LEDs de cualquier color
- Jumpers Macho-Macho
- 2 Resistencias de 330Ω
- Protoboard

SOFTWARE NECESARIO

ESP RainMaker:

<https://play.google.com/store/apps/details?id=com.espressif.rainmaker&hl=es&gl=US>

Amazon Alexa:

<https://play.google.com/store/apps/details?id=com.amazon.dee.app&hl=es&gl=US>

Práctica 5. Encendido de LEDs a través de Google Home y ESP RainMaker.

5.1 Implementar el circuito del diagrama que se muestra en la *Ilustración 5.2*

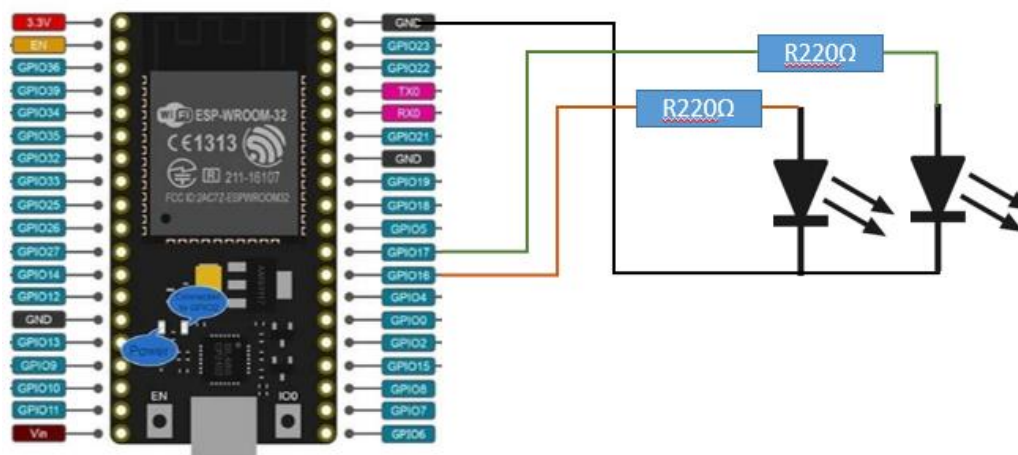


Ilustración 5.2 – Diagrama a implementar

5.2 Código para la Practica

```
/* *****  
 *   TITLE: ESP RainMaker + Manual Switch control 4 Relays using ESP32 (Real time  
feedback + no WiFi control)  
 *   Click on the following links to learn more.  
 *   YouTube Video: https://youtu.be/IMb52-h4tzQ  
 *   Related Blog : https://iotcircuithub.com/esp32-projects/  
 *   by Tech StudyCell  
 *   Preferences--> Additional boards Manager URLs :  
 *   http://arduino.esp8266.com/stable/package\_esp8266com\_index.json,https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package\_esp32\_index.json  
 *  
 *   Download Board ESP32 (2.0.3) :  
 *  
 *   Download the libraries :  
 *   N/A  
 * *****  
/  
  
#include "RMaker.h"  
#include "WiFi.h"  
#include "WiFiProv.h"  
  
const char *service_name = "JuanChi";  
const char *pop = "1234567";  
  
// define the Device Names  
char deviceName_1[] = "Switch1";  
char deviceName_2[] = "Switch2";  
char deviceName_3[] = "Switch3";  
char deviceName_4[] = "Switch4";  
  
// define the GPIO connected with Relays and switches  
static uint8_t RelayPin1 = 16; //D16  
static uint8_t RelayPin2 = 17; //D17  
static uint8_t RelayPin3 = 21; //D21  
static uint8_t RelayPin4 = 19; //D19  
  
static uint8_t SwitchPin1 = 26; //D26  
static uint8_t SwitchPin2 = 12; //D12  
static uint8_t SwitchPin3 = 14; //D14  
static uint8_t SwitchPin4 = 27; //D27  
  
static uint8_t wifiled = 2; //D2
```

```

static uint8_t gpio_reset = 0;

/* Variable for reading pin status*/
// Relay State
bool toggleState_1 = LOW; //Define integer to remember the toggle state for relay 1
bool toggleState_2 = LOW; //Define integer to remember the toggle state for relay 2
bool toggleState_3 = LOW; //Define integer to remember the toggle state for relay 3
bool toggleState_4 = LOW; //Define integer to remember the toggle state for relay 4

// Switch State
bool SwitchState_1 = LOW;
bool SwitchState_2 = LOW;
bool SwitchState_3 = LOW;
bool SwitchState_4 = LOW;

//The framework provides some standard device types like switch, lightbulb, fan,
temperature sensor.
static Switch my_switch1(deviceName_1, &RelayPin1);
static Switch my_switch2(deviceName_2, &RelayPin2);
static Switch my_switch3(deviceName_3, &RelayPin3);
static Switch my_switch4(deviceName_4, &RelayPin4);

void sysProvEvent(arduino_event_t *sys_event)
{
    switch (sys_event->event_id) {
        case ARDUINO_EVENT_PROV_START:
#ifdef CONFIG_IDF_TARGET_ESP32
            Serial.printf("\nProvisioning Started with name \"%s\" and PoP \"%s\" on
BLE\n", service_name, pop);
            printQR(service_name, pop, "ble");
#else
            Serial.printf("\nProvisioning Started with name \"%s\" and PoP \"%s\" on
SoftAP\n", service_name, pop);
            printQR(service_name, pop, "softap");
#endif
            break;
        case ARDUINO_EVENT_WIFI_STA_CONNECTED:
            Serial.printf("\nConnected to Wi-Fi!\n");
            digitalWrite(wifiLed, true);
            break;
    }
}

void write_callback(Device *device, Param *param, const param_val_t val, void
*priv_data, write_ctx_t *ctx)

```

```

{
    const char *device_name = device->getDeviceName();
    const char *param_name = param->getParamName();

    if(strcmp(device_name, deviceName_1) == 0) {

        Serial.printf("Lightbulb = %s\n", val.val.b? "true" : "false");

        if(strcmp(param_name, "Power") == 0) {
            Serial.printf("Received value = %s for %s - %s\n", val.val.b? "true" :
"false", device_name, param_name);
            toggleState_1 = val.val.b;
            (toggleState_1 == false) ? digitalWrite(RelayPin1, HIGH) :
digitalWrite(RelayPin1, LOW);
            param->updateAndReport(val);
        }

    } else if(strcmp(device_name, deviceName_2) == 0) {

        Serial.printf("Switch value = %s\n", val.val.b? "true" : "false");

        if(strcmp(param_name, "Power") == 0) {
            Serial.printf("Received value = %s for %s - %s\n", val.val.b? "true" :
"false", device_name, param_name);
            toggleState_2 = val.val.b;
            (toggleState_2 == false) ? digitalWrite(RelayPin2, HIGH) :
digitalWrite(RelayPin2, LOW);
            param->updateAndReport(val);
        }

    } else if(strcmp(device_name, deviceName_3) == 0) {

        Serial.printf("Switch value = %s\n", val.val.b? "true" : "false");

        if(strcmp(param_name, "Power") == 0) {
            Serial.printf("Received value = %s for %s - %s\n", val.val.b? "true" :
"false", device_name, param_name);
            toggleState_3 = val.val.b;
            (toggleState_3 == false) ? digitalWrite(RelayPin3, HIGH) :
digitalWrite(RelayPin3, LOW);
            param->updateAndReport(val);
        }

    } else if(strcmp(device_name, deviceName_4) == 0) {

```

```

    Serial.printf("Switch value = %s\n", val.val.b? "true" : "false");

    if(strcmp(param_name, "Power") == 0) {
        Serial.printf("Received value = %s for %s - %s\n", val.val.b? "true" :
"false", device_name, param_name);
        toggleState_4 = val.val.b;
        (toggleState_4 == false) ? digitalWrite(RelayPin4, HIGH) :
digitalWrite(RelayPin4, LOW);
        param->updateAndReport(val);
    }
}

void manual_control()
{
    if (digitalRead(SwitchPin1) == LOW && SwitchState_1 == LOW) {
        digitalWrite(RelayPin1, LOW);
        toggleState_1 = 1;
        SwitchState_1 = HIGH;
        my_switch1.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, toggleState_1);
        Serial.println("Switch-1 on");
    }
    if (digitalRead(SwitchPin1) == HIGH && SwitchState_1 == HIGH) {
        digitalWrite(RelayPin1, HIGH);
        toggleState_1 = 0;
        SwitchState_1 = LOW;
        my_switch1.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, toggleState_1);
        Serial.println("Switch-1 off");
    }
    if (digitalRead(SwitchPin2) == LOW && SwitchState_2 == LOW) {
        digitalWrite(RelayPin2, LOW);
        toggleState_2 = 1;
        SwitchState_2 = HIGH;
        my_switch2.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, toggleState_2);
        Serial.println("Switch-2 on");
    }
    if (digitalRead(SwitchPin2) == HIGH && SwitchState_2 == HIGH) {
        digitalWrite(RelayPin2, HIGH);
        toggleState_2 = 0;
        SwitchState_2 = LOW;
        my_switch2.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, toggleState_2);
        Serial.println("Switch-2 off");
    }
    if (digitalRead(SwitchPin3) == LOW && SwitchState_3 == LOW) {
        digitalWrite(RelayPin3, LOW);

```

```

toggleState_3 = 1;
SwitchState_3 = HIGH;
my_switch3.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, toggleState_3);
Serial.println("Switch-3 on");
}
if (digitalRead(SwitchPin3) == HIGH && SwitchState_3 == HIGH) {
    digitalWrite(RelayPin3, HIGH);
    toggleState_3 = 0;
    SwitchState_3 = LOW;
    my_switch3.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, toggleState_3);
    Serial.println("Switch-3 off");
}
if (digitalRead(SwitchPin4) == LOW && SwitchState_4 == LOW) {
    digitalWrite(RelayPin4, LOW);
    toggleState_4 = 1;
    SwitchState_4 = HIGH;
    my_switch4.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, toggleState_4);
    Serial.println("Switch-4 on");
}
if (digitalRead(SwitchPin4) == HIGH && SwitchState_4 == HIGH) {
    digitalWrite(RelayPin4, HIGH);
    toggleState_4 = 0;
    SwitchState_4 = LOW;
    my_switch4.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, toggleState_4);
    Serial.println("Switch-4 off");
}
}

void setup()
{
    uint32_t chipId = 0;

    Serial.begin(115200);

    // Set the Relays GPIOs as output mode
    pinMode(RelayPin1, OUTPUT);
    pinMode(RelayPin2, OUTPUT);
    pinMode(RelayPin3, OUTPUT);
    pinMode(RelayPin4, OUTPUT);
    pinMode(wifiLed, OUTPUT);

    // Configure the input GPIOs
    pinMode(SwitchPin1, INPUT_PULLUP);
    pinMode(SwitchPin2, INPUT_PULLUP);
    pinMode(SwitchPin3, INPUT_PULLUP);

```

```

pinMode(SwitchPin4, INPUT_PULLUP);
pinMode(gpio_reset, INPUT);

// Write to the GPIOs the default state on booting
digitalWrite(RelayPin1, !toggleState_1);
digitalWrite(RelayPin2, !toggleState_2);
digitalWrite(RelayPin3, !toggleState_3);
digitalWrite(RelayPin4, !toggleState_4);
digitalWrite(wifiLed, LOW);

Node my_node;
my_node = RMaker.initNode("ESP32_Relay_4");

//Standard switch device
my_switch1.addCb(write_callback);
my_switch2.addCb(write_callback);
my_switch3.addCb(write_callback);
my_switch4.addCb(write_callback);

//Add switch device to the node
my_node.addDevice(my_switch1);
my_node.addDevice(my_switch2);
my_node.addDevice(my_switch3);
my_node.addDevice(my_switch4);

//This is optional
RMaker.enableOTA(OTA_USING_PARAMS);
//If you want to enable scheduling, set time zone for your region using
setTimeZone().
//The list of available values are provided here
https://rainmaker.espressif.com/docs/time-service.html
// RMaker.setTimeZone("Asia/Shanghai");
// Alternatively, enable the Timezone service and let the phone apps set the
appropriate timezone
RMaker.enableTZService();
RMaker.enableSchedule();

//Service Name
for(int i=0; i<17; i=i+8) {
    chipId |= ((ESP.getEfuseMac() >> (40 - i)) & 0xff) << i;
}

Serial.printf("\nChip ID:  %d Service Name: %s\n", chipId, service_name);

Serial.printf("\nStarting ESP-RainMaker\n");

```

```

    RMaker.start();

    WiFi.onEvent(sysProvEvent);
#ifdef CONFIG_IDF_TARGET_ESP32
    WiFiProv.beginProvision(WIFI_PROV_SCHEME_BLE,
WIFI_PROV_SCHEME_HANDLER_FREE_BTDM, WIFI_PROV_SECURITY_1, pop, service_name);
#else
    WiFiProv.beginProvision(WIFI_PROV_SCHEME_SOFTAP, WIFI_PROV_SCHEME_HANDLER_NONE,
WIFI_PROV_SECURITY_1, pop, service_name);
#endif

    my_switch1.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, false);
    my_switch2.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, false);
    my_switch3.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, false);
    my_switch4.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, false);
}

void loop()
{
    // Read GPIO00 (external button to reset device
    if(digitalRead(gpio_reset) == LOW) { //Push button pressed
        Serial.printf("Reset Button Pressed!\n");
        // Key debounce handling
        delay(100);
        int startTime = millis();
        while(digitalRead(gpio_reset) == LOW) delay(50);
        int endTime = millis();

        if ((endTime - startTime) > 10000) {
            // If key pressed for more than 10secs, reset all
            Serial.printf("Reset to factory.\n");
            RMakerFactoryReset(2);
        } else if ((endTime - startTime) > 3000) {
            Serial.printf("Reset Wi-Fi.\n");
            // If key pressed for more than 3secs, but less than 10, reset Wi-Fi
            RMakerWiFiReset(2);
        }
    }
    delay(100);

    if (WiFi.status() != WL_CONNECTED)
    {
        //Serial.println("WiFi Not Connected");
        digitalWrite(wifiLed, false);
    }
}

```

```
else
{
    //Serial.println("WiFi Connected");
    digitalWrite(wifiLed, true);
}

manual_control();
}
```

5.3 Desarrollo de la practica

Descargamos la aplicación ESP RainMaker en nuestro dispositivo móvil. *Ilustración 5.3.*

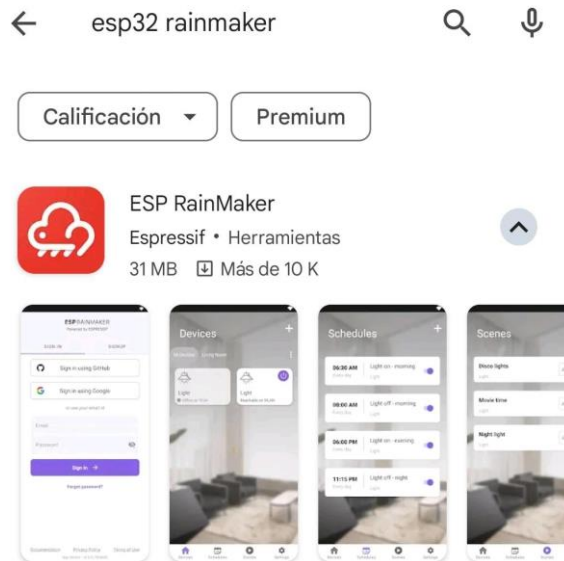


Ilustración 5.3 – App de ESP RainMaker

Una vez instalada, la abrimos, nos pedirá crear una cuenta así que procedemos a realizarlo. Al crear nuestra cuenta se nos redirigirá a la pantalla principal, por el momento no utilizaremos la aplicación, pero será importante para pasos posteriores. *Ilustración 5.4.*

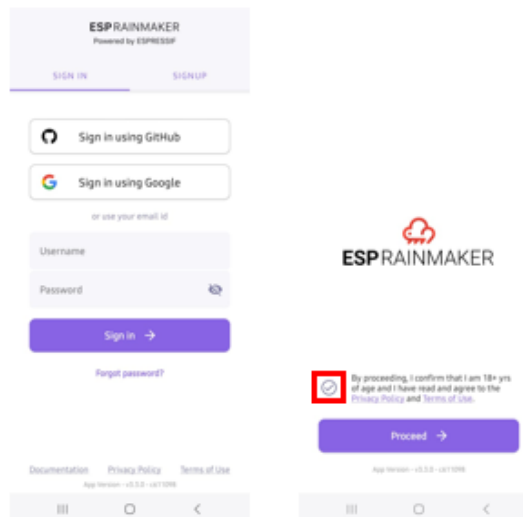
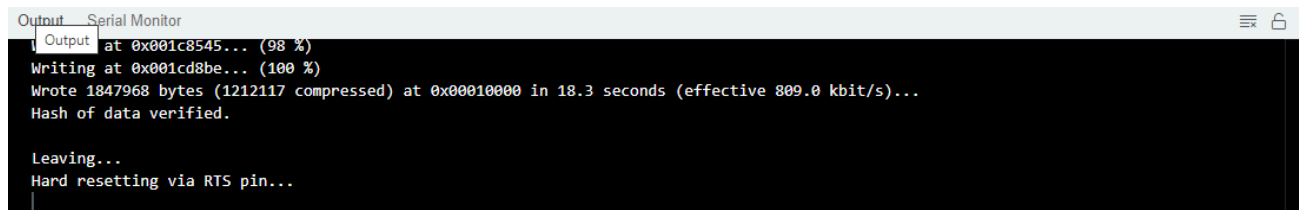


Ilustración 5.4 – Registro en la aplicación

Ahora, procedemos a cargar el código mostrado con anterioridad a nuestra tarjeta, esperamos a que se suba exitosamente. *Ilustración 5.5*



```
Output Serial Monitor
Output at 0x001c8545... (98 %)
Writing at 0x001cd8be... (100 %)
Wrote 1847968 bytes (1212117 compressed) at 0x00010000 in 18.3 seconds (effective 809.0 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
```

Ilustración 5.5 – Subida de código exitosa

Una vez cargado el código nos dirigimos al apartado de “Monitor Serial” como se observa en la *Ilustración 5.6*.

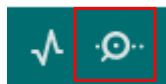


Ilustración 5.6 – Monitor Serial

Seleccionamos la velocidad y el modo correcto para poder visualizar la comunicación serial con nuestra NODEMCU. *Ilustración 5.7*.

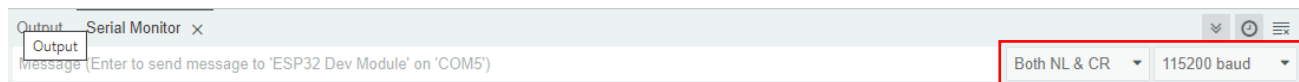


Ilustración 5.7 – Configuraciones para el monitor serial

Ahora procedemos a presionar el botón “BOOT” o “IO0” de nuestra tarjeta por aproximadamente 5 segundos. *Ilustración 5.8*.

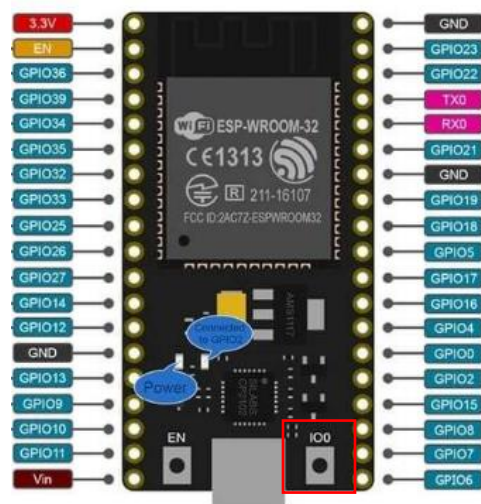


Ilustración 5.8 – Boton IO0

Nos aparecerá el siguiente mensaje, esto quiere decir que presionamos el boton correcto. Recuerda presionar este boton por al menos cinco segundos para luego soltarlo. *Ilustración 5.9.*

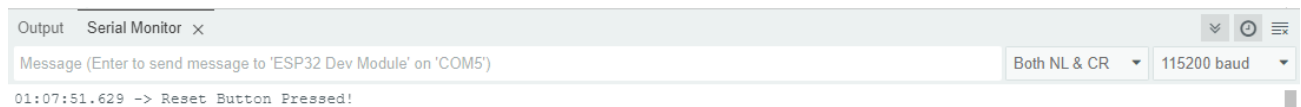


Ilustración 5.9 – Boton presionado correctamente

Una vez hecho esto, nos generará un codigo QR y un enlace, este codigo QR es complicado de escanear, por lo que, es preferible abrir el enlace que se nos muestra al final del parrafo. Este enlace nos dirigirá a una pagina en la cual se podra apreciar este mismo codigo QR de una manera mas clara, por lo que debemos copiar este enlace en nuestro navegador y dirigimos hacia el. *Ilustración 5.10.*



Ilustración 5.10 – Enlace generado

En este enlace nos aparecerá un código QR el cual debemos escanear con nuestra aplicación, la cual instalamos con anterioridad en nuestro dispositivo móvil. *Ilustración 5.11.*



Ilustración 5.11 – Código QR

Abrimos la aplicación, en la pantalla principal nos dirigimos al apartado “Devices” y seleccionamos el ícono “+”. *Ilustración 5.12.*



Ilustración 5.12 – Seleccionar el ícono “+”

Una vez seleccionado, se nos abrirá la cámara, se nos pedirán permisos, aceptamos todos para que la app funcione de manera correcta, ahora escaneamos el código QR que se nos generó en el paso anterior. *Ilustración 5.13.*

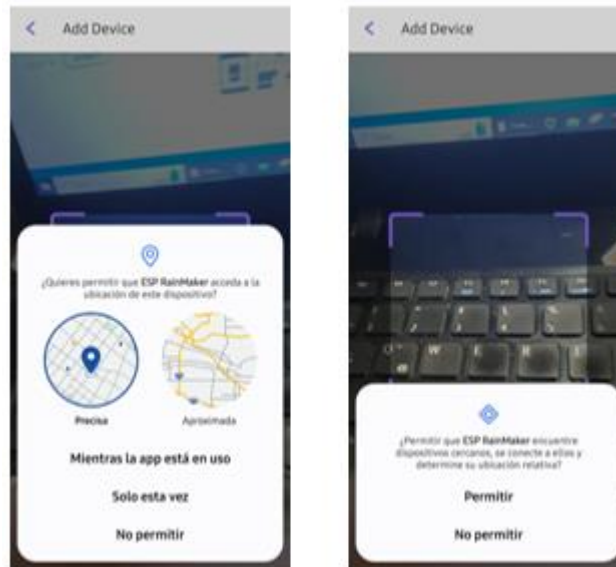


Ilustración 5.13 – Permisos de la app

Una vez que escaneemos el código, aparecerá una ventana en la cual debemos seleccionar nuestra red WiFi e ingresar la contraseña. *Ilustración 5.14.*

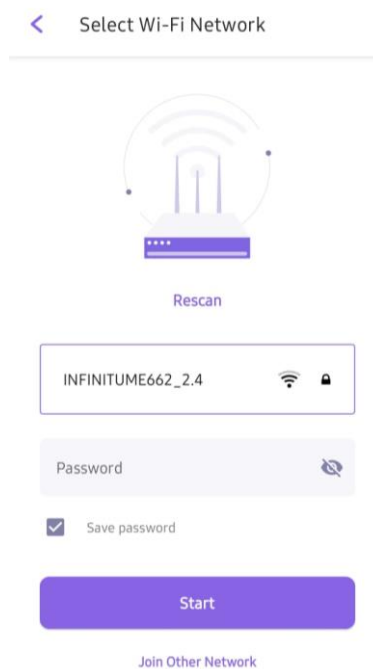


Ilustración 5.14 – Seleccionar nuestra red

Se nos iniciará la vinculación de la aplicación con nuestra tarjeta. Esperamos a que todas las casillas se completen para seguidamente seleccionar “Done”. *Ilustración 5.15.*

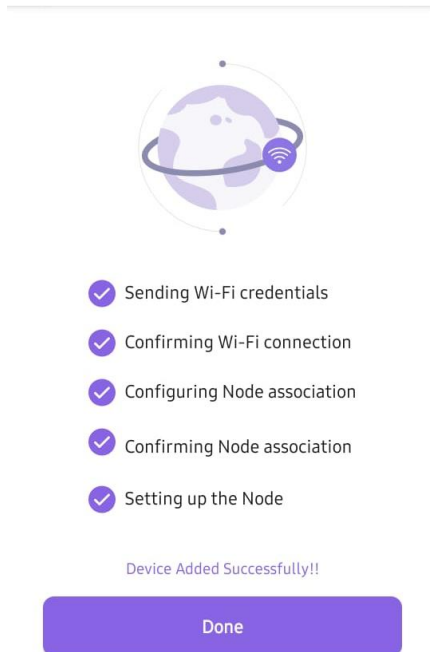


Ilustración 5.15 – Vinculación exitosa

La aplicación se vinculó exitosamente, esto podemos observarlo ya que nos aparecen unas notificaciones indicando que nuestros “Switch” se vincularon correctamente y se encuentran en línea. *Ilustración 5.16.*

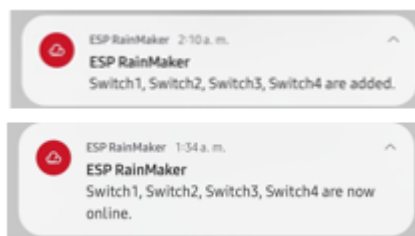


Ilustración 5.16 – Dispositivos añadidos y en línea

Ahora es posible manipular nuestros “Switch” desde la aplicación. *Ilustración 5.17.*

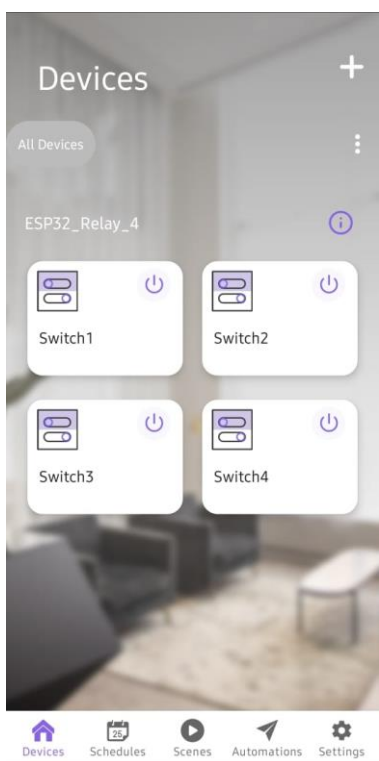


Ilustración 5.17 – “Switches” desde la aplicación

Nos dirigimos a la pestaña “Settings”, para seguidamente seleccionar “Voice Services”. *Ilustración 5.18.*

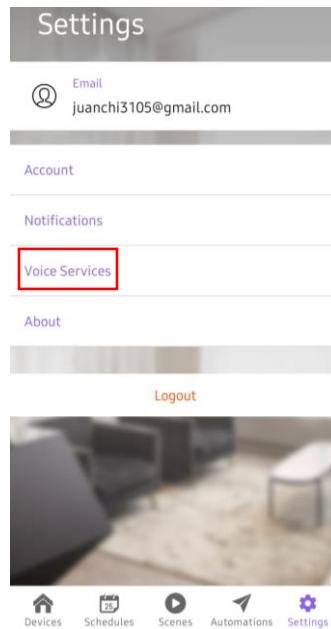


Ilustración 5.18 – Voice Services

Seleccionamos la aplicación “Amazon Alexa”. *Ilustración 5.19.*

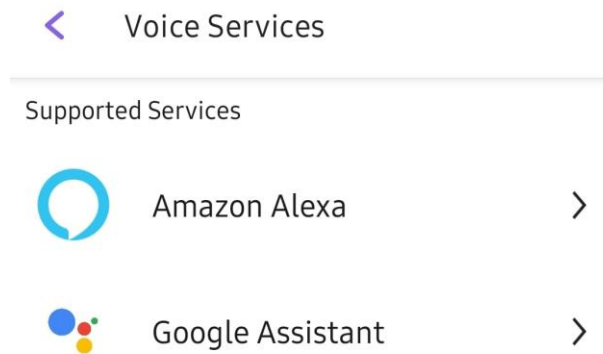


Ilustración 5.19 – Amazon Alexa

Seleccionamos “Link with Amazon Alexa”, se nos abrirá la aplicación para preguntarnos si deseamos activar la Skill de ESP RainMaker a lo que seleccionamos “Vincular”. *Ilustración 5.20.*

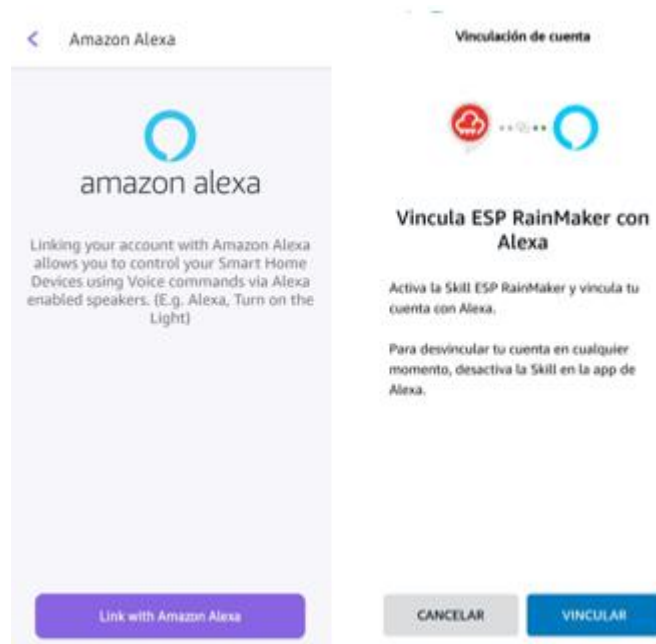


Ilustración 5.20 – Vinculación con Amazon Alexa

Cuando vinculemos la aplicación se nos añadirán automáticamente nuestros “Switches” como se muestra en la *Ilustración 5.21*.



Ilustración 5.21 – Dispositivos agregados correctamente

Nos dirigimos a la aplicación “Amazon Alexa”, podemos observar que nuestros dispositivos se encuentran disponibles, ahora es posible manipularlos desde el interruptor virtual al igual que mediante la voz, como por ejemplo diciendo “Alexa, enciende Switch1”. *Ilustración 5.22.*

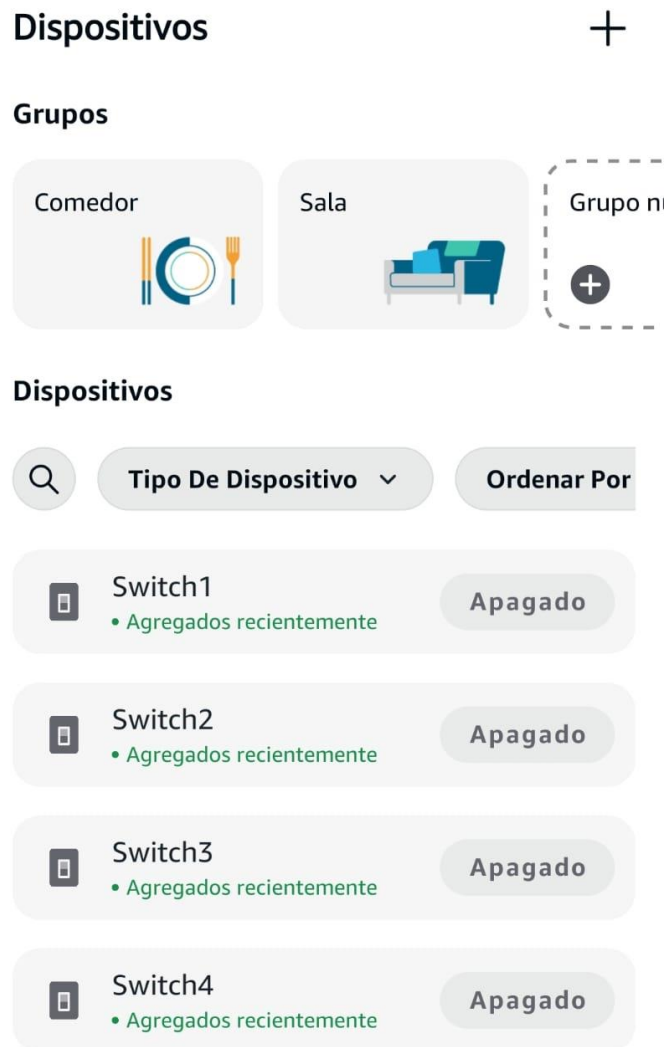
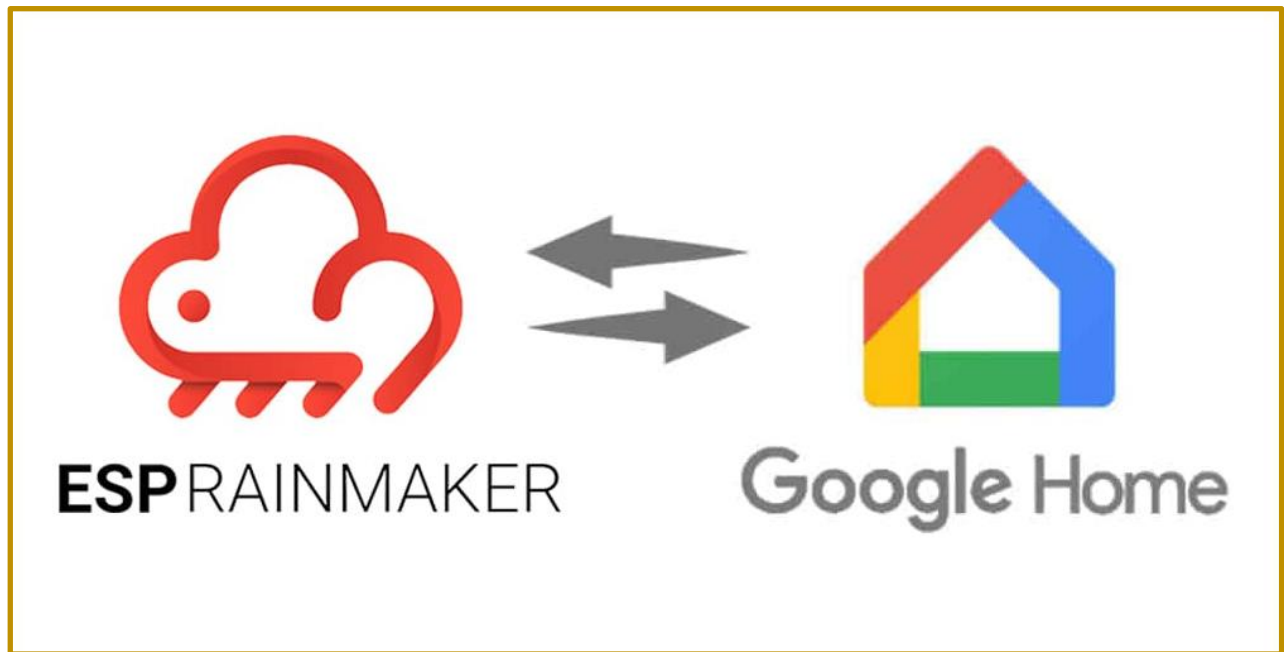


Ilustración 5.22 – Dispositivos disponibles

CONCLUSIONES

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

PRÁCTICA VI – ESP RAINMAKER CON GOOGLE HOME



OBJETIVO DE LA UNIDAD:

Conocerá los estándares de comunicación más utilizados en sistemas de IoT para implementar una aplicación funcional.

OBJETIVOS DE LA PRACTICA:

Integrar dispositivos IoT con servicios en la nube mediante la utilización de ESP RainMaker, y conectarlos a la aplicación 'Google Home' para habilitar el control remoto de dichos dispositivos.

ANTECEDENTES

ESP RainMaker es una plataforma desarrollada por Espressif Systems para facilitar el desarrollo y la implementación de soluciones IoT. Espressif Systems, una compañía conocida por sus chips Wi-Fi y Bluetooth, como el ESP8266 y el ESP32, ha sido un actor clave en el mercado de IoT, proporcionando hardware accesible y fácil de usar para desarrolladores y aficionados.



Ilustración 6.1 – ESP RainMaker

Características clave:

- **Configuración Simplificada:** ESP RainMaker simplifica el proceso de configuración de dispositivos IoT, eliminando muchas de las complejidades asociadas con la configuración de red y la conexión a la nube.
- **Control Remoto:** Los dispositivos integrados con ESP RainMaker pueden ser controlados remotamente a través de la aplicación ESP RainMaker, Google Home y Amazon Alexa.
- **Seguridad y Privacidad:** Espressif ha puesto un fuerte énfasis en la seguridad, asegurando que las comunicaciones entre los dispositivos y la nube estén encriptadas y sean seguras.
- **Actualizaciones OTA (Over-The-Air):** Permite la actualización de firmware de los dispositivos de manera remota, facilitando el mantenimiento y la implementación de nuevas funcionalidades sin necesidad de intervención física.

MATERIAL A UTILIZAR

- NodeMCU-32
- 2 LEDs de cualquier color
- Jumpers Macho-Macho
- 2 Resistencias de 330Ω
- Protoboard

SOFTWARE NECESARIO

ESP RainMaker:

<https://play.google.com/store/apps/details?id=com.espressif.rainmaker&hl=es&gl=US>

Google Home:

https://play.google.com/store/apps/details?id=com.google.android.apps.chromecast.app&hl=es_MX

Práctica 6. Encendido de LEDs a través de Google Home mediante ESP RainMaker.

6.1 Implementar el circuito del diagrama que se muestra en la *Ilustración 6.2*

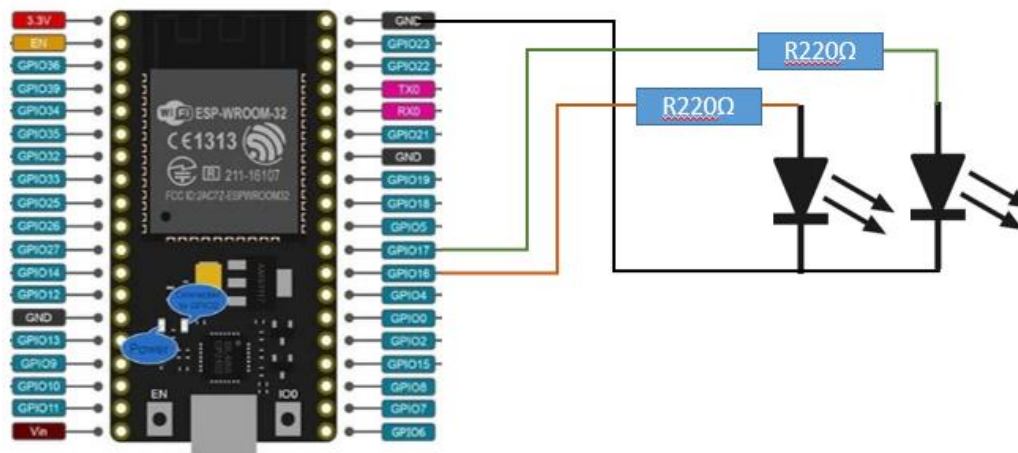


Ilustración 6.2 – Diagrama a implementar

6.2 Código para la Practica

```
/* *****  
 *   TITLE: ESP RainMaker + Manual Switch control 4 Relays using ESP32 (Real time  
feedback + no WiFi control)  
 *   Click on the following links to learn more.  
 *   YouTube Video: https://youtu.be/IMb52-h4tzQ  
 *   Related Blog : https://iotcircuithub.com/esp32-projects/  
 *   by Tech StudyCell  
 *   Preferences--> Additional boards Manager URLs :  
 *   http://arduino.esp8266.com/stable/package\_esp8266com\_index.json,https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package\_esp32\_index.json  
 *  
 *   Download Board ESP32 (2.0.3) :  
 *  
 *   Download the libraries :  
 *   N/A  
 * *****  
/  
  
#include "RMaker.h"  
#include "WiFi.h"  
#include "WiFiProv.h"  
  
const char *service_name = "JuanChi";  
const char *pop = "1234567";  
  
// define the Device Names  
char deviceName_1[] = "Switch1";  
char deviceName_2[] = "Switch2";  
char deviceName_3[] = "Switch3";  
char deviceName_4[] = "Switch4";  
  
// define the GPIO connected with Relays and switches  
static uint8_t RelayPin1 = 16; //D16  
static uint8_t RelayPin2 = 17; //D17  
static uint8_t RelayPin3 = 21; //D21  
static uint8_t RelayPin4 = 19; //D19  
  
static uint8_t SwitchPin1 = 26; //D26  
static uint8_t SwitchPin2 = 12; //D12  
static uint8_t SwitchPin3 = 14; //D14  
static uint8_t SwitchPin4 = 27; //D27  
  
static uint8_t wifiled = 2; //D2
```

```

static uint8_t gpio_reset = 0;

/* Variable for reading pin status*/
// Relay State
bool toggleState_1 = LOW; //Define integer to remember the toggle state for relay 1
bool toggleState_2 = LOW; //Define integer to remember the toggle state for relay 2
bool toggleState_3 = LOW; //Define integer to remember the toggle state for relay 3
bool toggleState_4 = LOW; //Define integer to remember the toggle state for relay 4

// Switch State
bool SwitchState_1 = LOW;
bool SwitchState_2 = LOW;
bool SwitchState_3 = LOW;
bool SwitchState_4 = LOW;

//The framework provides some standard device types like switch, lightbulb, fan,
temperature sensor.
static Switch my_switch1(deviceName_1, &RelayPin1);
static Switch my_switch2(deviceName_2, &RelayPin2);
static Switch my_switch3(deviceName_3, &RelayPin3);
static Switch my_switch4(deviceName_4, &RelayPin4);

void sysProvEvent(arduino_event_t *sys_event)
{
    switch (sys_event->event_id) {
        case ARDUINO_EVENT_PROV_START:
#ifdef CONFIG_IDF_TARGET_ESP32
            Serial.printf("\nProvisioning Started with name \"%s\" and PoP \"%s\" on
BLE\n", service_name, pop);
            printQR(service_name, pop, "ble");
#else
            Serial.printf("\nProvisioning Started with name \"%s\" and PoP \"%s\" on
SoftAP\n", service_name, pop);
            printQR(service_name, pop, "softap");
#endif
            break;
        case ARDUINO_EVENT_WIFI_STA_CONNECTED:
            Serial.printf("\nConnected to Wi-Fi!\n");
            digitalWrite(wifiLed, true);
            break;
    }
}

void write_callback(Device *device, Param *param, const param_val_t val, void
*priv_data, write_ctx_t *ctx)

```

```

{
    const char *device_name = device->getDeviceName();
    const char *param_name = param->getParamName();

    if(strcmp(device_name, deviceName_1) == 0) {

        Serial.printf("Lightbulb = %s\n", val.val.b? "true" : "false");

        if(strcmp(param_name, "Power") == 0) {
            Serial.printf("Received value = %s for %s - %s\n", val.val.b? "true" :
"false", device_name, param_name);
            toggleState_1 = val.val.b;
            (toggleState_1 == false) ? digitalWrite(RelayPin1, HIGH) :
digitalWrite(RelayPin1, LOW);
            param->updateAndReport(val);
        }

    } else if(strcmp(device_name, deviceName_2) == 0) {

        Serial.printf("Switch value = %s\n", val.val.b? "true" : "false");

        if(strcmp(param_name, "Power") == 0) {
            Serial.printf("Received value = %s for %s - %s\n", val.val.b? "true" :
"false", device_name, param_name);
            toggleState_2 = val.val.b;
            (toggleState_2 == false) ? digitalWrite(RelayPin2, HIGH) :
digitalWrite(RelayPin2, LOW);
            param->updateAndReport(val);
        }

    } else if(strcmp(device_name, deviceName_3) == 0) {

        Serial.printf("Switch value = %s\n", val.val.b? "true" : "false");

        if(strcmp(param_name, "Power") == 0) {
            Serial.printf("Received value = %s for %s - %s\n", val.val.b? "true" :
"false", device_name, param_name);
            toggleState_3 = val.val.b;
            (toggleState_3 == false) ? digitalWrite(RelayPin3, HIGH) :
digitalWrite(RelayPin3, LOW);
            param->updateAndReport(val);
        }

    } else if(strcmp(device_name, deviceName_4) == 0) {

```

```

    Serial.printf("Switch value = %s\n", val.val.b? "true" : "false");

    if(strcmp(param_name, "Power") == 0) {
        Serial.printf("Received value = %s for %s - %s\n", val.val.b? "true" :
"false", device_name, param_name);
        toggleState_4 = val.val.b;
        (toggleState_4 == false) ? digitalWrite(RelayPin4, HIGH) :
digitalWrite(RelayPin4, LOW);
        param->updateAndReport(val);
    }
}

void manual_control()
{
    if (digitalRead(SwitchPin1) == LOW && SwitchState_1 == LOW) {
        digitalWrite(RelayPin1, LOW);
        toggleState_1 = 1;
        SwitchState_1 = HIGH;
        my_switch1.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, toggleState_1);
        Serial.println("Switch-1 on");
    }
    if (digitalRead(SwitchPin1) == HIGH && SwitchState_1 == HIGH) {
        digitalWrite(RelayPin1, HIGH);
        toggleState_1 = 0;
        SwitchState_1 = LOW;
        my_switch1.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, toggleState_1);
        Serial.println("Switch-1 off");
    }
    if (digitalRead(SwitchPin2) == LOW && SwitchState_2 == LOW) {
        digitalWrite(RelayPin2, LOW);
        toggleState_2 = 1;
        SwitchState_2 = HIGH;
        my_switch2.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, toggleState_2);
        Serial.println("Switch-2 on");
    }
    if (digitalRead(SwitchPin2) == HIGH && SwitchState_2 == HIGH) {
        digitalWrite(RelayPin2, HIGH);
        toggleState_2 = 0;
        SwitchState_2 = LOW;
        my_switch2.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, toggleState_2);
        Serial.println("Switch-2 off");
    }
    if (digitalRead(SwitchPin3) == LOW && SwitchState_3 == LOW) {
        digitalWrite(RelayPin3, LOW);

```

```

toggleState_3 = 1;
SwitchState_3 = HIGH;
my_switch3.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, toggleState_3);
Serial.println("Switch-3 on");
}
if (digitalRead(SwitchPin3) == HIGH && SwitchState_3 == HIGH) {
    digitalWrite(RelayPin3, HIGH);
    toggleState_3 = 0;
    SwitchState_3 = LOW;
    my_switch3.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, toggleState_3);
    Serial.println("Switch-3 off");
}
if (digitalRead(SwitchPin4) == LOW && SwitchState_4 == LOW) {
    digitalWrite(RelayPin4, LOW);
    toggleState_4 = 1;
    SwitchState_4 = HIGH;
    my_switch4.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, toggleState_4);
    Serial.println("Switch-4 on");
}
if (digitalRead(SwitchPin4) == HIGH && SwitchState_4 == HIGH) {
    digitalWrite(RelayPin4, HIGH);
    toggleState_4 = 0;
    SwitchState_4 = LOW;
    my_switch4.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, toggleState_4);
    Serial.println("Switch-4 off");
}
}

void setup()
{
    uint32_t chipId = 0;

    Serial.begin(115200);

    // Set the Relays GPIOs as output mode
    pinMode(RelayPin1, OUTPUT);
    pinMode(RelayPin2, OUTPUT);
    pinMode(RelayPin3, OUTPUT);
    pinMode(RelayPin4, OUTPUT);
    pinMode(wifiLed, OUTPUT);

    // Configure the input GPIOs
    pinMode(SwitchPin1, INPUT_PULLUP);
    pinMode(SwitchPin2, INPUT_PULLUP);
    pinMode(SwitchPin3, INPUT_PULLUP);

```

```

pinMode(SwitchPin4, INPUT_PULLUP);
pinMode(gpio_reset, INPUT);

// Write to the GPIOs the default state on booting
digitalWrite(RelayPin1, !toggleState_1);
digitalWrite(RelayPin2, !toggleState_2);
digitalWrite(RelayPin3, !toggleState_3);
digitalWrite(RelayPin4, !toggleState_4);
digitalWrite(wifiLed, LOW);

Node my_node;
my_node = RMaker.initNode("ESP32_Relay_4");

//Standard switch device
my_switch1.addCb(write_callback);
my_switch2.addCb(write_callback);
my_switch3.addCb(write_callback);
my_switch4.addCb(write_callback);

//Add switch device to the node
my_node.addDevice(my_switch1);
my_node.addDevice(my_switch2);
my_node.addDevice(my_switch3);
my_node.addDevice(my_switch4);

//This is optional
RMaker.enableOTA(OTA_USING_PARAMS);
//If you want to enable scheduling, set time zone for your region using
setTimeZone().
//The list of available values are provided here
https://rainmaker.espressif.com/docs/time-service.html
// RMaker.setTimeZone("Asia/Shanghai");
// Alternatively, enable the Timezone service and let the phone apps set the
appropriate timezone
RMaker.enableTZService();
RMaker.enableSchedule();

//Service Name
for(int i=0; i<17; i=i+8) {
    chipId |= ((ESP.getEfuseMac() >> (40 - i)) & 0xff) << i;
}

Serial.printf("\nChip ID:  %d Service Name: %s\n", chipId, service_name);

Serial.printf("\nStarting ESP-RainMaker\n");

```

```

    RMaker.start();

    WiFi.onEvent(sysProvEvent);
#ifdef CONFIG_IDF_TARGET_ESP32
    WiFiProv.beginProvision(WIFI_PROV_SCHEME_BLE,
WIFI_PROV_SCHEME_HANDLER_FREE_BTDM, WIFI_PROV_SECURITY_1, pop, service_name);
#else
    WiFiProv.beginProvision(WIFI_PROV_SCHEME_SOFTAP, WIFI_PROV_SCHEME_HANDLER_NONE,
WIFI_PROV_SECURITY_1, pop, service_name);
#endif

    my_switch1.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, false);
    my_switch2.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, false);
    my_switch3.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, false);
    my_switch4.updateAndReportParam(ESP_RMAKER_DEF_POWER_NAME, false);
}

void loop()
{
    // Read GPIO00 (external button to reset device
    if(digitalRead(gpio_reset) == LOW) { //Push button pressed
        Serial.printf("Reset Button Pressed!\n");
        // Key debounce handling
        delay(100);
        int startTime = millis();
        while(digitalRead(gpio_reset) == LOW) delay(50);
        int endTime = millis();

        if ((endTime - startTime) > 10000) {
            // If key pressed for more than 10secs, reset all
            Serial.printf("Reset to factory.\n");
            RMakerFactoryReset(2);
        } else if ((endTime - startTime) > 3000) {
            Serial.printf("Reset Wi-Fi.\n");
            // If key pressed for more than 3secs, but less than 10, reset Wi-Fi
            RMakerWiFiReset(2);
        }
    }
    delay(100);

    if (WiFi.status() != WL_CONNECTED)
    {
        //Serial.println("WiFi Not Connected");
        digitalWrite(wifiLed, false);
    }
}

```

```
else
{
    //Serial.println("WiFi Connected");
    digitalWrite(wifiLed, true);
}

manual_control();
}
```

6.3 Desarrollo de la practica

Los pasos necesarios para vincular la tarjeta con ESP RainMaker y conectarla a internet se ejecutaron en la práctica anterior. Nos dirigimos a la pantalla principal de ESP RainMaker, seleccionamos la pestaña “Settings” para luego seleccionar “Voice Services”. Una vez en este apartado escogemos “Google Assistant”. *Ilustración 6.3.*

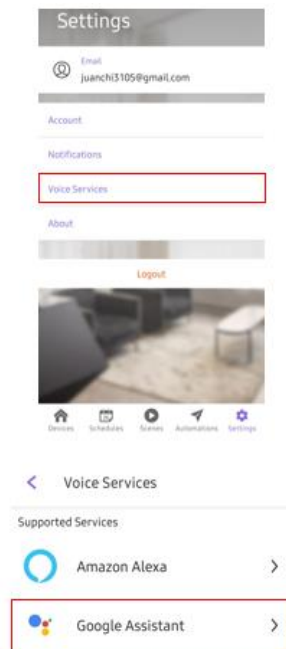


Ilustración 6.3 – Seleccionar “Google Assistant”

Se nos indicará los pasos que debemos seguir para vincular correctamente “Google Assistant”. *Ilustración 6.4.*

Habilitación del Asistente de voz de Google (GVA)

- Abre la aplicación Google Home en tu teléfono.
- Toque "+" -> Configurar dispositivo.
- Seleccione la opción "Funciona con Google" destinada a dispositivos ya configurados.
- Busque ESP RainMaker e inicie sesión con sus credenciales de RainMaker.
- Una vez que la vinculación de la cuenta sea exitosa, sus dispositivos RainMaker aparecerán y podrá comenzar a usarlos.

Ilustración 6.4 – Pasos a seguir

Iniciamos dirigiéndonos a la aplicación de “Google Home”. Nos dirigimos a “Config” para seguidamente seleccionar “Funciona con Google”. *Ilustración 6.5.*

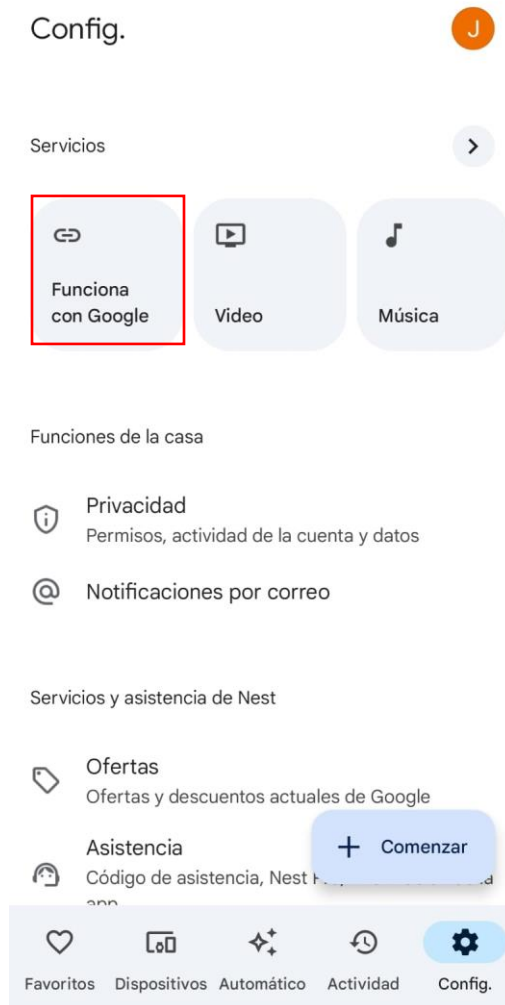


Ilustración 6.5 – Funciona con Google

Buscamos la aplicación “ESP RainMaker” una vez que la encontremos la seleccionamos y le damos click en “Continuar” para seguir con la vinculación. *Ilustración 6.6.*

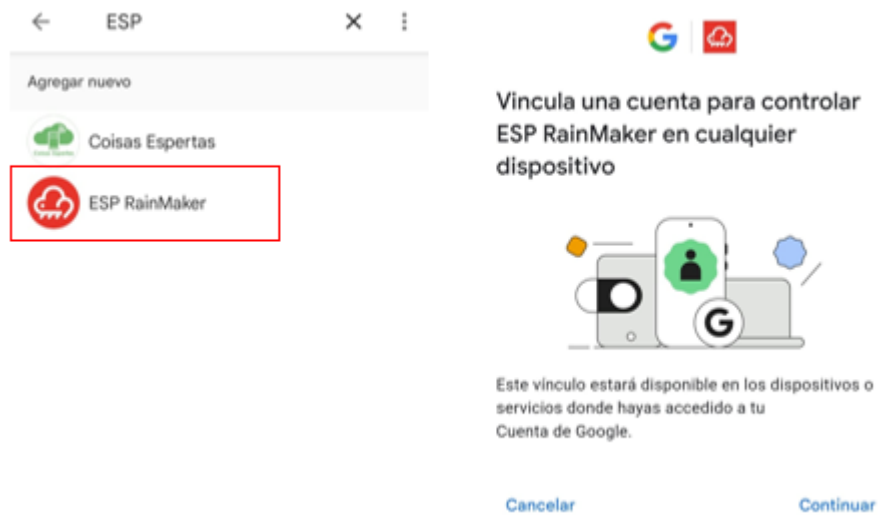


Ilustración 6.6 – Seleccionar ESP RainMaker

Se nos abrirá una ventana de inicio de sesión, por lo tanto, ingresamos nuestras credenciales. *Ilustración 6.7.*

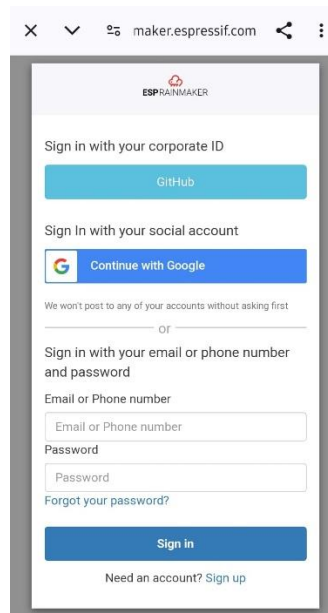


Ilustración 6.7 – Iniciar sesión

Una vez que la vinculación sea exitosa se nos añadirán nuestros “Switches” automáticamente. *Ilustración 6.8.*

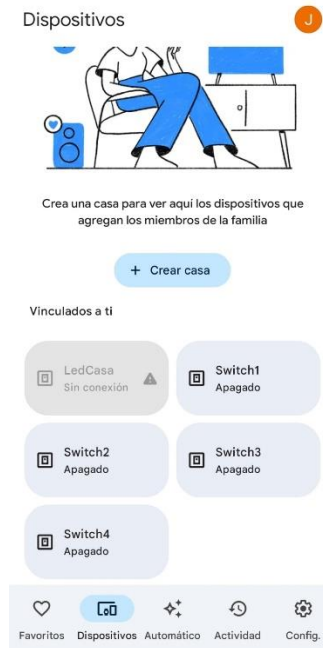


Ilustración 6.8 – Dispositivos añadidos con éxito

Ahora es posible manipular nuestros dispositivos desde el interruptor virtual al igual que mediante la voz, como por ejemplo diciendo “Hey Google, turn on Switch1”. *Ilustración 6.9.*



Ilustración 6.9 – Interruptor virtual

CONCLUSIONES

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins or other markings on the paper.

REFERENCIAS

- [1] Espressif Systems, «ESP32,» [En línea]. Available: <https://www.espressif.com/en/products/socs/esp32>. [Último acceso: 5 10 2023].

- [2] S. Guillermo, Internet de las cosas con ESP8266, ALFAOMEGA, 2021.

- [3] Sinric, «Sinric Pro,» [En línea]. Available: <https://sinric.pro/>. [Último acceso: 09 05 2024].

- [4] Espressif, «ESP RainMaker,» [En línea]. Available: <https://rainmaker.espressif.com/>. [Último acceso: 26 05 2024].

- [5] Arduino, «Arduino Cloud,» [En línea]. Available: <https://cloud.arduino.cc/>. [Último acceso: 23 03 2024].